

Nº
01

DOCUMENTATION

Osionos

Dossier de Projet

Développeur Web et Web Mobile (DWWM)



AUTHOR

Dylan Lesieur

DATE

June 22, 2026

EST · MMXXVI

Table des matières

01 Vue d'ensemble

02 Usage de l'IA

2.1 Les différents profils d'usage

03 Ma philosophie : apprendre avec l'IA sans sacrifier la compréhension

04 Remerciements

05 CHAPITRE 1 : synthèse des compétences mobilisées

5.1 Activité-type 1 : développer la partie front-end d'une application web sécurisée

5.2 Activité-type 2 : développer la partie back-end d'une application web sécurisée

5.3 Compétences transverses

06 CHAPTITRE 2: Présentation du projet

6.1 Présentation de l'entreprise et du service

6.2 Cahier des charges du projet:

6.3 les contraintes

6.4 Environnement humain et technique

6.5 Objectifs de qualité

07 CHAPITRE 3: Les réalisations personnelles, front-end

7.1 Maquette de l'application et schémas

7.2 Captures d'écran des interfaces utilisateur

7.3 Optimisations front-end et résultats mesurés

7.4 Extraits de code, interfaces utilisateur statiques (React / SCSS)

7.5 Extraits de code, partie dynamique

7.6 Accessibilité, sécurité front et qualité mesurable

08 CHAPITRE 4. Les réalisations personnelles, back-end

8.1 Architecture de l'API et modèle de données

8.2 Extrait de code, structure et sécurité de l'API

8.3 Extraits de code de composants d'accès aux données (DAO)

09 CHAPITRE 5. Eléments de sécurité de l'application

9.1 Sécurisation du transport : HTTPS / TLS (la liaison navigateur ↔ back-end)

9.2 Authentification et gestion des rôles

- 9.3 Validation des entrées et protection contre les injections
 - 9.4 Protections front-end et API
 - 9.5 Protections contre XSS et CSRF
 - 9.6 Conformité RGPD
 - 9.7 Déploiement de production et sécurité de bout en bout (fly.io · Vercel · 42ctl/vault42)
-

10 CHAPITRE 6. Veille technologique et sécurité

- 10.1 Sources de veille utilisées
 - 10.2 Vulnérabilités identifiées dans l'écosystème
 - 10.3 Failles potentielles et corrections à apporter
 - 10.4 Conclusion
-

11 CHAPITRE 7. Conclusion

12 Ressources et références

13 8.1 — Les trois langages : qui fait quoi, et pourquoi ?

- 13.1 Q. Pourquoi trois langages (Go, Rust, TypeScript) ? Ce ne serait pas plus simple avec un seul ?
 - 13.2 Q. C'est quoi exactement la « couture » entre Go et Rust dont tout le monde parle ?
 - 13.3 Q. Pourquoi Go pour le plan de contrôle ?
 - 13.4 Q. Pourquoi Rust pour le plan de données ?
 - 13.5 Q. Pourquoi TypeScript/NestJS pour le plan applicatif ?
-

14 8.2 — Docker : pourquoi construire nos propres images plutôt que tirer des images « grasses » ?

- 14.1 Q. C'est quoi une image « grasse » et pourquoi vous les évitez ?
 - 14.2 Q. Montre-moi à quoi ressemble une de ces images.
 - 14.3 Q. Le vocabulaire, en clair ?
 - 14.4 Q. Et le coût de ce choix (le trade-off honnête) ?
-

15 8.3 — Le réseau Docker : comment les services se parlent

- 15.1 Q. Comment un service en trouve un autre ? Avec des adresses IP en dur ?
 - 15.2 Q. Qui est exposé à internet ?
 - 15.3 Q. Et Kong, il fait quoi ?
 - 15.4 Q. Peut-on cloisonner encore plus ?
-

16 8.4 — Les volumes : où vivent vraiment les données

- 16.1 Q. Si je détruis un conteneur, je perds les données ?
 - 16.2 Q. Comment garantir que les services démarrent dans le bon ordre ?
-

17 8.5 — Comment on « communique avec les images » Docker (au quotidien)

- 17.1 Q. En pratique, quelles commandes pour piloter les conteneurs ?

17.2 Q. Piège classique : j'ai modifié un service, mais mes changements n'apparaissent pas ?

18 8.6 — L'efficacité des ressources : le classique trade-off d'un gros projet

18.1 Q. Un gros projet, c'est forcément lourd. Comment vous gardez ça léger ?

18.2 Q. Et la mémoire qui explose avec le nombre de clients ?

18.3 Q. Combien de requêtes ça encaisse ? (capacité mesurée, pas annoncée)

19 8.7 — Est-ce que c'est scalable ?

19.1 Q. Le projet passe-t-il à l'échelle ?

20 8.8 — Brancher un nouveau frontend demain : c'est facile ? rapide ?

20.1 Q. Si demain je démarre un front tout neuf, comment je me branche ?

20.2 Q. Le navigateur parle-t-il directement à la base ? (le fameux « gap » DB ↔ frontend)

20.3 Q. Et le temps réel (WebSocket) ?

20.4 Q. Et si j'ai déjà ma propre base ?

21 8.9 — À quoi servent les répertoires SDK (chacun) ?

21.1 Q. Pourquoi cinq dossiers sdks/* ?

21.2 Q. Comment est organisé le SDK TypeScript à l'intérieur ?

22 8.10 — Comment l'app garantit des transactions de données sécurisées (TOUTES les mesures)

22.1 (a) Le périmètre — WAF, puis Kong

22.2 (b) L'authentification — clés d'API jamais en clair

22.3 (c) L'isolation multi-tenant — par requête, pas par pool

22.4 (d) ABAC — décisions fines au-dessus des rôles

22.5 (e) Les injections — structurellement impossibles côté données

22.6 (f) Le chiffrement au repos — credentials & CMEK

22.7 (g) L'audit inviolable — chaîne de hash

22.8 (h) Transport, secrets & en-têtes

22.9 (i) RGPD — consentement, effacement, portabilité

22.10 (j) Sauvegarde / restauration

22.11 En une phrase

23 8.11 — Les tests : chaque type, comment les lancer, et ce qu'on voit dans le terminal

23.1 Q. Comment prouve-t-on que ça marche, concrètement ?

23.2 1. Tests unitaires — la logique pure, sans réseau ni base de données

23.3 2. Tests d'intégration / smoke — la vraie stack, en boîte noire (make test-smoke)

23.4 3. Preuve « stack live » rejouable — Newman / Postman

23.5 4. Conformité moteur — make conformance (le test « engine-agnostic »)

23.6 5. Parité shadow TS ↔ Rust — make parity NEW=<url>

23.7 6. Les gates de jalon — l'unité de « fini » (148 scripts mNN-*.sh)

23.8 7. Benchmarks — la perf est chiffrée, pas adjectivée (make bench-*)

23.9 8. Sécurité & chaîne d'approvisionnement — make audit-deps + scanners

23.10 9. La CI — tout cela tourne tout seul (GitHub Actions)

23.11 En une phrase

24 8.12 — HTTPS / TLS : comment la liaison navigateur ↔ back-end est-elle réellement chiffrée ?

24.1 Q. Concrètement, qu'est-ce que le HTTPS apporte, et où est-il en place ?

24.2 Q. Certificat, autorité de certification, terminaison TLS, production : où est tout ça ?

25 8.13 — Authentification vs autorisation : quelle différence, et où vivent-elles ?

25.1 Q. Le jury demande souvent la nuance — comment la fais-tu dans le code ?

26 8.14 — Pourquoi NestJS plutôt qu'Express ?

26.1 Q. Express aurait suffi, non ? Pourquoi avoir pris NestJS ?

27 8.15 — Que se passe-t-il si une couche — ou un service — tombe ?

27.1 Q. Et la résilience ? Si le WAF, Kong, ou un service interne lâche ?

28 8.16 — Comment gères-tu l'évolution du schéma de base de données ?

28.1 Q. Les migrations : comment ça marche, et pourquoi c'est important ?

29 8.17 — Comment les trois plans (TypeScript, Go, Rust) communiquent-ils ?

29.1 Q. Concrètement, par quoi passent les données entre les trois langages ?

29.2 Q. C'est quoi un « endpoint API » ici, et qui appelle qui ?

29.3 Q. Et l'identité / la confiance dans ces appels internes ?

29.4 Q. Et l'asynchrone (les « promises ») dans tout ça ?

30 8.18 — Les « injectables » NestJS : c'est quoi, et comment ça marche ?

30.1 Q. Tu parles d'injectables et d'injection de dépendances — explique simplement.

30.2 Q. Pourquoi c'est utile, concrètement ?

31 8.19 — Les « guardrails » de chaque langage : qu'est-ce qui me protège en TS, Go, Rust ?

31.1 Q. Pourquoi ces trois langages précisément ? Qu'apporte chacun en garde-fous ?

32 8.20 — Comment un binaire (Go, Rust) « fait du web » ? Et WebAssembly là-dedans ?

32.1 Q. Un programme compilé, ça sert des requêtes HTTP comment ?

32.2 Q. Et WebAssembly (WASM) ? Vous l'utilisez ?

33 8.21 — Docker : comment chaque langage est conteneurisé, et par quel protocole les services se parlent

33.1 Q. Chaque service (TS, Go, Rust) tourne comment, et comment se trouvent-ils ?

33.2 Q. Ils communiquent en HTTP, ou par d'autres protocoles ?

34 8.22 — OLTP vs OLAP : la différence, et comment je « vois » dans l'OLTP

34.1 Q. C'est quoi la différence, et nous, on est quoi ?

34.2 Q. Comment « voir » dans l'OLTP — inspecter et observer la base ?

35 8.23 — Comment être sûr que toutes les connexions aux bases sont fiables ?

35.1 Q. Avec 8 moteurs et du multi-tenant, comment garantir la fiabilité des connexions ?

36 8.24 — Le front sait-il si c'est MongoDB, PostgreSQL ou MariaDB ? Et comment demande-t-on un CRUD ?

36.1 Q. Le front doit-il connaître le moteur derrière ? Est-ce normalisé ?

36.2 Q. Donc on ne code pas en dur les besoins par moteur ?

36.3 Q. Concrètement, comment demande-t-on un get / set / fetch / delete ?

i NOTE AU LECTEUR — AVANT DE COMMENCER

Deux noms apparaissent tout au long de ce dossier, et c'est important de pas les confondre : - **Osionos** — c'est le nom de **l'application**, la plateforme principale (équivalent Notion-like, là où on travaille avec les pages, blocs, bases de données, dashboards). - **Prismatica** — c'est le nom de **la page web publique** : le site marketing / vitrine qui présente Osionos au monde extérieur. C'est l'enveloppe, pas le moteur.

En résumé : si on parle de l'app, c'est **Osionos** ; si on parle du site, c'est **Prismatica**.

Pour les visuels : les captures d'écran et diagrammes intégrés dans ce PDF sont en résolution réduite pour des questions de poids. Les **versions haute définition** ainsi que les images sources sont disponibles dans le repository GitHub : [Univers42/ft_transcendence](#) sous [wiki/assets/](#) . Les liens cliquables dans le PDF renvoient directement vers ces fichiers.

CONVENTION DE CHEMINS — DÉPÔT `GROBASE` AUTONOME

Ce dossier a d'abord été rédigé dans le monorepo *Track-Binocle* ; le back-end vit désormais dans le dépôt autonome **grobase** , à l'arborescence aplatie. Concrètement : - le préfixe historique [apps/baas/mini-baas-infra/...](#) n'existe plus : les chemins sont désormais à la racine du dépôt — [src/...](#) (planes applicatif/contrôle/données), [infra/...](#) (Docker, config, OpenAPI, Postman), [orchestrators/...](#) (compose, makes), [scripts/...](#) et [wiki/...](#) ; - les liens vers [apps/osionos/...](#) et [apps/opposite-osiris/...](#) désignent les **front-ends du monorepo** (l'app Osionos et le site Prismatica) — ils ne font **pas** partie de ce dépôt back-end, ils sont conservés pour illustrer comment un client réel consomme le BaaS.

Osionos a été pensé à l'image d'une fourmilière : un écosystème organisé qui exploite les ressources de son environnement et s'adapte en permanence. Les fourmis transportent, communiquent, construisent des galeries interconnectées où circulation et stockage restent fluides. Avec les collègues, on a fait le même constat : une application moderne accumule vite des données et des services à maintenir. Osionos cherche à rendre cette complexité lisible — un seul back-end mutualisé que plusieurs front-ends peuvent consommer, plutôt qu'un serveur réécrit pour chaque projet.

Linus Torvalds, créateur de Linux, avait besoin d'un outil de gestion de version pour piloter son propre projet — c'est ainsi que Git est né. De la même manière, nous avons voulu créer un side project capable d'accompagner nos futurs projets.

Vue d'ensemble

Le projet Osionos est né d'une frustration : on cherchait un outil de dashboarding complet, rapide et agréable à utiliser — un truc genre Notion, mais mieux adapté à nos besoins. Outil après outil, nous nous heurtions aux mêmes limitations : manque de personnalisation, performances insuffisantes, intégrations trop rigides. On a décidé de créer notre propre solution. Nous sommes pleinement conscients que c'est un projet long terme.

Usage de l'IA

L'IA est aujourd'hui un élément incontournable dans les projets modernes, et Osionos ne fait pas exception. Nous l'avons intégrée de manière stratégique pour améliorer l'expérience développeur et enrichir notre apprentissage : génération de code, débogage, compréhension de problèmes complexes, documentation normalisée, et analyse critique de l'avancement du projet.

Les différents profils d'usage

Avant d'entrer dans le détail, voici les différents profils d'usage de l'IA que nous avons identifiés et expérimentés.

Profil	Origine	Description
Vibe coder	Andrej Karpathy, 2025	Délègue entièrement à l'IA, sans lire l'output — il “vibe” avec le résultat
AI-augmented developer	GitHub / Stack Overflow surveys	Utilise l'IA comme couche de productivité tout en gardant la maîtrise et la compréhension du code
Prompt engineer	Écosystème OpenAI	Spécialiste de la formulation de prompts précis pour obtenir des outputs de qualité — une discipline à part entière
Agentic developer / AI orchestrator	Émergent (2024-2025)	Conçoit et supervise des pipelines d'agents IA autonomes multi-étapes ; pense en workflows, pas en complétions individuelles
LLM engineer	Communauté ML	Construit au-dessus des LLM (fine-tuning, RAG, evals, inférence) — distinct de l'usage d'un LLM pour écrire du code applicatif
No-code / AI-native builder	Communauté produit	Assemble des applications entièrement en langage naturel et outils visuels, sans code traditionnel (Replit Agent, Lovable, etc.)
Reviewer / Human-in-the-loop	Communauté DevSecOps	Traite chaque suggestion de l'IA comme une pull request non vérifiée — rien n'est mergé sans audit humain

Comme nous n'avions aucune idée de ce que nous faisons au départ — et que c'était la première fois que nous devions mener un projet aussi personnalisé — la route a été complexe. Nous avons donc tout testé. Voici nos conclusions.

VIBE CODING

L'un des pièges dans lequel sont tombés des millions de développeurs juniors est ce que j'appellerais le “vibe coding non intentionnel”. La communauté développeur le voit d'un mauvais œil, et à raison : c'est genre posséder une Tesla, activer le pilote auto et regarder passivement. Jusqu'au moment où quelque chose se passe mal — et là, le temps de réaction est trop lent.

Notre équipe de cinq s'est prêtée à l'expérience pour en comprendre les limites. Voici ce que nous avons constaté :

- 1. Rapidité sans direction.** Le vibe coding génère du code vite, mais sans cap réel. L'output est souvent de mauvaise qualité, le refactoring est inévitable, et la dette technique s'accumule rapidement. Un projet de cette envergure ne peut pas tenir sur cette base.
- 2. Des cas d'usage valables malgré tout.** Pour esquisser des idées d'architecture ou amorcer une réflexion, ça peut être utile. Nous l'avons utilisé pour explorer plusieurs pistes de conception — les résultats n'étaient pas convaincants, mais ça a permis de débayer le terrain rapidement.
- 3. Même les grands professionnels l'utilisent ponctuellement.** Le professeur David J. Malan de Harvard l'a mentionné dans une interview, notamment pour la génération de tests unitaires. Preuve que cet outil a sa place, dans un cadre délimité.

Le vibe coding est un outil, pas un état d'esprit permanent. Bien utilisé — sur des tâches légères et ciblées — il peut faire gagner des heures. Le piège est de l'appliquer là où la rigueur est indispensable.

PROMPT ENGINEERING

Nous avons également testé le prompt engineering pour générer du code de meilleure qualité. Nous avons suivi les recommandations de la communauté : few-shot learning, chaînes de raisonnement, structuration précise des consignes. Résultat : c'est un outil efficace, mais qui demande du temps pour être maîtrisé. La qualité de l'output est directement corrélée à la qualité du prompt.

Quelques ressources qui nous ont été utiles :

- <https://www.ibm.com/fr-fr/think/prompt-engineering>
- <https://www.ibm.com/fr-fr/think/topics/prompt-optimization>
- <https://www.promptingguide.ai/fr>

AI ASSISTED / STYLE COPILOT

GitHub Copilot est l'exemple emblématique de l'assistance à la programmation : suggestions en temps réel, intégration dans l'éditeur, utile pour les tâches répétitives. Mais il ne remplace pas la compréhension du code — chaque suggestion doit être lue et validée.

On a exploré différents modèles selon les contextes. Le site [Artificial Analysis](#) publie des benchmarks quotidiens sur les principaux modèles — une référence utile pour choisir le bon outil selon les besoins du moment. Les modèles diffèrent en vitesse, en coût et en qualité du code produit : y a pas de modèle parfait — c'est contexte par contexte.

DEVELOPER AGENTIC

Le developer agentic — ou AI orchestrator — est un profil qui va au-delà des assistants classiques. Il conçoit et supervise des pipelines d'agents IA autonomes : une IA génère du code, une autre en vérifie la qualité, une troisième le déploie si les tests passent. On ne pense plus en complétions individuelles, mais en workflows.

C'est un profil exigeant, mais on a dû nous en approcher lorsque l'équipe s'est réduite à deux personnes. On a mis en place des agents spécialisés — un pour tester, un pour résoudre les problèmes identifiés — sous supervision constante.

Andrej Karpathy décrit bien ce changement de paradigme : *“on n'écrit plus du code directement 99% du temps. On orchestre des agents IA pour faire le travail, et on se concentre sur la supervision et l'optimisation de ces pipelines.”*

Limite principale : le coût en crédits IA est élevé, ce qui a rapidement freiné notre utilisation à grande échelle.

NO-CODE / AI-NATIVE BUILDER

Des outils comme Lovable, Replit Agent ou Figma AI permettent de construire des applications entièrement en langage naturel, sans code traditionnel. Nous les avons testés pendant plusieurs semaines. Ils prototypent vite, mais leurs limites sont bien réelles :

- Peu flexibles sur les projets complexes
- Code généré souvent non maintenable
- Dette technique élevée à long terme
- Lents sur les projets de grande envergure

- Coûteux en crédits IA

REVIEWER / HUMAN-IN-THE-LOOP

Ce profil traite chaque suggestion de l'IA comme une pull request non vérifiée : rien n'est intégré sans audit humain. C'est une approche prudente et efficace pour maintenir la qualité du code tout en profitant de l'assistance de l'IA.

Des expériences dans des communautés comme GitHub ont montré les limites de l'automatisation complète : les suggestions pouvaient être hors sujet, trop génériques, ou saturer les PR — générant de la friction et de la dette technique plutôt que du gain.

Ma philosophie : apprendre avec l'IA sans sacrifier la compréhension

Ce projet est né dans un contexte particulier. Je fais partie d'une génération qui n'a pas eu le choix de se confronter à l'IA — elle s'est imposée dans mes pratiques, dans mes outils, dans le marché du travail. L'ignorer aurait été se mettre délibérément en retard.

Mais j'ai voulu être honnête avec moi-même sur un point essentiel : **utiliser l'IA ne signifie pas comprendre moins**. Ce projet en est, je l'espère, la démonstration.

On a volontairement testé tous les profils d'usage décrits ci-dessus — non pas pour trouver la solution de facilité, mais pour comprendre concrètement ce que chacun apporte et ce qu'il coûte. Nous avons touché aux limites du vibe coding, mesuré les gains du prompt engineering, expérimenté l'orchestration d'agents. À chaque fois, nous avons lu ce que l'IA produisait, questionné ses choix, corrigé ses erreurs, et appris de ses approximations.

L'IA m'a souvent obligé à aller plus loin dans ma compréhension qu'un simple cours ne l'aurait fait. Comprendre pourquoi un output est mauvais, c'est comprendre ce que le bon aurait dû être.

Je veux être honnête là-dessus parce que je sais que l'usage de l'IA dans les projets scolaires est un sujet sensible. Mon intention n'a jamais été de contourner l'apprentissage — c'était de l'aborder différemment, dans une époque où ces outils font déjà partie du quotidien professionnel. Pour moi, ce projet c'est autant un apprentissage du code qu'un apprentissage de la posture à adopter face à l'IA : curiosité, esprit critique, responsabilité.

Remerciements

Ce projet n'aurait pas existé sans les personnes qui l'ont porté, dans les moments difficiles comme dans les bons.

Un grand merci à mon équipe pour avoir tenu dans la durée, pour avoir accepté de tester des approches incertaines, et pour avoir continué d'apprendre même quand la route était longue. Chacun a apporté quelque chose d'essentiel — une idée, une solution, une présence dans les moments où on doutait.

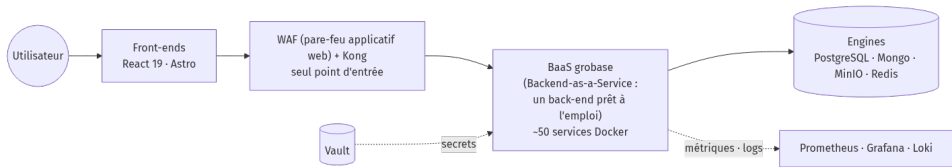
Merci à l'école 42, qui nous a appris que l'autonomie et la débrouillardise sont des compétences à part entière. Ce projet en est le reflet.

Merci aux communautés open source, aux auteurs de documentation, aux développeurs qui partagent leurs retours d'expérience en ligne — vous êtes une ressource invisible mais indispensable.

Et enfin, merci à ceux qui liront ce document avec l’œil ouvert et la curiosité de comprendre ce qu’on a cherché à faire ici.

CHAPITRE 1 : synthèse des compétences mobilisées

Osionos est un workspace collaboratif de type Notion (pages, blocs, bases de données, agents) qui s’appuie sur un écosystème de services internes assemblés en parallèle. La vue minimale ci-dessous suffit pour situer les compétences mobilisées dans ce chapitre ; le diagramme complet, plan par plan, est donné au [chapitre 2 « Vue d’ensemble des connexions entre services »](#).



Les compétences mobilisées s’inscrivent dans le référentiel CDA — *Concepteur Développeur d’Applications* — sur les deux activités-types front et back. Le « back » ici n’est **pas** une API Express classique : c’est une infrastructure assemblée à partir de briques open source éprouvées (PostgREST, GoTrue, Kong, Vault), que nous avons configurées, sécurisées et orchestrées avec Docker Compose. La justification détaillée de chaque choix est au chapitre 2.

Activité-type 1 : développer la partie front-end d’une application web sécurisée

Côté front, l’enjeu n’était pas d’écrire le plus de lignes de React possible, mais de tenir une promesse simple : **un utilisateur ouvre Osionos, et tout répond instantanément, même sur une page qui contient un millier de blocs**. Tout part de là — le choix du framework, l’organisation du code, l’accessibilité, jusqu’aux tokens SCSS. Ce que nous avons fait, et avec quoi :

Compétence CDA	Ce que ça veut dire chez nous	Outils / preuves dans le repo
Maquetter une interface	Penser desktop d’abord (Osionos est un outil de travail dense, pas un feed mobile), traiter l’accessibilité comme une contrainte de design et pas un audit final	Wireframes Figma, design tokens SCSS <code>_brand-tokens.scss</code> , <code><dialog></code> natif avec focus trap, régions <code>aria-live</code> , contraste vérifié
Intégrer des interfaces statiques	Deux frontends, deux outils choisis pour leur job réel : Astro pour le marketing (HTML statique, SEO), React pour l’app (interactivité dense)	<code>apps/opposite-osiris/</code> en Astro 6 + SCSS modulaire ; <code>apps/osionos/</code> en React 19 + Vite + organisation Feature-Sliced Design
Développer la partie dynamique	Stores granulaires sans cérémonie Redux, formulaires validés avant tout aller-retour réseau, virtualisation des longues listes	Zustand 5 (<code>usePageStore</code> , <code>useDatabaseStore</code>), <code>@tanstack/react-virtual</code> , SDK <code>@grobase/js</code> , flux GoTrue (email/mot de passe + magic link, un lien de connexion à usage unique reçu par mail + WebAuthn, la connexion sans mot de passe par empreinte/clé physique, via <code>@simplewebauthn/browser</code>)

Compétence CDA	Ce que ça veut dire chez nous	Outils / preuves dans le repo
Sécuriser le front	Chaque surface HTML est traitée selon son contexte : <code>sanitize-html</code> côté marketing, échappement HTML + <code>sanitizeUrl()</code> dans le moteur Markdown de l'app, scripts dédiés pour SVG, médias et CSP (détail du modèle de sécurité au chapitre 5)	<code>sanitize-html</code> , <code>svg-security.mjs</code> , <code>media-security.mjs</code> , <code>verify-csp.mjs</code> , <code>markengine</code>

Le fil rouge : chaque comportement non trivial (validation des formulaires, virtualisation des longues listes, appels au SDK) est isolé dans un fichier identifiable et couvert par les tests, plutôt que dilué dans les composants.

Activité-type 2 : développer la partie back-end d'une application web sécurisée

Côté back, le pari assumé est de **ne pas réécrire ce qui existe déjà** : la communauté open source a produit des briques (PostgREST, qui génère une API REST depuis la base ; GoTrue, qui gère les comptes et la connexion ; Kong, la passerelle qui filtre et route toutes les requêtes entrantes ; Vault, le coffre-fort à secrets) plus sûres et plus rapides que ce qu'on aurait pu produire en quelques mois. Notre travail a été de les **assembler, durcir, orchestrer**, et de combler les trous avec une poignée de micro-services NestJS sur mesure.

Compétence CDA	Ce que ça veut dire chez nous	Outils / preuves dans le repo
Modéliser et gérer la base de données	Schéma PostgreSQL avec contraintes, index et RLS (Row-Level Security) ; MongoDB pour le semi-structuré, chaque document marqué d'un <code>owner_id</code> pour cloisonner les données. Mécanisme détaillé au chapitre 4 , propriété de sécurité au chapitre 5	<code>001_initial_schema.sql</code> , <code>016_unify_rls.sql</code> , <code>065_least_privilege_rls.sql</code> , service <code>mongo-api</code>
Développer les composants d'accès aux données	Pas d'ORM (<i>Object-Relational Mapping</i>) : PostgREST génère l'API REST directement depuis le schéma → zéro code de liaison à maintenir, zéro injection SQL ; pour Mongo, façade NestJS dédiée. L' <code>adapter-registry</code> chiffre les identifiants de connexion externes (AES-256-GCM + clé dérivée par scrypt) — détail au chapitre 4	<code>postgrest</code> 12.2.3, <code>mongo-api</code> (NestJS), <code>adapter-registry</code>

Compétence CDA	Ce que ça veut dire chez nous	Outils / preuves dans le repo
Développer les composants métier	La logique vit là où c'est le plus sûr : autorisation/propriété dans la base (RLS + PL/pgSQL), coordination événementielle dans des services dédiés	Politiques RLS PG, <code>email-service</code> (relais SMTP NestJS générique via nodemailer ; les mails d'inscription/réinitialisation sont gérés par GoTrue), <code>realtime-agnostic</code> (WebSocket Rust), <code>storage-router</code> (URLs présignées MinIO : un lien temporaire et signé qui autorise un seul upload/download sans exposer les identifiants du stockage)
Sécuriser la stack	Défense en profondeur : WAF en amont de Kong, secrets jamais dans Git (Vault), certificats locaux proches prod, audit en cours d'extension — modèle complet au chapitre 5	WAF nginx + ModSecurity + OWASP CRS, HashiCorp Vault + <code>vault-env.mjs</code> , <code>generate-localhost-cert.sh</code> , <code>trust-localhost-cert.sh</code>
Déployer et documenter	Une commande <code>make</code> doit suffire à tout monter, qu'on soit un nouvel arrivant ou la CI ; chaque décision a une note écrite	Docker Compose + profils (<code>control-plane</code> , <code>data-plane</code> , <code>observability</code> , <code>extras</code>), <code>docker-bake.hcl</code> , <code>orchestrators/makes/</code> , images sur GHCR + Docker Hub, <code>wiki/architecture/</code> , <code>wiki/security/</code>

Le fil rouge ici : **le principe du moindre privilège est encodé au plus bas niveau possible**. Quand PostgreSQL peut refuser une lecture grâce à RLS, on ne fait pas de `if (user.id === resource.owner)` en TypeScript — la règle d'autorisation existe une seule fois, dans la base, et ne se contourne pas en appelant l'API autrement (mécanisme RLS au **chapitre 4**, propriété de sécurité au **chapitre 5**).

Compétences transverses

Au-delà des deux activités-types, le projet a mobilisé des compétences peu visibles mais structurantes :

Domaine	Ce qu'on a mis en place	Pourquoi c'était nécessaire
Observabilité	Prometheus (métriques), Grafana (dashboards), Loki + Promtail (logs)	Rendre la stack auditable plutôt qu'opaque — un service muet est un service qu'on ne peut pas exploiter en confiance
Tests	Playwright (E2E osionos), Newman/Postman (contrats API), scripts CTF maison (<code>scripts/security/ctf/</code>), suite BaaS en 16 phases (15 scripts shell + 1 phase Python)	Un filet de sécurité avant chaque merge ; sans lui, refactorer une stack à 50 services devient ingérable
Gestion de version et release	Monorepo, conventions de branche, versionnage des images (<code>docker.io/dlesieur/mini-baas-*</code> et <code>ghcr.io/univers42/grobase-*</code> , tag <code>latest</code> par défaut ; tag de version pour <code>dlesieur/realtime-agnostic:0.2.1</code>), tags Git alignés sur les releases d'images	Pouvoir revenir en arrière proprement, et tracer ce qui tourne en prod à chaque instant

Domaine	Ce qu'on a mis en place	Pourquoi c'était nécessaire
Posture vis-à-vis de l'IA	Usage assumé et tracé de l'assistance IA, lecture critique du code produit comme règle	Apprendre vite sans déléguer la compréhension — voir la section <i>Usage de l'IA</i> en début de dossier

CHAPTITRE 2: Présentation du projet

Présentation de l'entreprise et du service

j'ai effectué mon projet d'application dans le contexte de l'école 42, originellement appelé "ft_transcendence". Un projet qui a évolué pour devenir "Osionos", une plateforme de dashboarding collaboratif. L'objectif était de créer un outil à la fois complet, rapide et agréable à utiliser, en s'inspirant de Notion mais avec une personnalisation, performances et scalabilité plus poussé. Permettant de travailler avec de vrai données.

C'est un travail de Groupe, j'ai donc décider de former mon équipe en Février 2026. Durant cette phase de formation d'équipe, nous avons défini les rôles et responsabilités de chacun, ainsi que les objectifs à atteindre pour le projet. Nous avons également établi une communication régulière en pratiquant les méthodes agiles. On a particulièrement utilisé Scrumban étant un concept hybride entre Scrum et Kanban, qui nous a permis de bénéficier de la structure de Scrum tout en conservant la flexibilité de Kanban. Mon expérience s'est déroulé dans un environnement exigeant, marqué par la nécessité d'adhérer strictement aux méthodologies de développement et aux normes de sécurité d'un grand groupe.

Cahier des charges du projet:

A. CONTEXTE ET OBJECTIFS

Le projet Osionos a été initié pour pallier les lourdeurs d'un processus de création de dashboard. Cette frustration a été le moteur de notre volonté de créer une plateforme qui rendrait la création de dashboard plus rapide, plus flexible et plus agréable à utiliser. Comme on l'a dit auparavant, cette plateforme fonctionne comme une fourmière. En terme de vision pure, on voyait ce projet plus comme une sorte de red sociale géante fait pour le travail. Collaboratif. Il faut imaginer un espace de travail où les utilisateurs pourront travailler dans un espace public et un espace privée. L'espace public permettra d'accepter un trafic plus ou moins dense de changement de laisser gérer l'administrateur..

voici quelques exemples de fonctionnalités que nous avons imaginé pour Osionos:

- **Pages et blocs** : les utilisateurs peuvent créer des pages et les remplir avec des blocs de contenu (texte, images, tableaux, etc.) pour construire leur dashboard.
- **dashboarding** : à l'aide de `/dashboard` ou `/layout` ou bien encore directement depuis le `/database`, les utilisateurs peuvent créer des vues personnalisées de leurs données, avec des filtres, des tris, et des options de visualisation avancées. L'idée ici est que l'on veut pouvoir dans le temps proposer aux gens des formes préétablis mais s'ils veulent pourront ajouter les leurs à travers le code ou même au travers de plugins
- **home**: c'est le dashboard d'accueil, entièrement personnalisable, où les utilisateurs peuvent épingler leurs pages et bases de données préférées pour un accès rapide. L'idée est que ce dashboard d'accueil puisse être partagé entre les membres d'un même workspace, pour créer une sorte de point de

ralliement commun. On va aussi s'inspirer de ce qu'Obsidian a fait avec son "graph view" pour proposer une visualisation de l'ensemble des pages et de leurs interconnexions. Pour faire les nodes et les edges on se basera sur la librairie d3.js, taillée pour ce genre de visualisation. Comme dans linux tout est une archive... Dans notre système tout est une donnée. Chaque donnée peut prendre des formes distinctes (page, bloc, base de données, etc.) et être reliée à d'autres données. L'idée est que le graph view puisse représenter visuellement ces connexions, pour aider les utilisateurs à naviguer dans leur espace de travail et à découvrir des relations entre leurs données.

- **bases de données** : les utilisateurs devraient pouvoir connecter leurs bases de données (PostgreSQL, MongoDB, etc.) à Osionos pour visualiser et interagir avec leurs données en temps réel. L'idée est que les utilisateurs puissent créer des vues personnalisées de leurs données, avec des filtres, des tris, et des options de visualisation avancées. On veut aussi permettre aux utilisateurs de créer des dashboards à partir de ces données, pour suivre les indicateurs clés de performance (KPI) et prendre des décisions éclairées.
- **note**: bien sûr ce système est plus ou moins facile de reproduire ce que font notion. La difficulté réside plus dans l'infrastructure que dans la fonctionnalité en elle-même.
- **wiki**: ici le constat est que notion est beaucoup trop lent. Ne peut pas charger de longue page. Obsidian est plus rapide mais n'est pensé que pour faire des notes. L'aspect général d'Obsidian reste très austère, très bon outil pour un usage professionnel mais trop spécifique. Donc on a eu l'idée de gérer en infrastructure une manière que l'on peut avoir une database statique et une database dynamique. Vscod est sûrement l'un des outils graphiques les plus rapides et les plus polyvalents mais pas adapté pour tous. L'avantage que j'y vois c'est que l'on peut quasiment tout faire avec le clavier et réduire les frictions liés à l'usage de la souris. Donc le wiki sera basé sur toutes ces frictions. Le but c'est que ça soit beaucoup plus rapide que Notion, plus agréable à utiliser qu'Obsidian, et plus accessible que Vscod. Tout en utilisant de vrais données et en laissant les users faire le choix d'écrire en brut ou convertir directement les valeurs dans notre propre markdown en bloc ou en inline.

Objectifs

Le projet Osionos poursuivait trois objectifs majeurs et distincts, chacun rattaché à un profil utilisateur précis et à un problème mesuré dans les outils existants (Notion, Obsidian, VS Code, Confluence).

- 1. Pour l'utilisateur final — unifier la note, la base de données et le dashboard dans un espace de travail rapide et pilotable au clavier.** Il fallait que la même page puisse contenir du texte libre, une vue tabulaire connectée à une vraie base, un graphe de liens, et un dashboard de KPI, sans changer d'outil ni attendre qu'une page de mille blocs se charge. La cible chiffrée : ouvrir n'importe quelle page en moins d'une seconde, peu importe sa taille.
- 2. Pour l'administrateur de workspace — disposer d'un contrôle fin et auditable sur l'espace partagé.** Le Planificateur d'un workspace devait pouvoir définir qui voit quoi (public / privé / partagé), gérer les rôles, brancher ses propres bases de données externes (PostgreSQL, MongoDB, plus tard MySQL et HTTP), et retrouver dans un journal d'audit toute opération critique. Aucun secret en clair, aucune action sans trace.
- 3. Pour l'équipe projet — consolider la fiabilité générale en centralisant l'authentification, les permissions, l'audit et l'observabilité de toute la plateforme.** Plutôt que ré-écrire dix couches de sécurité, on s'est appuyé sur des briques éprouvées (GoTrue, PostgREST, Vault, Kong) assemblées et durcies. Les services applicatifs que nous construisons tournent avec un utilisateur non-root, et le flux public passe par une passerelle unique.

Chaque choix technique de la section suivante répond aux trois objectifs ci-dessus, mais aussi à deux contraintes concrètes : qu'un service puisse tomber sans entraîner les autres (chaque service a son container, redémarrable seul), et que la sécurité soit posée dans la base via RLS plutôt que dans des vérifications applicatives dispersées.

Architecture de la solution et choix techniques

Au début du projet, on avait une feuille blanche et une intuition : “on veut faire un Notion, mais qui sait parler à n'importe quelle base de données, sans rien casser quand on change d'avis sur l'infrastructure”. C'est elle qui a piloté chaque décision technique. À chaque carrefour, on a choisi l'option qui gardait deux portes ouvertes : celle de l'expérimentation rapide pour l'équipe, et celle d'une éventuelle mise en production sérieuse.

Le reste de cette section décrit, dimension par dimension, *ce que nous avons choisi* et surtout *pourquoi nous en sommes arrivés là* — y compris les chemins que nous avons abandonnés en cours de route.

CÔTÉ BACK-END

a. Micro-services, tout en Docker

Notre premier réflexe a été le plus classique : un monolithe Node/Express avec une base PostgreSQL. C'est ce qu'on connaissait, c'est ce qu'on voit dans 90 % des tutos. On a tenu deux semaines. Le problème est apparu très vite : dès qu'on a voulu ajouter MongoDB pour les blocs flexibles d'Osionos, puis Redis pour le cache, puis MinIO pour les fichiers, le monolithe a commencé à ressembler à un sac de nœuds où chaque dépendance tirait sur les autres. Un bug dans la couche fichiers faisait tomber l'auth. Un redémarrage pour ajouter une variable d'environnement coupait toute l'app.

On a fait marche arrière et on a posé une règle simple : **chaque responsabilité a son container, et les services applicatifs que nous construisons ont un Dockerfile reproductible avec un utilisateur non-root**. À partir de là, la stack a commencé à se dessiner naturellement — une brique pour l'auth (GoTrue), une pour la base relationnelle (PostgreSQL), une pour les documents (MongoDB), une pour le cache (Redis), une pour les fichiers (MinIO), une pour la passerelle (Kong), et une série de micro-services NestJS pour la logique qui nous appartient en propre (mongo-api, query-router, storage-router, permission-engine, gdpr-service, etc.). Chaque service est isolé et redémarrable indépendamment. Les images sont versionnées autant que possible — l'image Rust du realtime est épinglée à `dlesieur/realtime-agnostic:0.2.1` ; la dette de hardening restante est le **pinning par digest** des images de service (qui portent encore une étiquette de repli GHCR `:latest`), pas un tag flottant sur le realtime.

Le déclic, c'était de comprendre que **ne pas réécrire ce qui existe déjà** est en soi une compétence. On n'allait pas refaire un PostgREST ou un GoTrue qui sont meilleurs que ce qu'on aurait pu produire en deux mois. On les a assemblés et durcis.

b. Fédération de données

La vraie ambition d'Osionos, c'est de laisser un utilisateur connecter sa base de données, qu'elle soit PostgreSQL, MongoDB, MySQL ou autre, et de naviguer dedans comme s'il s'agissait d'une page Notion. Au début on a tenté l'approche naïve : un connecteur par engine, codé en dur dans le front. Ça marchait pour un, douloureux pour deux, intenable pour trois.

On s'est rendu compte qu'on était en train de réinventer un problème connu : c'est exactement ce que résolvent les **moteurs de fédération SQL**. On a évalué Presto, Trino, Apache Drill, et même quelques options propriétaires. On a retenu **Trino** pour deux raisons : il est open source, et il sait lire PostgreSQL et MongoDB *avec la même syntaxe SQL*, ce qui permet une requête joignant les deux bases — sinon il aurait fallu écrire à la main le code de liaison entre les deux.

Mais Trino, c'est un moteur **analytique**, pas transactionnel. Pour les opérations métier classiques (créer un bloc, modifier une page), on avait besoin d'un chemin court et sécurisé. On a donc construit un service maison, le **query-router**, qui prend une description abstraite de requête (`list`, `insert`, `update`, etc.) et la traduit vers le bon engine via des adapters spécialisés (un *adapter* = un petit module de traduction propre à chaque type de base). La beauté de l'approche, c'est qu'**ajouter un nouvel engine ne demande qu'un nouvel adapter** — pas de refonte du reste. Ce dispatcher a depuis été **réécrit en Rust** (`data-plane-router`), le `query-router` NestJS restant le point d'entrée qui transmet l'exécution au moteur Rust (détail de la bascule TS→Rust au **chapitre 4**).

c. Cohérence multi-engine

Très vite, une question gênante s'est posée : si l'utilisateur écrit dans PostgreSQL et qu'on veut répercuter cette écriture dans MongoDB (par exemple pour mettre à jour une vue dénormalisée), comment on s'assure que les deux restent synchronisés ? La réponse intuitive — “on fait les deux écritures dans la même transaction” — est physiquement impossible dès qu'on traverse deux moteurs différents. C'est un théorème, pas un manque d'effort.

On a regardé comment les grandes plateformes résolvent ce problème. **Supabase, Hasura, AWS** appliquent toutes la même recette : le **pattern outbox** (on écrit dans une seule base, et on dépose dans la même opération une ligne « à rejouer ailleurs » qu'un autre service ira relire). L'écriture applicative se fait dans une seule base (la “source de vérité”), avec en plus une ligne dans une table d'événements. Un service relais lit ces événements et les rejoue vers les autres systèmes — Mongo, Elasticsearch, webhook externe, etc. On perd l'atomicité immédiate (les deux bases ne sont pas mises à jour dans le même instant), mais on gagne la **cohérence éventuelle** : les bases finissent toujours par converger, avec en prime l'audit et la possibilité de rejouer les événements.

C'est ce qu'on a retenu pour le jalon M3 de la roadmap, en s'appuyant sur Redis (déjà présent) comme futur bus d'événements via Redis Streams — pas besoin d'ajouter Kafka tant que l'échelle du projet ne l'impose pas. Aujourd'hui, Redis sert surtout au cache applicatif du plan de données ; le relais outbox (`outbox-relay`) et le connecteur `debezium` sont désormais câblés dans la stack et tournent en *shadow* — il reste à généraliser le projecteur vers Mongo avant de couper.

d. API unifiée et SDK client

Une plateforme qui expose dix services différents avec dix conventions différentes est ingérable. On voulait que le développeur front (nous-mêmes, en l'occurrence) n'ait qu'**une seule façon** de parler au back, peu importe ce qui se passe en coulisse.

On a regardé les API qu'on aimait utiliser : Supabase, Firebase, PocketBase. Le point commun, c'est un SDK qui ressemble à `client.from('table').select().eq(...)` — proche du SQL, mais portable, typé, et indépendant de l'engine. On a repris cette idée et on l'a câblée à notre query-router. Le résultat, c'est notre SDK `@grobases/js`, qui est consommé par les deux frontends d'Osionos sans qu'ils aient à se soucier de savoir si la donnée vient de PostgreSQL, de Mongo ou d'une base externe enregistrée par l'utilisateur.

Côté gateway, on a choisi **Kong en mode déclaratif YAML**, sans base de données. C'est plus rigide qu'un Kong classique, mais ça veut dire que toute la configuration d'ingress vit dans Git, donc reviewable et reproductible. Un nouveau route ne se déploie pas par un clic dans une UI, il passe par une pull request — c'est exactement la garantie qu'on cherchait.

e. Sécurité et observabilité intégrées

Deux principes directeurs ont émergé. Sur la sécurité, on a posé le **moindre privilège au plus bas niveau possible** — dans la base via la RLS PostgreSQL, pas dans des `if` dispersés (voir [chapitre 5 — Sécurité](#)). Sur les secrets, on a quitté les `.env` versionnés au profit de **HashiCorp Vault** dès qu'on a commencé à manipuler des credentials de bases externes — un `.env` en git était un risque inacceptable.

Sur l'observabilité, l'équipe a passé suffisamment de nuits à déboguer en aveugle pour en faire une priorité. **Prometheus + Grafana + Loki + Promtail** sont en place aujourd'hui ; les traces distribuées (OpenTelemetry + Tempo) sont planifiées en M4 — l'architecture est déjà câblée pour les accueillir.

Le détail des couches de défense et des outils est consolidé plus bas dans la section [Stratégie de sécurisation](#).

f. Outillage de développement et de déploiement

Une stack à 50 services Docker devient incompréhensible sans bons outils. On a investi délibérément dans l'outillage **dès le départ**, en suivant trois principes : **tout est dans Docker Compose** (un nouveau dev fait `make up` (ou `make quickstart`) et retrouve la même topologie), **tout est testable en local** (16 phases de tests, dont 15 scripts shell et une phase Python, validant auth, RLS, isolation, storage, realtime, etc.), **tout est reproductible** (`docker-bake.hcl` multi-arch, migrations idempotentes, images tracées par version quand elles sont publiées).

On a délibérément résisté à Kubernetes : tant que la stack tient sur une machine en Docker Compose, on garde la complexité minimale. La liste complète des outils est dans la section [Outillage de développement](#).

g. Auditabilité et traçabilité

Le dernier choix qui structure tout le back est moins glamour mais peut-être le plus important : **tout ce qui se passe dans la plateforme doit pouvoir être expliqué après coup**. Chaque requête HTTP reçoit un `X-Request-ID` à l'entrée de Kong, propagé jusqu'à la base. Chaque migration est numérotée et tracée. La table `audit_log` (qui garde acteur, action, ressource, payload) est en cours de généralisation à toutes les écritures (jalon M1) — combinée aux logs Loki, elle donne une plateforme où « *que s'est-il passé à 14h32 hier pour l'utilisateur X ?* » devient une question à une minute, pas à une journée.

Cette discipline de la trace est une discipline de **respect du futur de l'équipe** : on sait qu'on oubliera, et on construit la mémoire de la plateforme pendant qu'on a encore le contexte en tête.

CÔTÉ FRONT-END

Le front a été pensé avec une logique différente du back, parce que la contrainte n'est pas la même : sur le front, l'ennemi numéro un n'est pas la cohérence des données, c'est la **friction de l'utilisateur**. Une page qui met une seconde de trop à répondre, c'est un utilisateur perdu. Une interface qu'on ne comprend pas, c'est un projet abandonné.

Deux frontends, deux philosophies

On s'est retrouvés très tôt à avoir besoin de deux choses très différentes : (1) un site **marketing** rapide à charger, bien référencé, sobre, qui présente Osionos au monde — c'est `opposite-osiris` ; (2) une **application produit** dense, interactive, en quasi temps réel, qui ressemble à un IDE plus qu'à un site — c'est `osionos`. Vouloir résoudre les deux avec la même stack aurait été une erreur.

Pour le marketing, on a choisi **Astro**. La raison est simple : Astro produit du HTML statique par défaut et ne sert du JavaScript que quand c'est strictement nécessaire (le fameux "islands architecture"). Pour un site dont l'objectif est de charger vite et d'être bien indexé par les moteurs de recherche, c'est le choix le plus rationnel disponible aujourd'hui. On a accepté de ne pas avoir l'écosystème React pour ça — et c'était la bonne décision.

Pour l'application produit, on est partis sur **React 19 + Vite**. React parce que c'est ce que l'équipe maîtrise, qu'on trouve facilement de la doc et que l'écosystème (tanstack, simplewebauthn, etc.) est sans rival. Vite parce qu'après avoir souffert sur Webpack et Create-React-App dans d'autres projets, on n'avait plus envie d'attendre trente secondes à chaque sauvegarde. Vite démarre en une seconde et recompile en moins de cent millisecondes — c'est non négociable quand on développe une UI complexe.

Pourquoi pas Next.js

La question est revenue trois fois pendant la conception : "et si on faisait tout en Next.js, marketing et app, dans un seul projet ?". On a creusé, et on a écarté. Trois raisons :

1. Next.js force une certaine vision du rendu (SSR / RSC) qui complique l'intégration avec notre BaaS auto-hébergé. On voulait un client qui parle à *notre* gateway, pas un framework qui présuppose Vercel.
2. Le coût d'apprentissage des React Server Components nous semblait disproportionné par rapport au gain pour une équipe de cinq devenue deux.
3. Séparer marketing (statique) et app (SPA) nous donne deux pipelines de build plus simples, deux scopes mentaux clairs, et la possibilité d'itérer sur l'un sans casser l'autre.

State management : Zustand plutôt que Redux

On a tenté Redux Toolkit au début. C'est puissant, mais c'est aussi trois fichiers à toucher pour ajouter un champ à un store. À l'échelle d'Osionos (des dizaines d'états : page courante, bloc en édition, filtres, vues, sélection multiple, etc.), la friction devenait insupportable. On a migré vers **Zustand**, qui tient en une fonction par store et qui colle au modèle mental de React. On a payé ce choix par l'obligation de discipliner nos sélecteurs (React 19 est strict sur les snapshots stables) — mais c'est un compromis que l'équipe a accepté.

Architecture en Feature-Sliced Design

Quand on a réalisé qu'on allait dépasser cent composants, on a posé une architecture explicite : **Feature-Sliced Design**. C'est une convention publique qui range le code en couches (`entities` , `features` , `widgets` , `pages` , `app`) avec des règles strictes sur qui a le droit d'importer quoi. Le bénéfice s'est vu immédiatement : on ne se demande plus où mettre un nouveau composant, la convention répond. Et un nouveau membre de l'équipe sait lire la structure sans qu'on ait à lui expliquer.

Le SDK comme contrat

Le front ne parle jamais directement à PostgreSQL ou à Mongo. Il parle à notre SDK `@grobase/js`, qui parle à Kong, qui dispatche vers le bon service. C'est volontaire : ça veut dire que **changer le back ne casse pas le front**, tant que le contrat SDK reste stable. Cette indirection a un coût (une couche supplémentaire à maintenir), mais elle nous a déjà sauvés deux fois : une fois quand on a basculé de Supabase hébergé vers notre BaaS auto-hébergé, et une fois quand on a refondu le format des sessions.

Accessibilité et performance perçue

Deux choses qu'on a traitées dès le départ et pas en fin de projet : l'accessibilité et la performance perçue. On a appris en cours de route qu'**ajouter l'accessibilité à la fin coûte dix fois plus cher que de la penser dès le départ** — et que la même chose vaut pour la performance. Les techniques concrètes (virtualisation, code splitting, suspense, tokens ARIA, focus trap) sont décrites dans la section [Performance et qualité du code](#).

L'architecture d'Osionos ne sort pas d'un seul jet de tableau blanc : elle s'est construite par accumulation de décisions, chacune prise en réaction à un mur réel. C'est ce qui en fait la cohérence — chaque couche se justifie par un problème qu'on a vécu.

SYNTHÈSE DES CHOIX DE STACK

Côté **interfaces utilisateur**, on a séparé le site qui présente Osionos et l'application qu'on utilise. Le site marketing est en **Astro**, parce qu'il doit charger vite et bien se référencer ; l'application est en **React 19 + Vite**, parce que c'est ce qui nous permet de tenir un éditeur dense sans devenir lent. Entre les deux, on partage le **SDK interne** `@grobase/js` — le contrat front ↔ back décrit plus haut (*Le SDK comme contrat*). L'état côté navigateur passe par **Zustand**, l'organisation du code par **Feature-Sliced Design**, et l'authentification sans mot de passe par **WebAuthn** (le standard du navigateur qui permet de se connecter via une *passkey* : empreinte, code de l'appareil ou clé physique, au lieu d'un mot de passe).

Côté **cœur du BaaS**, on a assumé de ne pas réécrire ce qui existe déjà : un **WAF** comme seul point d'entrée public, **Kong** comme passerelle interne derrière lui, **GoTrue** pour l'authentification, **PostgREST** pour exposer PostgreSQL en REST, et **PostgreSQL** comme source de vérité (l'enchaînement complet des barrières est détaillé au **chapitre 5 — Sécurité**). À côté, **MongoDB** sert pour les blocs semi-structurés, avec une façade maison `mongo-api` qui injecte automatiquement le propriétaire depuis le JWT. **Redis** sert de cache et de futur bus d'événements. **MinIO** stocke les fichiers, et notre `storage-router` génère des URLs présignées pour que les uploads ne traversent jamais nos services.

Autour, on a écrit une **poignée de services NestJS** qui portent la logique qui nous appartient : `query-router` pour dispatcher les requêtes vers le bon moteur, `permission-engine` pour centraliser les règles, `session-service` pour le cycle de vie des sessions, `schema-service` pour l'introspection multi-moteur, `gdpr-service` pour l'export et l'anonymisation, plus quelques services utilitaires (logs, mail, newsletter, IA, analytique). **Trino** vient se brancher en lecture sur PostgreSQL et MongoDB pour permettre des requêtes analytiques cross-moteur sans casser le chemin transactionnel.

Enfin, la **sécurité et l'exploitation** combinent un WAF en amont de Kong, **HashiCorp Vault** pour les secrets hors Git, le chiffrement AES-256-GCM des credentials externes par le registre d'adapters (modèle de sécurité au **chapitre 5** ; chiffrement détaillé au **chapitre 4**) et la pile d'observabilité **Prometheus**, **Grafana**, **Loki**, **Promtail**. Tout est **orchestré en Docker Compose**, buildé via `docker-bake.hcl`, publié sur GHCR et Docker Hub avec des tags de version, et couvert par une suite de tests système organisée par phases 1 à 16 avant les merges importants.

Front-ends et SDK

Brique	Rôle
Astro 6	Site marketing statique, SEO
React 19 + Vite 6	Application produit interactive
Zustand 5	État côté navigateur, sans boilerplate
Feature-Sliced Design	Organisation du code par couches
@grobase/js	Contrat stable front ↔ back
@simplewebauthn/browser	Login passkey FIDO2

Cœur du BaaS

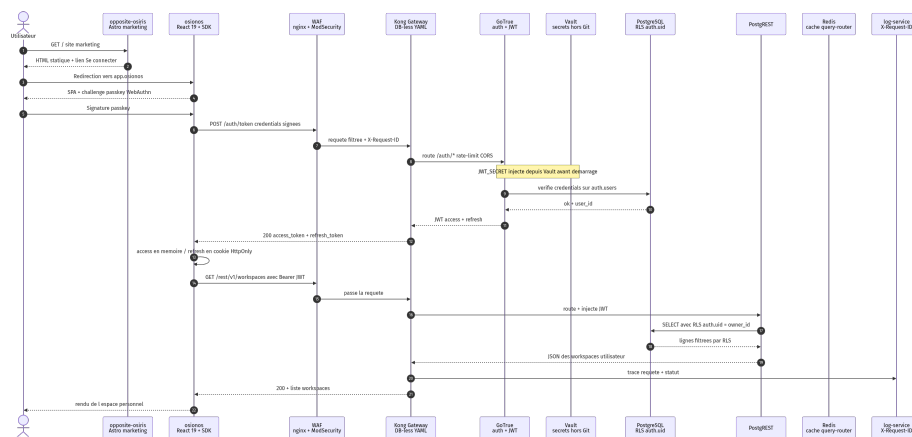
Brique	Rôle
Kong 3.8 (DB-less)	Passerelle interne (routes, JWT, rate-limit, CORS), exposée sur <code>127.0.0.1</code> derrière le WAF
GoTrue 2.188	Authentification, JWT, sessions
PostgREST 12	REST automatique sur PostgreSQL avec RLS
PostgreSQL 16	Source de vérité, RLS, migrations
MongoDB 7 + <code>mongo-api</code>	Blocs semi-structurés, <code>owner_id</code> depuis le JWT
Redis 7	Cache du <code>query-router</code> , pub/sub et futur bus d'événements
MinIO + <code>storage-router</code>	Fichiers, URLs présignées, ACL
<code>realtime-agnostic</code> (Rust)	WebSocket, écoute le WAL de PostgreSQL (le journal interne où la base note chaque modification) et les <i>change streams</i> de Mongo (son flux de changements en direct) pour pousser les mises à jour aux clients
<code>data-plane-router-rust</code> (Rust)	Plan de données : exécution CRUD multi-moteur ; cible de la bascule TS→Rust (forward via <code>/query/v1</code> ; <i>shadow</i> par défaut côté code, actif dans le compose déployé)
<code>query-router</code> , <code>permission-engine</code> , <code>session-service</code> , <code>schema-service</code> , <code>gdpr-service</code> , etc.	Services NestJS internes (le <code>query-router</code> est désormais le chemin legacy derrière le plan de données Rust)
<code>adapter-registry-go</code> , <code>tenant-control</code> , <code>orchestrator</code> , <code>webhook-dispatcher</code> (Go)	Plan de contrôle : registre d'adapters, provisioning des tenants, consolidation des orchestrateurs, webhooks — tous <i>live</i> (<code>PRODUCT_MODE=enabled</code> par défaut depuis la bascule A4 ; seul <code>function-scheduler</code> reste en <i>shadow</i>)
MySQL 8.4 • MariaDB 11 • CockroachDB • MSSQL 2022	Moteurs additionnels au-delà de PG + Mongo (profils <code>data-plane / engines-extra</code>)
Trino 467	Requêtes analytiques cross-moteur

Sécurité, exploitation, qualité

Brique	Rôle
WAF nginx + ModSecurity + OWASP CRS	Filtrage HTTP en amont de Kong
HashiCorp Vault	Stockage chiffré des secrets
AES-256-GCM + scrypt	Chiffrement des credentials de bases externes
Prometheus, Grafana, Loki, Promtail	Métriques, logs, dashboards
Docker Compose + <code>docker-bake.hcl</code>	Orchestration locale, build multi-arch
GHCR + Docker Hub	Distribution des images avec tags de version ; pinning par digest à terminer avant production stricte
Suite BaaS phasée	Tests système : phases 1 à 16, avec 15 scripts shell et une phase Python

SCÈNE DE CONNEXION : DU CLIC AU SESSION TOKEN

Le schéma ci-dessous suit un utilisateur qui ouvre le site marketing `opposite-osiris`, clique sur “Se connecter”, arrive sur `osionos`, s’authentifie, et reçoit la session qui lui ouvre son espace de travail. Chaque flèche est une interaction réelle ; chaque service intervient à un moment précis pour une raison précise.

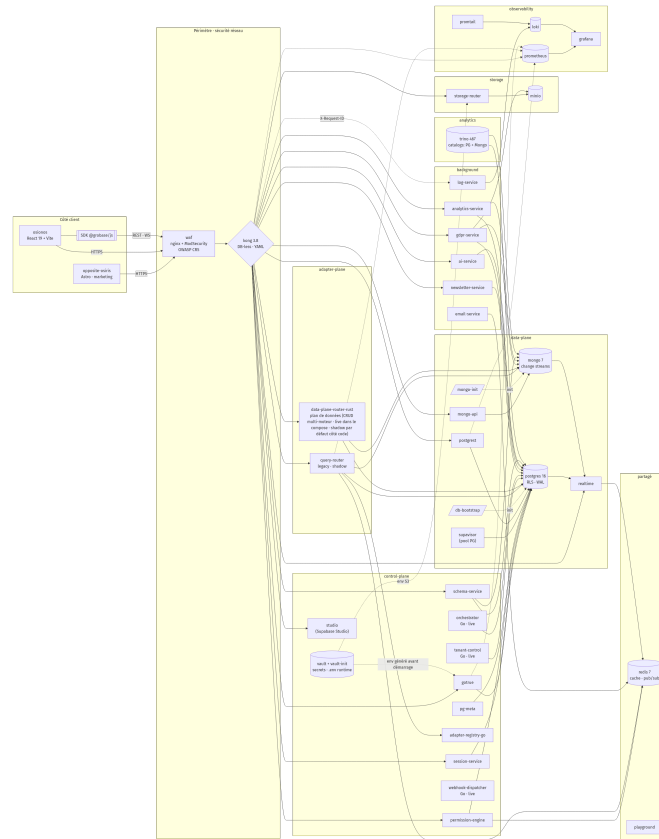


Lecture du schéma : l'utilisateur ne parle jamais directement à une base. Le flux public passe par **WAF** → **Kong**, qui attribue un `X-Request-ID` et applique les contrôles d'entrée. **GoTrue** valide les credentials et signe un JWT avec un `JWT_SECRET` fourni par l'environnement, lui-même généré ou récupéré par les scripts Vault/Makefile hors Git. Le refresh token est protégé côté gateway applicative par un cookie `HttpOnly; Secure; SameSite=Lax`. Le `session-service` existe bien, mais il persiste ses sessions dans PostgreSQL (`session.user_sessions`).

Redis est utilisé pour le cache du `query-router` et comme base de futur bus d'événements. Sur la requête métier qui suit, le JWT est rejoué : **PostgREST** le passe à **PostgreSQL**, qui applique automatiquement la **RLS** (`auth.uid() = owner_id`) — la sécurité finale est dans la base, pas dans le code applicatif.

VUE D'ENSEMBLE DES CONNEXIONS ENTRE SERVICES (ÉTAT ACTUEL)

Le schéma ci-dessous reflète l'état réel du `docker-compose.yml` (orchestrateur fin qui `include:` les fichiers de `orchestrators/compose/base/`) au moment de la rédaction. Les services sont regroupés par **plan d'exécution** (≈17 Compose profiles) ; les principaux sont `control-plane`, `data-plane`, `adapter-plane`, `go-control-plane`, `rust-data-plane`, `storage`, `analytics`, `background`, `observability`, `functions` et `backups` (plus `engines-extra`, `extras`, `ops`, `studio`, `playground`, `realtime`).



Comment lire ce schéma :

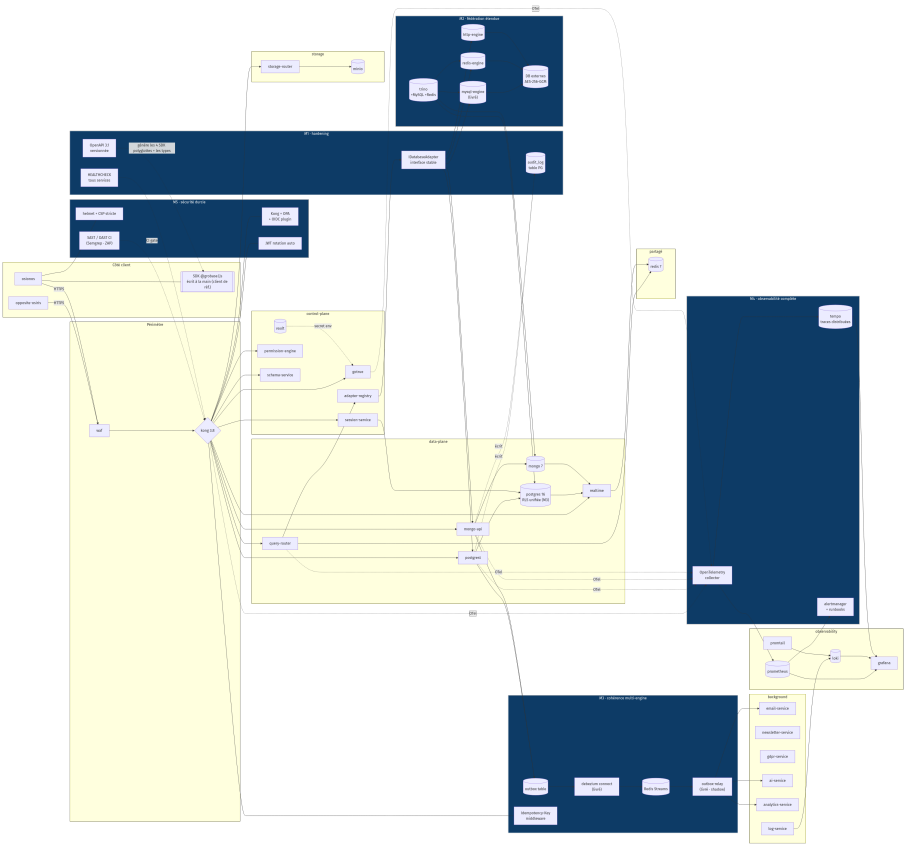
- Côté client** — deux frontends indépendants partagent le SDK `@grobase/js`. Aucun appel direct à la donnée depuis le navigateur.
- Périmètre réseau** — toute requête traverse `waf` (filtrage OWASP CRS) puis `kong` (routage, JWT, rate-limit, CORS). Seul point d'entrée public.
- control-plane** — gouvernance : `gotrue`, `vault`, `session-service`, `permission-engine`, `schema-service`, `pg-meta`, `studio`, plus un **plan de contrôle Go** (`adapter-registry-go`, `tenant-control`, `webhook-dispatcher`, `orchestrator`) désormais *live* (`PRODUCT_MODE=enabled` par défaut dans le compose ; seul `function-scheduler` reste en *shadow*), ayant repris la main sur les services NestJS correspondants.
- data-plane** — engines et leurs façades : `postgres` derrière `postgrest`, `mongo` derrière `mongo-api`, `realtime` qui écoute WAL + change streams, `supavisor` qui pool PG.
- adapter-plane** — le `query-router` (NestJS) consulte l'`adapter-registry` pour résoudre le montage, puis **forward**e l'exécution au `data-plane-router-rust` (Rust) qui dispatche le CRUD vers le bon engine (bascule par requête, détaillée au **chapitre 4**).
- storage** — `storage-router` parle à `minio` avec des credentials S3 injectés par environnement ; le chiffrement des credentials de bases externes est porté par `adapter-registry`.

7. **background** — services à durée de vie longue : `email-service` , `newsletter-service` , `gdpr-service` , `ai-service` , `analytics-service` , `log-service` .
8. **analytics** — `trino` avec catalogs PG + Mongo, pour requêtes analytiques cross-engine.
9. **observability** — `prometheus` , `grafana` , `loki` , `promtail` . Les traces distribuées (Tempo/OTel) ne sont pas encore en place, voir la cible 10/10 ci-dessous.

La règle de circulation se lit dans le sens des flèches : **du moins privilégié vers le plus privilégié**. Le client ne connaît que Kong, Kong que les services applicatifs, les services que leur engine — une compromission d'un étage ne se propage pas sans franchir une nouvelle barrière. C'est la défense en profondeur (voir **chapitre 5 — Sécurité**).

CIBLE 10/10 — À QUOI RESSEMBLERA L'INFRASTRUCTURE UNE FOIS LES MILESTONES M1-M5 LIVRÉES

Le schéma ci-dessous représente l'**état cible** une fois les cinq jalons décrits dans [wiki/todo/README.md](https://wiki.todo/README.md) réalisés. Les nouveautés par rapport à l'état actuel sont regroupées dans les sous-graphes **M1** à **M5** . Tout ce qui apparaît en dehors de ces blocs existe déjà aujourd'hui.



Ce que les milestones ajoutent concrètement :

Jalon	Apport sur le schéma	Pourquoi c'est nécessaire pour passer à 10/10
M1 • hardening	<code>HEALTHCHECK</code> sur tous les services, interface <code>IDatabaseAdapter</code> , spec OpenAPI 3.1 versionnée, table <code>audit_log</code> PG	Rendre la stack auto-décrite (Compose ne tolère plus de service muet) et tracer chaque écriture

Jalon	Apport sur le schéma	Pourquoi c'est nécessaire pour passer à 10/10
M2 • fédération étendue	<code>mysql-engine</code> (livré ; MariaDB, CockroachDB, MSSQL également présents), <code>redis-engine</code> , <code>http-engine</code> + catalogs Trino correspondants, registre de DB externes chiffrées	Tenir la promesse “connecte n’importe quelle base”, pas seulement PG + Mongo
M3 • cohérence multi-engine	Table <code>outbox</code> , <code>debezium connect</code> (livré), <code>Redis Streams</code> comme bus, <code>outbox-relay</code> (livré • shadow), middleware <code>Idempotency-Key</code>	Garantir la cohérence éventuelle entre engines sans rouler de transaction distribuée (un même “tout ou rien” appliqué à plusieurs bases en même temps, complexe et fragile à réaliser)
M4 • observabilité complète	Collecteur OpenTelemetry , Tempo pour les traces distribuées, Alertmanager + runbooks	Pouvoir suivre une requête de bout en bout (Tempo absent aujourd’hui) et être alerté avant l’utilisateur
M5 • sécurité durcie	Plugins Kong OPA + OIDC , helmet + CSP stricte côté front, rotation JWT automatique, SAST/DAST en CI (Semgrep + ZAP)	Vérifier automatiquement chaque dépendance et chaque route en CI plutôt que de s’en remettre à une configuration manuelle

Les briques **déjà présentes** (gateway, auth, RLS, Vault, fédération PG/Mongo/MySQL, Trino, RGPD, audit applicatif) ne sont pas remplacées : chaque jalon n’ajoute que des éléments ciblés (Tempo, Alertmanager, plugins Kong), sans réécrire l’existant.

Outillage de développement

À effectif réduit — cinq au départ, deux à la fin — on n’avait pas le luxe de jongler avec dix chaînes d’outils différentes. On a donc tout fait passer par le même socle, en s’imposant une règle simple : si une commande ne s’exécute pas pareil sur ma machine, sur celle d’un coéquipier et dans la CI, c’est qu’elle n’est pas finie.

Le socle commun, c’est **Docker Compose**. Toute la stack — front, BaaS, observabilité, outils — démarre depuis le même `docker-compose.yml`, avec des builds multi-architecture orchestrés par `docker-bake.hcl` et publiés sur GHCR et Docker Hub avec des tags de version. Le pinning strict par digest reste une cible de hardening : l’image `realtime-agnostic` est elle-même épinglée par version (`dlesieur/realtime-agnostic:0.2.1`), mais de nombreuses images de service portent encore une étiquette de build `:latest` (repli GHCR `ghcr.io/univers42/grobase-<svc>:latest`), à figer par digest avant production. Par-dessus, un **Makefile** sert de façade unique : depuis la racine du dépôt, `make up`, `make tests`, `make health`, `make doctor`. Un nouveau membre n’a pas besoin de connaître chaque service pour être productif, il a besoin de connaître les cibles `make`.

Les outils applicatifs sont volontairement homogènes en **TypeScript**. Le front produit utilise React 19 + Vite 6, parce qu’on voulait du HMR quasi instantané et des tests end-to-end fiables avec Playwright. Le site marketing utilise Astro 6, parce qu’il doit charger vite et bien se référencer. Les micro-services métier sont en NestJS, parce que le format module/contrôleur/service donnait un cadre clair sans imposer une architecture trop lourde. Les dépendances sont gérées en `pnpm` avec workspaces, ce qui nous évite de recompiler dix fois la même chose en CI.

La qualité statique passe par **ESLint** et **SonarQube/SonarCloud** selon les paquets, avec Prettier configuré au moins sur le workspace BaaS NestJS. Le suivi des mises à jour de dépendances s'appuie sur les outils intégrés à GitHub (Dependabot/Renovate), configurés au niveau de l'organisation plutôt que dans ce dépôt. La qualité dynamique passe par une **suite de tests système organisée par phases 1 à 16** (sous [scripts/test/](#)) qui valide bout à bout l'authentification, la RLS, l'isolation par utilisateur, le cycle de vie des JWT, le storage, le realtime, le rate-limit et le CORS. La règle projet est de faire tourner **make tests** (depuis la racine du dépôt) avant les merges importants ; la CI BaaS rejoue aujourd'hui un sous-ensemble critique des phases.

Stratégie de sécurisation

La sécurité d'Osionos n'a pas été ajoutée à la fin comme un vernis : elle est posée par couches successives — WAF, Kong, GoTrue, RLS PostgreSQL, **owner_id** côté Mongo, Vault — selon une règle constante, si l'une cède la suivante doit encore tenir. C'est la **défense en profondeur**, détaillée au **chapitre 5 — Sécurité** ; je ne rappelle ici que les paliers qui ont pesé comme contraintes de conception.

Concrètement, ces paliers descendent du réseau vers la donnée — WAF, Kong, GoTrue, chiffrement AES-256-GCM des credentials externes (**voir chapitre 4**), puis le dernier mot à la base via RLS PostgreSQL et **owner_id** Mongo. Le détail de chaque barrière (terminaison TLS, claims Kong, hachage bcrypt) est **détaillé au chapitre 5 — Sécurité** ; ce qui pesait ici, c'est qu'aucun de ces paliers n'était optionnel dès la conception.

Le dernier palier est **côté front** (entrées jamais rendues brutes, tokens stockés selon leur usage — détail au **chapitre 5 — Sécurité**). Deux points relèvent toutefois de la conception, pas du seul durcissement. L'accessibilité (RGAA — le *Référentiel Général d'Amélioration de l'Accessibilité*, la norme française qui rend un site utilisable par les personnes en situation de handicap) est traitée dès le design : sémantique HTML, contraste, focus visible, navigation clavier complète. Et la conformité RGPD est portée par le **gdpr-service**, dont les endpoints d'export, de consentement et de suppression sont **détaillés au chapitre 5 — Sécurité**.

S'y ajoutent les revues de code obligatoires sur GitHub, le suivi des dépendances (Dependabot/Renovate au niveau de l'organisation) et la portion isolation/auth de la suite de tests système qui rejoue les scénarios d'attaque courants.

Performance et qualité du code

Un workspace réel peut contenir des milliers de blocs, et la page doit rester fluide dans ce cas. La règle qu'on s'est donnée : aucune page ne ralentit parce qu'elle est devenue sérieuse.

Premier levier : la **virtualisation**. Les longues listes — blocs d'une page, lignes d'une vue base de données — passent par **@tanstack/react-virtual**, qui ne rend dans le DOM que ce qui est réellement visible. On peut ainsi faire défiler des milliers d'éléments sans perte de fluidité. Côté serveur, PostgREST porte la pagination via les en-têtes **Range**, ce qui évite de tout télécharger pour n'afficher qu'une fenêtre.

Le deuxième levier, c'est le **cache et la latence**. Redis sert au cache du **query-router** et prépare le futur bus d'événements ; les sessions applicatives qui passent par **session-service** sont persistées en PostgreSQL. Les uploads de fichiers ne traversent jamais nos services applicatifs : MinIO génère des URLs présignées et le client uploadé directement, ce qui retire un goulot d'étranglement potentiel.

Le troisième levier, c'est le **chargement différé**. Vite découpe le bundle par route, React 19 et Suspense reportent les sections non critiques, et le site marketing en Astro charge zéro JavaScript par défaut. Un visiteur qui arrive sur une page produit n'a pas à payer le coût de toute l'application avant de pouvoir lire.

Côté qualité de code, on s'est appuyé sur des **conventions explicites** plutôt que sur la discipline individuelle. Le front suit Feature-Sliced Design avec des règles d'import strictes entre couches, le BaaS est découpé en micro-services NestJS par domaine, et tout passe par le SDK `@grobase/js`, le contrat front ↔ back (voir **chapitre 2**, *Le SDK comme contrat*). La documentation reste vivante — ce wiki, les `README.md` par service, les diagrammes Mermaid — et les commentaires sont concentrés là où le « pourquoi » n'est pas lisible dans le code : RLS, chiffrement, dispatch du `query-router`. Le reste est censé se lire seul.

Enfin, la veille n'est pas laissée au hasard : Dependabot et Renovate rendent les mises à jour visibles et reviewables, la CI rejoue les contrôles critiques, et les images Docker sont progressivement stabilisées par tags de version puis par digest lorsque le pipeline de release le permet. Le tag flottant restant sur `realtime-agnostic` est explicitement traité comme une dette de hardening.

B. PUBLIC CIBLE ET PROFILS UTILISATEURS

Osionos peut toucher beaucoup de monde, et c'est justement sa force autant que son risque. Si on dit que l'outil est fait pour tout le monde, on ne cible plus personne. J'ai donc préféré distinguer les publics par **niveau d'usage** : ceux qui consomment l'information, ceux qui construisent l'espace de travail, et ceux qui administrent la plateforme.

Profil	Besoin principal	Pourquoi Osionos les concerne
Utilisateur final	Écrire, consulter, organiser et retrouver rapidement l'information	Il veut un espace plus rapide qu'un wiki lourd, plus structuré qu'un dossier de fichiers, et plus agréable qu'un outil trop technique
Builder / power-user	Créer des pages, des bases, des vues, des dashboards et des automatisations	Il veut transformer ses données en outil de travail sans repartir de zéro à chaque projet
Équipe projet / startup	Construire vite un espace commun pour suivre un produit, un MVP ou une organisation interne	Elle veut avancer sans perdre du temps dans l'infrastructure, tout en gardant une base évolutive
Analyste / profil data	Connecter plusieurs sources de données et produire des vues exploitables	Il veut arrêter de copier-coller des exports entre outils et travailler sur des données réelles
Administrateur de workspace	Gérer les membres, les rôles, les espaces publics/privés et les permissions	Il doit garder le contrôle sans bloquer la collaboration
Équipe technique	Brancher des bases existantes, surveiller la stack, sécuriser les accès	Elle veut une plateforme auto-hébergeable, observable, et assez claire pour être maintenue dans le temps

La cible principale n'est donc pas "tout Internet". La cible réelle, c'est une équipe ou une organisation qui a déjà trop de données dispersées, trop d'outils séparés, et qui veut une station de travail commune pour écrire, visualiser, connecter et piloter ces données.

C. FONCTIONNALITÉS ATTENDUES

Les fonctionnalités attendues ont été formulées sous forme de cas d'usage, parce que cela oblige à rester concret : *qui veut faire quoi, et pourquoi* ? Plutôt qu'une liste exhaustive, voici comment elles se regroupent par profil d'utilisateur.

L'utilisateur final veut d'abord centraliser son travail au lieu de l'éparpiller entre cinq outils. Il veut créer une page avec du texte, des blocs, des images et des tableaux, et il veut surtout qu'elle reste fluide quand elle devient longue — un outil de productivité perd tout son intérêt s'il ralentit dès que le contenu devient sérieux. Il veut aussi pouvoir naviguer au clavier, retrouver vite une information ancienne, et retrouver sa session sans se reconnecter en permanence ni risquer d'exposer ses données à un autre utilisateur. Le vrai gain de productivité vient souvent de la réduction des petites frictions répétées toute la journée.

Le builder et l'analyste data ont une autre attente : transformer leurs données sans devoir écrire de SQL ni monter une application complète. Le builder veut créer une base de données visuelle depuis l'interface, puis exposer la même source sous forme de tableau, de dashboard, de graphe ou de vue filtrée — parce qu'une donnée n'a pas toujours la même valeur selon la manière dont on la regarde. L'analyste, lui, veut brancher une base PostgreSQL ou MongoDB existante et travailler avec les *vraies* données du projet, pas avec des exports copiés à la main.

L'équipe projet et son administrateur ont besoin d'un point de ralliement commun. Ils veulent partager un dashboard d'accueil dans un workspace pour suivre l'avancement, les priorités et les documents importants. Ils veulent aussi pouvoir séparer espaces publics, privés et partagés, et gérer des rôles et droits d'accès — parce que toutes les informations n'ont pas le même niveau de visibilité, et qu'une plateforme collaborative devient dangereuse si tout le monde peut tout lire ou tout modifier.

L'équipe technique et le responsable conformité, enfin, attendent que la plateforme soit défendable. Toutes les requêtes doivent passer par une gateway unique, qui sert de point de contrôle clair pour l'authentification, les logs, le CORS et le rate-limit. Les logs, les métriques et les traces d'erreur doivent être consultables, parce qu'une stack composée de nombreux services devient impossible à maintenir si elle reste opaque. Et le responsable conformité doit pouvoir exporter, anonymiser ou supprimer les données d'un utilisateur — parce que le respect du RGPD doit être prévu dans le produit, pas traité comme une tâche manuelle après coup.

D. MINIMUM VIABLE PRODUCT (MVP)

Pour Osionos, le MVP ne doit pas être une version miniature de tous les rêves du projet. Il doit plutôt répondre à une question simple : **est-ce qu'une équipe peut utiliser Osionos comme espace de travail réel pour créer des pages, connecter des données, produire une vue utile, et le faire dans un cadre sécurisé ?**

Dans l'idéal, le MVP d'Osionos serait donc une version volontairement réduite, mais complète sur un flux principal : **un utilisateur crée un workspace, écrit une page, connecte une source de données, construit une vue, la partage avec son équipe, et tout reste protégé par l'authentification et les permissions.**

Bloc du MVP	Fonctionnalités minimales attendues	Critère de réussite
Authentification et session	Inscription, connexion, déconnexion, session persistante, récupération du profil utilisateur	Un utilisateur peut revenir dans son espace sans perdre sa session, et ne peut jamais accéder aux données d'un autre utilisateur
Workspace collaboratif	Création d'un workspace, invitation ou ajout de membres, distinction entre espace privé et espace partagé	Une petite équipe peut se créer un espace commun et y organiser son travail

Bloc du MVP	Fonctionnalités minimales attendues	Critère de réussite
Pages et blocs	Création, édition, suppression et réorganisation de blocs simples : texte, titre, liste, image, tableau léger	Une page peut remplacer un document de suivi classique sans devenir lente ni confuse
Base de données interne	Création d'une base simple depuis l'interface : colonnes, lignes, types de base, filtres et tris	Un utilisateur non technique peut structurer des données sans écrire directement de SQL
Connexion à une source réelle	Connexion à PostgreSQL ou MongoDB via le BaaS, lecture sécurisée des données, affichage dans une vue Osionos	La promesse centrale est démontrée : Osionos travaille avec de vraies données, pas seulement avec des données fictives internes
Dashboard d'accueil	Une page home personnalisable avec liens, vues épinglées et indicateurs simples	L'équipe dispose d'un point de ralliement commun pour suivre ce qui compte
Recherche et navigation	Recherche dans les pages, accès rapide aux espaces récents, navigation clavier minimale	L'utilisateur retrouve vite l'information sans fouiller manuellement dans toute l'arborescence
Sécurité minimale sérieuse	JWT GoTrue, RLS PostgreSQL, owner_id côté Mongo, passage obligatoire par Kong, secrets dans Vault	Le MVP n'est pas seulement fonctionnel : il est défendable techniquement et juridiquement
Observabilité minimale	Logs applicatifs, métriques Prometheus, dashboard Grafana simple, erreurs visibles	L'équipe peut comprendre pourquoi quelque chose casse sans deviner à l'aveugle
Déploiement reproductible	Docker Compose, Makefile, migrations idempotentes, seed de démonstration	N'importe quel membre de l'équipe peut lancer le MVP localement et retrouver le même état de départ

Le MVP idéal ne chercherait donc pas à concurrencer immédiatement Notion, Obsidian, Retool et Supabase en même temps. Il chercherait à prouver une seule chose : **on peut créer un espace de travail rapide, collaboratif et sécurisé, capable de transformer des données réelles en pages, vues et dashboards utilisables.**

Ce qui doit rester **hors MVP** pour ne pas perdre le projet : marketplace de plugins, moteur d'automatisation complet, IA avancée, support de tous les moteurs de bases de données, graph view avancé, édition collaborative temps réel façon Google Docs, mobile app native, et architecture 10/10 complète (outbox, Debezium, OTel/Tempo, OPA, SAST/DAST). Ces éléments sont importants, mais ils appartiennent aux perspectives d'évolution, pas à la première version prouvable.

La bonne définition du MVP est donc : **le plus petit Osionos capable d'être utilisé par une vraie petite équipe pendant une semaine sans devoir retourner sur cinq outils différents.**

E. PERSPECTIVES D'ÉVOLUTION

Une fois le MVP stabilisé, les perspectives d'évolution d'Osionos se divisent en deux grandes familles : **faire grandir le produit** (ce que les utilisateurs voient directement) et **durcir la plateforme** (ce qui rend le produit fiable, sécurisé et maintenable à long terme). L'idée n'est pas d'ajouter des fonctionnalités pour faire joli, mais de faire évoluer Osionos sans perdre la promesse initiale : un espace de travail rapide, connecté à de vraies données, et assez solide pour être utilisé en équipe.

Axe d'évolution	Ce que cela apporterait	Pourquoi ce n'est pas dans le MVP
Marketplace de plugins	Permettre à des utilisateurs ou développeurs d'ajouter leurs propres blocs, vues, connecteurs ou automatisations	Cela demande un modèle de permissions, une sandbox, une validation de sécurité et une gouvernance communautaire : trop large pour une première version
Moteur d'automatisation complet	Créer des règles du type "quand une ligne change, envoyer un email", "quand une page est publiée, notifier un channel", etc.	L'automatisation nécessite un moteur d'événements fiable, des retries, de l'idempotence et une interface de configuration claire
IA avancée	Résumer une page, générer une vue, suggérer un dashboard, interroger les données en langage naturel	L'IA n'a de valeur que si les données, les permissions et l'audit sont déjà propres ; sinon elle amplifie le désordre
Support multi-engine étendu	Ajouter MySQL, Redis, HTTP APIs, puis d'autres moteurs via <code>query-router</code> et <code>adapter-registry</code>	Le MVP doit prouver PostgreSQL + MongoDB avant d'étendre la promesse à "n'importe quelle base"
Graph view avancé	Visualiser les relations entre pages, blocs, bases, tags, membres et sources de données comme une carte vivante du workspace	La version simple peut attendre ; un graphe utile demande un modèle de liens propre et une UX soignée
Édition collaborative temps réel	Éditer une même page à plusieurs, façon Google Docs, avec curseurs, présence et résolution de conflits	C'est un sujet complexe : CRDT/OT, conflits réseau, historique, performance. À ne pas mélanger avec la première preuve produit
Application mobile native	Accès plus confortable sur téléphone, notifications push, consultation hors bureau	Osionos est d'abord un outil dense et desktop-first ; le mobile viendra quand le cœur produit sera stable
Architecture 10/10	Outbox, Debezium, OpenTelemetry/Tempo, OPA, SAST/DAST, rotation JWT, hardening complet	Ce sont des chantiers de robustesse indispensables pour une vraie production, mais ils doivent venir après le MVP démontrable

ROADMAP D'ÉVOLUTION PROPOSÉE

La suite logique serait de faire évoluer Osionos par paliers, en évitant le piège du "tout en même temps".

- Palier 1 — stabiliser le produit de base.** Finaliser le cycle workspace → page → base → dashboard → partage. À ce stade, le produit doit être utilisable par une petite équipe sans accompagnement direct des développeurs.
- Palier 2 — ouvrir les données.** Étendre les connecteurs au-delà de PostgreSQL et MongoDB (MySQL, Redis, API HTTP), générer le SDK depuis une spec OpenAPI, et rendre le `query-router` extensible par adapters.

3. **Palier 3 — rendre les événements fiables.** Ajouter le pattern outbox (on écrit la donnée et l'événement à publier dans la même base, dans la même transaction), Debezium (un outil qui lit le journal des modifications de la base et les rediffuse comme un flux d'événements) et Redis Streams pour synchroniser les écritures entre moteurs sans transaction distribuée. C'est le socle du futur moteur d'automatisation.
4. **Palier 4 — rendre la plateforme observable.** Ajouter OpenTelemetry, Tempo, Alertmanager et des runbooks. L'objectif : suivre une requête de bout en bout et être alerté avant que l'utilisateur ne découvre la panne.
5. **Palier 5 — durcir la sécurité.** Ajouter OPA/OIDC côté Kong, rotation automatique des JWT, SAST/DAST en CI, CSP stricte et contrôles de dépendances renforcés. À ce stade, la plateforme commence à ressembler à un produit exploitable sérieusement.
6. **Palier 6 — enrichir l'expérience utilisateur.** Une fois le socle fiable, ajouter graph view avancé, automatisations visuelles, IA assistée, plugins et éventuellement mobile natif.

VISION LONG TERME

À long terme, Osionos pourrait devenir une sorte de **poste de travail universel pour les données d'une équipe** : un endroit où l'on écrit, où l'on connecte des bases, où l'on visualise, où l'on automatise, et où l'on peut demander de l'aide à une IA sans quitter son contexte de travail.

La vision n'est pas seulement de refaire Notion. Notion gère la page, Obsidian la note locale, Retool l'interface métier, Supabase le backend applicatif. L'ambition d'Osionos est de chercher l'intersection : **une interface de travail lisible pour l'humain, branchée sur de vraies données, avec une infrastructure que l'équipe peut comprendre et posséder.**

Le risque principal de cette évolution est évident : vouloir tout faire et finir par ne rien finir. C'est pour cela que la roadmap doit rester stricte : chaque nouvelle capacité doit soit améliorer l'usage réel d'une équipe, soit renforcer la fiabilité de la plateforme. Si elle ne fait ni l'un ni l'autre, elle doit attendre.

les contraintes

Le développement d'Osionos a été encadré par des contraintes fortes, à la fois scolaires, techniques, de sécurité et de qualité. Ce n'était pas un projet que l'on pouvait simplement lancer avec `npm install` sur une machine personnelle et corriger au feeling. L'environnement de travail s'inspire directement de l'esprit des projets **Born2beroot / Inception** de l'école 42 (deux exercices imposant respectivement de durcir une machine virtuelle et de tout faire tourner en conteneurs Docker) : une machine virtuelle stricte, une exposition réseau limitée, des services isolés, et une règle simple — **tout ce qui tourne doit être reproductible.**

La **reproductibilité** n'était pas un "nice to have" : c'était la condition pour que le projet survive au passage d'une machine à l'autre, d'un OS à l'autre, et au jour de l'évaluation. C'est pour ça qu'on a poussé l'idée jusqu'au bout : la VM de référence elle-même est versionnée dans un repo dédié, `Univers42/born2root`. Ce repo permet de régénérer, depuis zéro, une VM moderne et durcie (Debian + Docker + pare-feu + utilisateurs + SSH) qui sert ensuite de socle pour cloner et lancer `ft_transcendence` / Osionos. C'est une vraie **inception** : une VM reproductible qui héberge une stack Docker reproductible.

A. CONTRAINTES D'ENVIRONNEMENT : VM STRICTE, DOCKER PARTOUT, ZÉRO DÉPENDANCE LOCALE

La contrainte la plus structurante était l'environnement d'exécution. Le repo documente explicitement que la stack doit passer par **Docker Compose uniquement** : il ne faut pas installer les dépendances applicatives sur l'hôte, ni démarrer le website ou Osionos avec des scripts locaux `npm`, `pnpm` ou `node`.

C'est le README racine du **monorepo Track-Binocle** qui en fait la source de vérité pour le backend, le site marketing, l'application Osionos et les bridges ; le [README.md](#) du dépôt `grobace` extrait, lui, ne couvre que la stack BaaS et n'évoque ni Osionos ni les bridges.

En pratique, le développement se faisait dans une VM `b2b` sous VirtualBox — la VM construite à partir du repo `Univers42/born2root` — avec Docker à l'intérieur de la VM et parfois le navigateur sur la machine hôte. Cela a créé une vraie contrainte réseau : le chemin complet devenait `navigateur hôte -> localhost hôte -> NAT VirtualBox -> VM -> ports Docker -> proxy HTTPS -> container`. Ce pipeline (génération des certificats locaux, import de la CA dans le navigateur, vérification des ports publiés) a été documenté et automatisé en détail pendant le projet. Une stack verte dans Docker ne suffisait pas : il fallait aussi que les ports soient publiés sur `0.0.0.0`, que le certificat local soit reconnu par le navigateur, et que l'utilisateur n'ouvre pas un port VS Code transféré au hasard à la place du port Compose canonique.

Cette contrainte nous a forcés à automatiser beaucoup de choses : génération des certificats locaux, import de la CA dans les stores système et navigateur, vérification des ports, `make all` comme pipeline principal, et `container-only.mjs` côté `opposite-osiris` pour empêcher l'exécution hors container.

B. CONTRAINTES DE SÉCURITÉ : DONNÉES SENSIBLES, SECRETS, RGPD

Osionos manipule des données sensibles : comptes, sessions, workspaces privés, rôles, bases externes branchées par l'utilisateur, chaînes de connexion, fichiers, logs et traces d'activité. La contrainte n'était donc pas de protéger un formulaire de login, mais un **écosystème de données**.

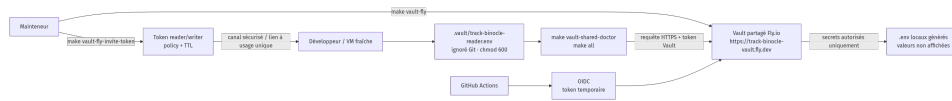
Ces obligations recouvrent l'authentification (GoTrue, rôles `anon` / `authenticated` / `service_role`), l'isolation par RLS et `owner_id`, les secrets hors Git via Vault, le filtrage en entrée (WAF, Kong) et la conformité RGPD via le `gdpr-service` — toutes décrites au **chapitre 5 — Sécurité**. La contrainte propre à ce chapitre était ailleurs : comment partager ces secrets dans une équipe sans jamais les exposer.

La **récupération partagée des secrets** est cette contrainte. Au début, chaque machine génère ses propres `.env` locaux — suffisant pour travailler seul. Mais démarrer la stack sur la machine d'un coéquipier ou dans une VM fraîche a posé le problème : on n'avait pas le droit d'envoyer les vraies clés JWT, credentials OAuth ou secrets SMTP par message, ni de les versionner. Il fallait partager les mêmes valeurs sensibles sans jamais les exposer en clair.

La solution repose sur deux usages de **HashiCorp Vault**. En local, Vault tourne dans le `docker-compose.yml` racine via le profil `secrets` et reste accessible derrière le proxy HTTPS local `https://localhost:18200` ; les scripts et certains containers de bootstrap lui parlent sur le réseau Docker interne (`http://vault:8200`). Pour le partage équipe, on a déployé une instance partagée sur **Fly.io**, exposée à l'adresse HTTPS `https://track-binocle-vault.fly.dev` via `make vault-fly`. Cette instance ne sert pas à héberger l'application — elle sert uniquement de **point d'accès aux secrets partagés**.

Le parcours type ressemble à ceci : un mainteneur génère un token avec `make vault-fly-invite-token VAULT_TEAM_ROLE=reader`, choisit éventuellement une durée de vie courte, et transmet ce token via un canal sécurisé à usage unique (typiquement OneTimeSecret). Le développeur place le fichier ignoré `.vault/track-binocle-reader.env` dans son clone, le passe en `chmod 600`, lance `make vault-shared-doctor` pour vérifier le câblage sans afficher de valeurs, puis simplement `make all` : le Makefile contacte Vault en HTTPS, récupère les variables autorisées par la policy associée au token, et génère les `.env` locaux dans les bons sous-dossiers. Si quelqu'un essaie de

partager un token `localhost`, le Makefile refuse (sauf dérogation explicite pour du test sur la même machine), parce qu'un tel token ne prouve rien sur une autre VM. Côté CI, GitHub Actions ne stocke jamais de token Vault statique : la pipeline s'authentifie par OIDC et reçoit un token temporaire ne valant que le temps d'un run.



La règle tient en une ligne : un secret se récupère seulement avec un token valide, privé, limité par une policy, éventuellement expirant, et jamais versionné — et même alors, un fichier local qui fuiterait ne contient pas de secret en clair.

Cette logique de récupération de secrets s'inscrit dans la **défense en profondeur** — aucune couche suffisante seule, la base gardant le dernier mot sur l'accès réel aux données. Le schéma complet de cette défense, du WAF jusqu'à la base, est **détaillé au chapitre 5 — Sécurité**.

C. CONTRAINTES D'ARCHITECTURE : PAS UNE API EXPRESS CLASSIQUE

Osionos ne devait pas se réduire à une stack `React / Node.js / PostgreSQL`. Le projet utilise bien React et PostgreSQL, mais derrière le front il y a un assemblage — Kong, GoTrue, PostgREST, PostgreSQL, MongoDB, Redis, MinIO, Vault, Trino, plus les micro-services NestJS — qui impose une discipline stricte.

La première règle, c'est que **tout trafic public passe par WAF puis Kong**. Aucun front ne parle directement à une base ; chaque moteur de données a sa façade contrôlée (PostgREST, `mongo-api`, `storage-router`), et c'est cette façade qui porte l'authentification et l'isolation. La deuxième règle, c'est que **chaque service doit pouvoir vivre séparément** : isolé dans son container, configurable par variables d'environnement, et redémarrable sans interrompre le reste de la plateforme. La troisième règle, c'est que **la configuration du gateway doit être lisible dans Git** : Kong tourne en mode DB-less avec sa configuration en YAML versionné, donc toute modification de routes ou de plugins passe par une revue de code, pas par une UI cliquable. Enfin, la stack doit pouvoir **démarrer localement sans dépendre d'un cloud externe** : un développeur sur sa VM doit avoir exactement la même plateforme qu'en CI.

Cette contrainte a complexifié le projet — il aurait été plus rapide de tout coller dans un Express monolithique — mais c'est aussi ce qui en fait la cohérence. On n'a pas construit juste une application, on a construit une petite plateforme, et chaque service peut être justifié par un problème concret qu'on a rencontré.

D. CONTRAINTES QUALITÉ CÔTÉ FRONT-END

Côté front, la contrainte était double : produire une interface riche sans sacrifier la maintenabilité. Osionos est un outil dense, avec des pages, des blocs, du drag and drop, des menus contextuels, des dashboards et beaucoup d'interactions clavier. Le moindre détail UX cassé peut rendre l'outil pénible à utiliser, et il devient vite impossible à réparer si on n'a pas mis en place de garde-fous dès le départ.

Le garde-fou est une chaîne de contrôles qui tourne avant chaque merge, entièrement dans Docker via `apps/osionos/app/scripts/docker-run.sh` : `tsc --noEmit` pour les types, ESLint en `--max-warnings=0`, Playwright pour le end-to-end, plus des tests canvas (blocs, parsing markdown), bridge (liaison Osionos ↔ BaaS) et UX/browser (focus, drag and drop, inline toolbar, menus, indentation, paste, assets, context menu). Un doctor vérifie d'abord que l'environnement de test est sain — un test qui passe dans un environnement cassé ne prouve rien.

La règle est constante : **la qualité front ne dépend pas de la machine du développeur**. Un pipeline qui ne tourne pas pareil chez moi, chez un coéquipier et en CI n'est pas finalisé.

E. CONTRAINTES QUALITÉ CÔTÉ BACK-END ET INFRASTRUCTURE

Côté BaaS, on ne pouvait pas se contenter de tests unitaires classiques, parce que la majeure partie du risque ne vit pas dans une fonction isolée — elle vit dans l'**intégration entre services**. Quand Kong, GoTrue, PostgREST, PostgreSQL, MongoDB, Redis, Vault, MinIO et le realtime doivent collaborer pour qu'un utilisateur lise simplement sa propre page, le risque est dans les coutures, pas dans les briques.

On a donc mis en place une CI locale dédiée, dans [scripts/ci/run-ci-local.sh](#), qui vérifie d'abord les prérequis (Docker, Docker Compose, Make, curl), valide la syntaxe Bash de tous les scripts et passe ShellCheck quand il est disponible. Elle nettoie ensuite entièrement l'état Compose pour ne pas hériter d'un ancien volume, génère un `.env` déterministe, démarre la stack, joue le `db-bootstrap`, vérifie la santé de la gateway sur `/auth/v1/health`, puis exécute `make tests`.

Ce `make tests` du mini-BaaS exécute la matrice complète de tests ; sa famille « smoke » (`make test-smoke` / `make test-scripts`) enchaîne les scripts `phase*-*.*sh` / `phase*-*.*py` dans l'ordre, et chaque phase couvre un risque précis : smoke tests, authentification, accès DB authentifié, isolation utilisateur, méthodes HTTP, codes d'erreur, cycle de vie des tokens, storage, mutations complexes, realtime WebSocket, rate-limit, CORS, Mongo MVP, flux d'auth complet. À côté, SonarCloud est configuré via [sonar-project.properties](#), et `vendor/QA` joue le rôle de registre de tests : il catalogue les scripts existants et stocke leurs résultats.

La contrainte qualité back ne se résumait donc pas à « les routes répondent ». Elle était plus exigeante : **la plateforme doit pouvoir être détruite, reconstruite, testée et expliquée**, sans intervention manuelle fragile entre les étapes.

F. CONTRAINTES DE MÉTHODE ET DE PLANNING

Le projet a démarré avec une équipe de cinq personnes, puis s'est progressivement resserré. Cela a imposé une priorisation forte : tout ne pouvait pas être terminé en même temps. Nous avons donc travaillé avec une logique Scrumban : assez de structure pour garder un cap, assez de flexibilité pour absorber les imprévus.

Cette contrainte explique la séparation entre :

- le **MVP**, qui doit prouver le flux principal ;
- les **perspectives d'évolution**, qui contiennent les ambitions fortes mais non indispensables à la première preuve ;
- la **roadmap 10/10**, qui sert à durcir la plateforme sans prétendre que tout est déjà terminé.

Le vrai risque n'était pas seulement technique : c'était de vouloir faire Notion, Supabase, Retool, Obsidian et Grafana en même temps. La contrainte de qualité nous a donc obligés à réduire le périmètre, documenter les arbitrages et assumer ce qui restait hors MVP.

G. CONTRAINTES DE CENTRALISATION : UN MONOREPO DEVENU STUDIO MULTI-APPS

Une contrainte qu'on n'avait pas anticipée au démarrage est apparue vite : à effectif réduit, on n'avait pas les moyens de maintenir cinq dépôts Git indépendants, cinq pipelines CI distincts, cinq systèmes de versions, cinq backlogs séparés. À chaque fois qu'on essayait de découper proprement (un dépôt pour le

BaaS, un pour `osionos`, un pour `opposite-osiris`, un pour le SDK, un pour les outils internes), on perdait plus de temps à synchroniser les versions et à rejouer les contrats inter-services qu'à avancer sur le produit.

On a donc pris une décision pragmatique : **transformer ce dépôt en studio de travail unique**. Tout vit ici — le BaaS, les deux frontends, le SDK, la documentation, les outils, les scripts d'infrastructure — et chaque application sort progressivement du monorepo quand elle devient assez stable pour vivre seule. Concrètement, le studio nous donne un `make` unique qui sait builder, tester et publier chaque app, un seul `pnpm-workspace.yaml` qui partage les dépendances, et un seul historique Git où l'on peut suivre une refonte de bout en bout. Le coût, c'est un dépôt qui paraît énorme au premier coup d'œil ; le bénéfice, c'est qu'à deux personnes on tient encore une plateforme à plusieurs services sans s'épuiser sur la plomberie.

L'idée n'est pas que tout reste à jamais dans ce monorepo. C'est plutôt un **incubateur** : une app grandit ici jusqu'au moment où la sortir devient moins risqué que la garder. Le BaaS a déjà été **extraît** dans son dépôt autonome `grobase` (images publiées, tags Git alignés sur les releases), et le SDK `@grobase/js` est conçu pour pouvoir être publié séparément le jour où le contrat sera stable. En attendant, le studio fait office d'**atelier partagé**.

Environnement humain et technique

A. ENVIRONNEMENT HUMAIN ET MÉTHODOLOGIE

Le projet a été réalisé dans le cadre de l'école 42, à partir du sujet `ft_transcendence`, puis progressivement transformé en Osionos. L'équipe s'est constituée début 2026 autour de cinq étudiants de 42, avec des profils volontairement complémentaires : pilotage produit, architecture, développement front, développement back, infrastructure, et QA. Chacun avait un rôle principal et un rôle secondaire, pour qu'aucune fonction critique du projet ne dépende d'une seule personne en cas d'absence.

Login 42	Nom	Rôle principal	Rôle secondaire	GitHub	Spécialisation
<code>dlesieur</code>	Dylan Lesieur	ALL	ALL	@LESdylan	Auth, OAuth 2.0, pilotage produit, dossier
<code>danfern3</code>	Daniel Fernández	PO	PM	@danielfdez17	Game engine, WebSockets
<code>serjimen</code>	Sergio Jiménez	PM	TL	@DJSurgeon	Architecture back-end, CI
<code>rstancu</code>	Roxana Stancu	TL	PM	@esettes	Front-end, design system SCSS
<code>vjan-nie</code>	Vadim Jan Nieto	TL	ALL	@vjan-nie	Base de données, Prisma, Docker

Dans les faits, j'ai porté une partie importante du rôle de **product owner / manager de projet** — cadrage de la vision, priorisation du MVP, arbitrage entre les fonctionnalités, écriture du dossier et coordination avec les contraintes techniques posées par les profils architecture. **Vadim** et **Roxana** ont beaucoup pesé sur les exigences d'architecture et de qualité, notamment sur la séparation des services, la sécurité, la

reproductibilité et la stratégie de tests. **Sergio** a porté l'architecture back-end et la CI, et **Daniel** a travaillé sur les fondations temps réel (WebSockets, moteur de jeu) qui ont nourri par la suite la brique `realtime` du BaaS.

La méthode de travail s'est rapprochée d'un **Scrumban** : backlog et priorisation comme en Scrum, exécution plus souple comme en Kanban. On tenait des plannings courts au début de chaque cycle, on suivait l'avancement sur un board Kanban, et on s'autorisait à réordonner sans cérémonie quand la réalité technique nous le demandait. Ce choix était adapté au contexte : beaucoup d'inconnues techniques, une équipe qui apprend en avançant, et un périmètre qui devait rester maîtrisable malgré l'ambition du produit.

Côté outils, on a délibérément séparé la communication temps réel et le suivi de projet. Pour la **communication**, on utilisait **Discord** comme socle principal (voix + salons écrits par sujet), **WhatsApp** pour les échanges rapides et hors-sujet, et **Slack** pour certains canaux plus formels. Pour le **suivi du projet**, on est passé directement par **GitHub Projects** sur l'organisation [Univers42](#) : board Kanban, issues liées aux PR, milestones, le tout au même endroit que le code.

On avait aussi essayé **Notion** au démarrage pour la documentation, et on l'a finalement abandonné : ça créait deux sources de vérité (Notion d'un côté, le repo de l'autre), et au moindre changement d'architecture la doc Notion devenait fautive en silence. On a donc tout rapatrié dans ce wiki, à côté du code, pour que les PR puissent corriger la doc dans le même geste que le code qu'elles modifient.

Côté contrôle de version, on a travaillé en **Git + GitHub avec un modèle proche de Git Flow** : une branche `main` protégée qui représente l'état stable, une branche d'intégration `develop`, des branches `feature/*` pour les nouveautés, `fix/*` pour les correctifs et `release/*` pour les préparations de version. Sur GitHub, on avait activé des **règles de protection de branche** sur `main` (et plus tard sur `develop`) : pas de push direct, une **pull request obligatoire** avec au moins une revue de code approuvée, et la CI verte comme condition de merge. Pour garder un historique lisible, on s'était également imposés des **commits au format Conventional Commits**, contrôlés par des **hooks Git locaux** (`commit-msg`, `pre-commit`) qui refusaient les messages non conformes et lançaient un `lint` rapide avant le commit. Ce dispositif a tourné pendant plusieurs mois et il fonctionnait correctement — il a fini par être **allégé** quand l'équipe s'est resserrée à deux personnes, non pas parce qu'il était inefficace, mais parce qu'à deux on perdait plus de temps à attendre la revue formelle qu'à corriger un commit mal formaté. On a gardé les hooks, on a gardé la PR sur `main`, et on a accepté d'être plus pragmatiques sur les autres branches.

B. ENVIRONNEMENT TECHNIQUE

L'environnement technique peut se résumer en une phrase : **un poste Linux, Docker comme unique runtime, VS Code comme éditeur, et Make comme interface de pilotage**. Le détail compte, parce que c'est cette homogénéité qui permet à chaque membre de l'équipe d'avoir exactement la même plateforme, indépendamment de sa machine personnelle.

Côté **poste de travail**, on s'est appuyés sur l'écosystème Linux dans toute sa diversité. La VM de référence est une VM `b2b` sous VirtualBox générée depuis [Univers42/born2root](#) (dans l'esprit Born2beroot), mais en pratique les membres de l'équipe ont fait tourner la stack sur **Ubuntu, Debian, Kali Linux et Arch Linux** sans rencontrer de problème bloquant. C'est précisément ce qu'on cherchait : tant que Docker, Docker Compose et Make sont disponibles, le reste de la stack ne fait pas la différence. L'éditeur principal était **VS Code**, avec quelques extensions partagées (ESLint, Prettier, Docker, GitLens, Mermaid Preview) pour que la revue de code se fasse dans le même cadre que l'écriture.

Côté **pires applicatives**, on a quatre piles distinctes mais cohérentes, qu'il vaut mieux détailler séparément.

Front application (*osionos*) — React 19, Vite 6, TypeScript strict, Zustand 5 pour l'état, *@tanstack/react-virtual* pour la virtualisation, Playwright pour les tests end-to-end, ESLint + Prettier, le tout buildé et testé via [apps/osionos/app/scripts/docker-run.sh](#) dans un container.

Front marketing (*opposite-osiris*) — Astro 6, TypeScript, SCSS, *@simplewebauthn/browser* pour les passkeys, *sanitize-html* côté contenu, et un garde-fou *container-only.mjs* qui refuse purement et simplement l'exécution si on tente de lancer le projet hors Docker.

BaaS et services applicatifs — Kong 3.8 (DB-less, YAML versionné) comme passerelle, GoTrue 2.188 pour l'auth, PostgREST 12 sur PostgreSQL, NestJS pour les services internes (*mongo-api* , *query-router* , *storage-router* , *permission-engine* , *session-service* , *schema-service* , *gdpr-service* , *log-service* , *email-service* , *newsletter-service* , *ai-service* , *analytics-service*), *realtime-agnostic* en Rust pour le WebSocket, MinIO derrière *storage-router* , et Trino 467 pour la fédération analytique.

Bases de données et stockage — PostgreSQL 16 comme source de vérité (avec RLS, migrations idempotentes et seeds de démonstration), MongoDB 7 pour les blocs semi-structurés avec injection d' *owner_id* par *mongo-api* , Redis 7 pour le cache du *query-router* et le futur bus d'événements, MinIO pour les fichiers, HashiCorp Vault pour les secrets, et un proxy HTTPS local pour que le navigateur hôte puisse parler aux containers en TLS sans erreur de certificat. Les sessions du *session-service* sont persistées en PostgreSQL.

Le tableau ci-dessous sert de résumé visuel, pas de catalogue.

Couche	Pile retenue	Contrainte associée
Poste de travail	Ubuntu, Debian, Kali, Arch ; VM <i>b2b</i> VirtualBox de référence	Linux uniquement, l'OS exact ne doit jamais bloquer un développeur
Éditeur	VS Code + extensions partagées (ESLint, Prettier, Docker, GitLens)	Revue de code et écriture dans le même cadre
Runtime applicatif	Docker + Docker Compose racine	Zéro dépendance applicative installée directement sur l'hôte
Orchestration	Makefile (<i>make all</i> , <i>make playground</i> , <i>make healthcheck</i>), profils Compose	Une commande doit reconstruire et vérifier la stack
Front app	React 19, Vite 6, TypeScript, Zustand 5, Playwright	Tous les scripts passent par <i>docker-run.sh</i>
Front marketing	Astro 6, TypeScript, SCSS, <i>container-only.mjs</i>	Exécution refusée hors container
BaaS	Kong, GoTrue, PostgREST, NestJS, <i>realtime-agnostic</i> (Rust), Trino	Architecture multi-services, aucun accès direct navigateur → base
Bases & stockage	PostgreSQL 16, MongoDB 7, Redis 7, MinIO, Vault	Source de vérité côté PG, <i>owner_id</i> côté Mongo, secrets hors Git
Sécurité locale	HTTPS local, CA projet, WAF, Vault, <i>.env</i> générés	Reproduire un environnement proche production sans exposer les secrets

Couche	Pile retenue	Contrainte associée
Versionnement	Git + GitHub, modèle Git Flow, PR + revue, hooks <code>commit-msg</code> / <code>pre-commit</code>	Historique lisible, branches stables protégées
Qualité	ESLint, TypeScript, Playwright, tests canvas/bridge, smoke tests BaaS, ShellCheck, SonarCloud, QA registry	Pas de merge fiable sans pipeline vérifiable

C. ENVIRONNEMENTS DE DÉPLOIEMENT

Contrairement à un projet client classique — par exemple un projet livré à un grand compte avec trois environnements canoniques (développement local, recette interne, production client) — Osionos n’a pas de client final qui héberge l’application sur ses propres serveurs. Le projet est avant tout un **dossier RNCP/CDA + une plateforme auto-hébergée** ; la « production » au sens strict n’existe pas encore. Cela ne nous a pas dispensés d’organiser nos environnements proprement, mais en les adaptant à notre réalité.

Concrètement, on travaille sur trois environnements imbriqués. Le premier, le plus utilisé, est l’environnement **local de développement** : la stack complète tourne en Docker Compose sur la machine ou la VM de chaque développeur, avec des `.env` générés soit à partir du Vault local, soit à partir du Vault partagé sur Fly.io pour les secrets communs. C’est dans cet environnement qu’on écrit du code, qu’on lance les tests Playwright, la suite BaaS phasée (phases 1 à 16, dont une phase Python) et les scénarios CTF. Aucune variable sensible n’est censée être committée.

Le deuxième environnement est un environnement de **recette / intégration**, qui correspond aux exécutions de la **CI GitHub Actions** et à ce que produit `make ci-run-local` (qui rejoue exactement ce que fait la CI, mais sur une machine de développeur). Il sert à valider qu’une PR est réellement intégrable : reset complet de l’état Compose, génération de `.env` déterministes, `db-bootstrap`, santé de la gateway, puis suite de tests système. Aucune donnée réelle d’utilisateur n’y vit ; les seeds sont des données de démonstration anonymisées. C’est ici qu’on attrape les casses d’intégration avant qu’elles ne touchent `main`.

Le troisième environnement est ce qu’on appelle pour l’instant le **bac de démonstration interne** — une stack identique à la stack locale, mais démarrée sur la VM commune de l’équipe à partir des **images Docker versionnées quand elles sont publiées** sur GHCR et Docker Hub. Il sert aux démonstrations, aux tests d’acceptation manuels, et aux vérifications de bout en bout d’un scénario utilisateur complet (inscription, création de workspace, connexion d’une base externe, partage). À ce stade, les sauvegardes restent simples : snapshot du volume PostgreSQL et export `mongodump` à la demande, parce qu’il n’y a pas encore d’utilisateurs réels à protéger. Le jour où une vraie production sera mise en place pour des utilisateurs externes, ce bac de démonstration sera promu en environnement de pré-production, et la production proprement dite recevra ses propres rituels (snapshots planifiés, retention, restore drills, alerting Prometheus complet).

Environnement	Ce qu’il contient	Ce qu’on y vérifie	Données
Local / dev	Stack complète en Docker Compose sur poste ou VM <code>b2b</code>	Écriture de code, tests E2E Playwright, tests CTF front, debug	Données de développement, seeds locaux

Environnement	Ce qu'il contient	Ce qu'on y vérifie	Données
CI / recette	Même stack rejouée par GitHub Actions ou localement via <code>make all</code> / cibles CI	<code>db-bootstrap</code> , santé gateway, sous-ensemble critique des phases BaaS en CI, ShellCheck, Sonar sur les paquets concernés	Données générées par les seeds, aucune donnée réelle
Démo interne	Images Docker versionnées quand disponibles (GHCR + Docker Hub), VM commune de l'équipe	Tests d'acceptation manuels, scénario utilisateur complet	Données d'exemple anonymisées

La différence par rapport au modèle « local + recette + prod client » classique est donc surtout une question de périmètre : on n'a pas (encore) de prod client, mais on a un environnement qui *jouerait* le rôle de pré-production si on devait en avoir une demain. Les mécanismes de sécurité (secrets récupérés via Vault, RLS PostgreSQL, isolation `owner_id` Mongo, images publiables avec tags de version et pinning à finaliser) sont déjà câblés pour ce scénario, ce qui évite d'avoir à tout refaire le jour où cette étape arrivera.

Objectifs de qualité

Les objectifs qualité ont été définis à partir des contraintes ci-dessus. Ils ne sont pas seulement esthétiques : ils servent à éviter qu'une plateforme aussi distribuée devienne impossible à maintenir.

Objectif qualité	Moyen de contrôle	Résultat attendu
Reproductibilité	Docker Compose, Makefile, <code>.env</code> générés, Vault, migrations idempotentes	Un nouvel environnement peut être reconstruit sans procédure manuelle fragile
Sécurité	WAF, Kong, JWT, RLS, <code>owner_id</code> , Vault, AES-256-GCM, scripts security/CTF	Aucune donnée utilisateur accessible sans identité et permission valides
Qualité front	TypeScript, ESLint <code>--max-warnings=0</code> , Playwright, tests canvas, tests browser/UX	L'interface reste stable malgré la richesse des interactions
Qualité back	<code>run-ci-local.sh</code> , <code>make tests</code> , phases BaaS, healthchecks, ShellCheck	Les services critiques sont testés comme système complet, pas seulement comme fichiers isolés
Observabilité	Prometheus, Grafana, Loki, Promtail, <code>X-Request-ID</code>	Une erreur doit pouvoir être suivie depuis la gateway jusqu'au service concerné
Maintenabilité	Feature-Sliced Design, micro-services par responsabilité, documentation Mermaid et README	Un nouveau membre peut comprendre où intervenir sans casser toute la stack
Conformité	RGPD, <code>gdpr-service</code> , data map, export/anonymisation/suppression	Les données personnelles ont un cycle de vie maîtrisé
Performance	Virtualisation front, cache Redis, pagination PostgREST, Playwright/perf notes	Les longues pages et les vues de données restent utilisables

L'objectif global : **une application ambitieuse, mais vérifiable**. Chaque choix devait laisser une trace — un test, un script, une règle de lint, une doc, un diagramme — pour défendre le projet et pouvoir le reprendre plus tard sans repartir de zéro.

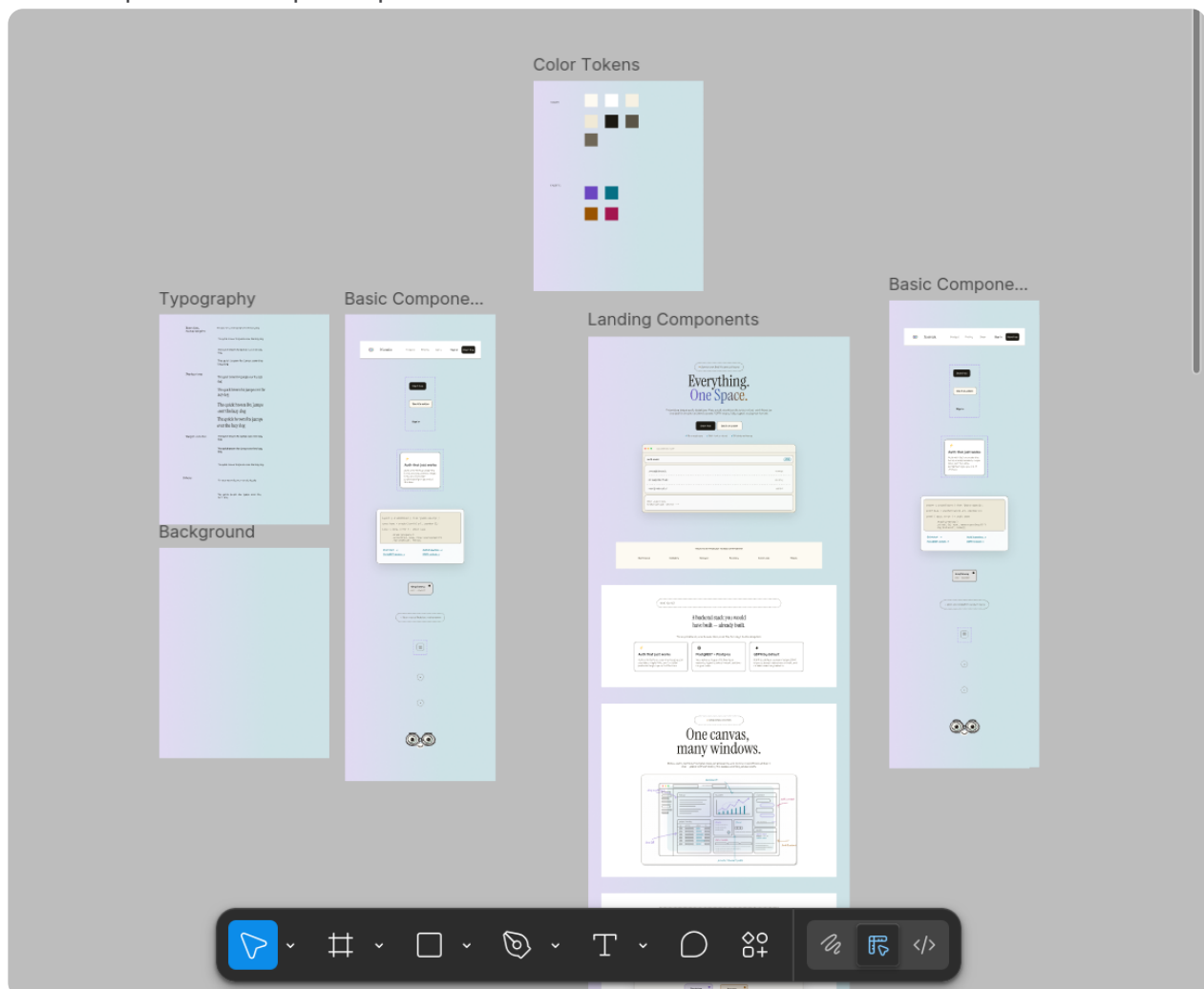
CHAPITRE 3: Les réalisations personnelles, front-end

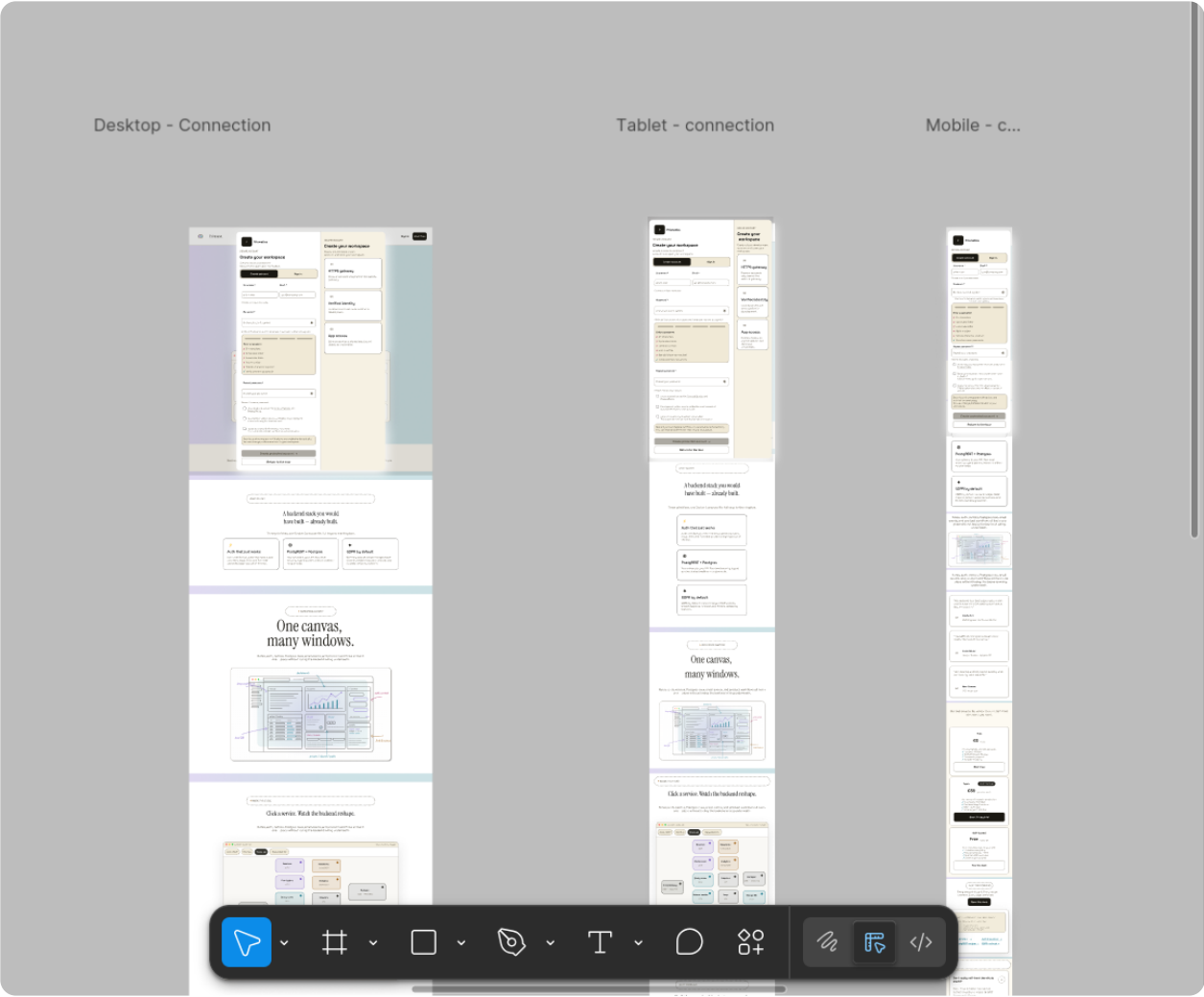
Maquette de l'application et schémas

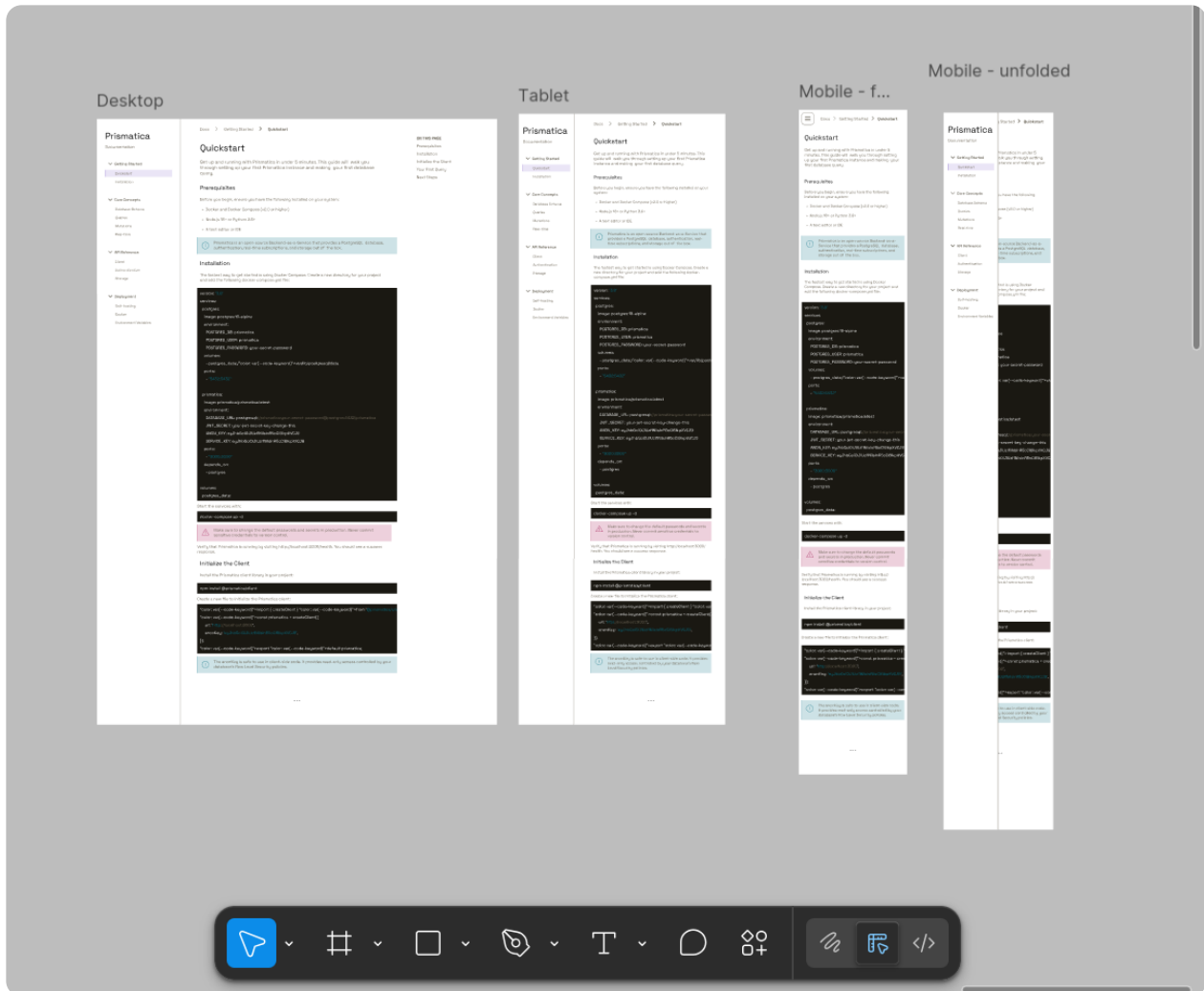
A. CONCEPTION “DESKTOP FIRST”

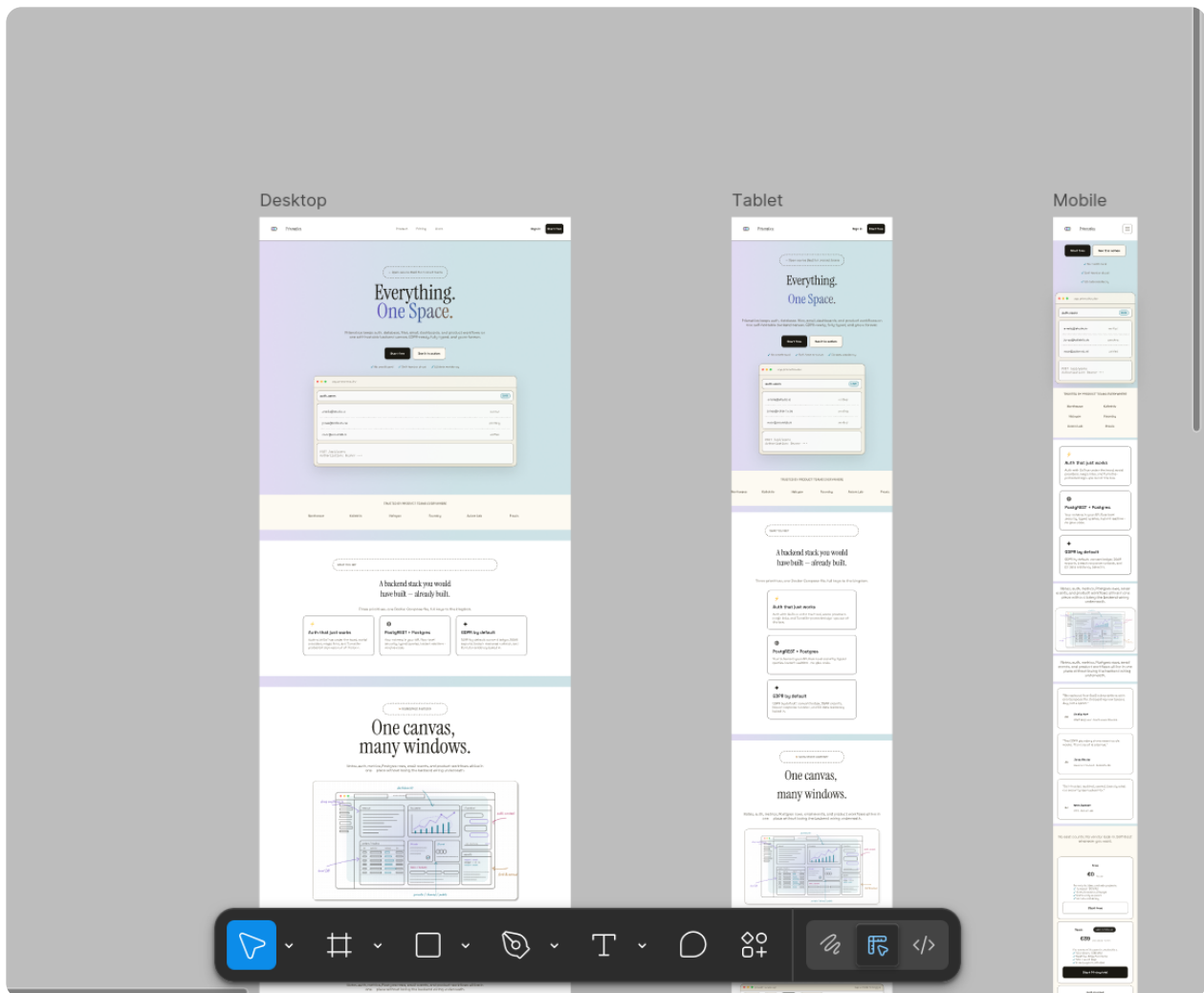
Sergio était le spécialiste front-end de l'équipe. J'ai travaillé en étroite collaboration avec lui pour définir les DoD (Definition of Done) de chaque composant et vérifier que les choix d'implémentation tenaient les exigences de qualité. On a adopté une approche “desktop first”. Pas pour aller à contre-courant des tendances : notre cible était des utilisateurs professionnels qui ouvriraient Osionos sur un poste de travail. Cette approche nous a permis de nous concentrer sur une expérience riche et fonctionnelle, sans être limités par les contraintes d'un design mobile dès le départ. Nous avons cependant veillé à ce que le design soit responsive, pour que l'application reste accessible sur différents types d'appareils.

Avec vladim, nous avons créé les maquettes pour les écrans suivants (voir fig.5 à fig.8) Les maquettes sont compliquées à prendre en main au départ, mais elles nous ont fait gagner du temps ensuite : on repère un choix d'implémentation qui dérape avant de l'avoir codé.



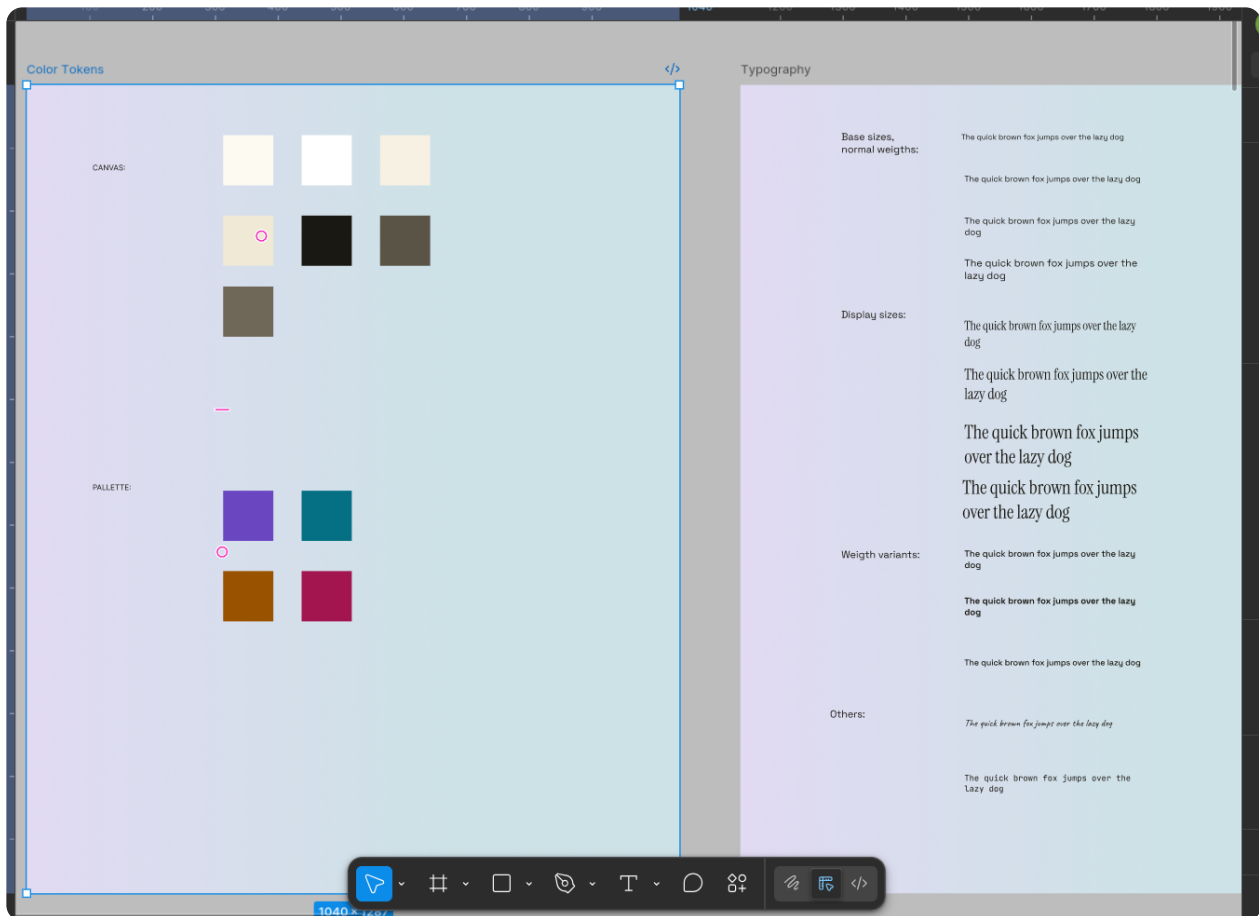




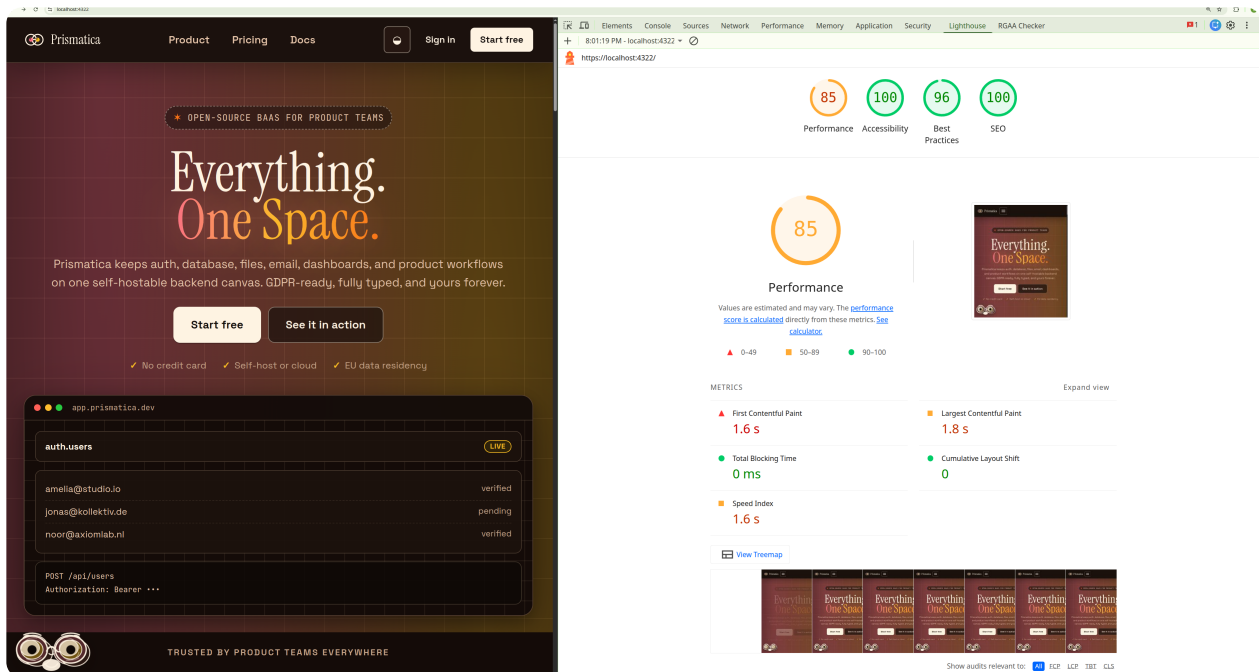


CHARTRE GRAPHIQUE

Pour une identité visuelle cohérente, on a défini une charte graphique. Le choix des couleurs et de la typographie répond à deux impératifs : l'accessibilité et une bonne lisibilité, sur la page web comme dans l'interface de l'application.

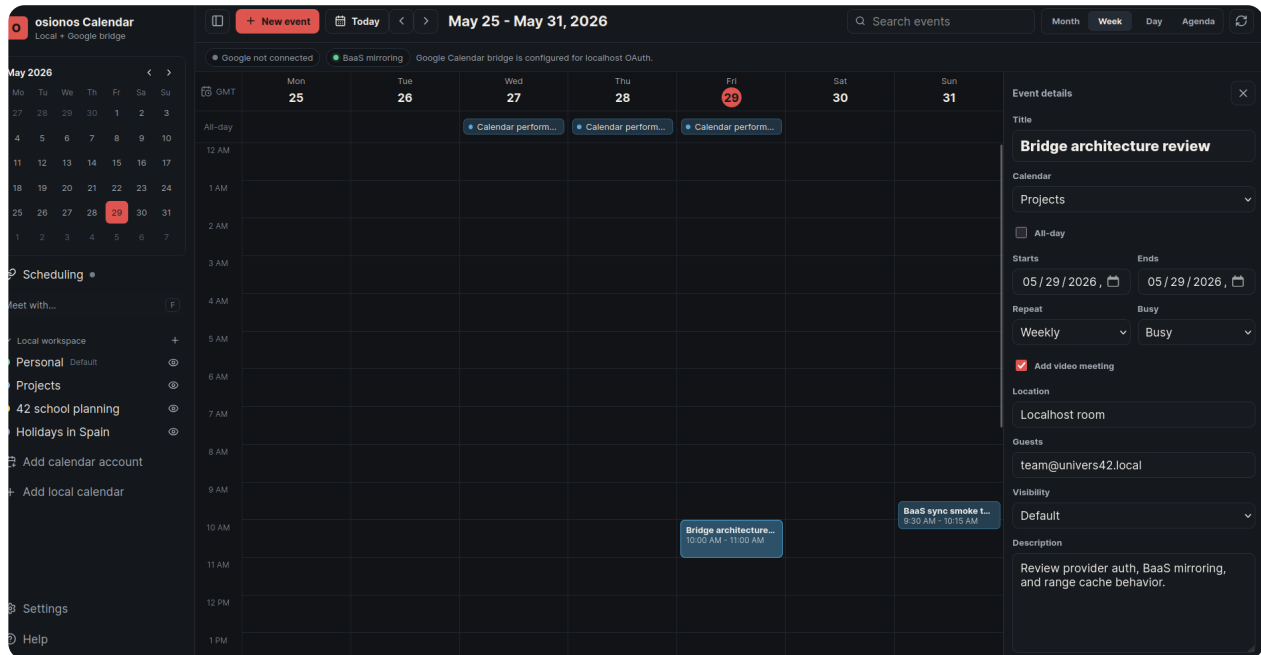


Sur Lighthouse, le contraste atteint 7.5:1 pour le texte courant et 4.5:1 pour les titres — au-dessus du seuil WCAG 2.1. avec un score de 100/100 en accessibilité.



Captures d'écran des interfaces utilisateur

Vue Calendrier — agenda intégré pour les pages de type date/planification, avec navigation mensuelle et gestion des blocs de contenu liés à chaque entrée.



Dashboard d'accueil — première chose qu'on voit en ouvrant l'app : un tableau de bord personnalisable avec des widgets créés à la volée depuis la page d'accueil. C'est ici que l'utilisateur configure son espace de travail.

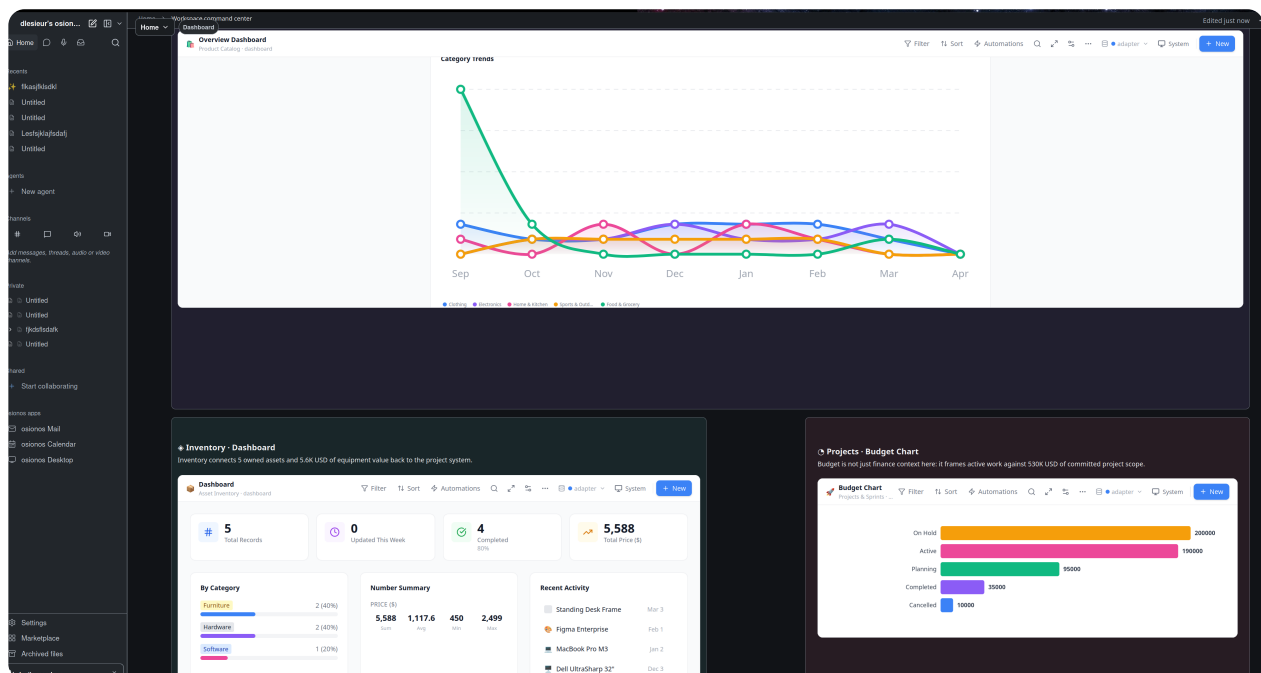
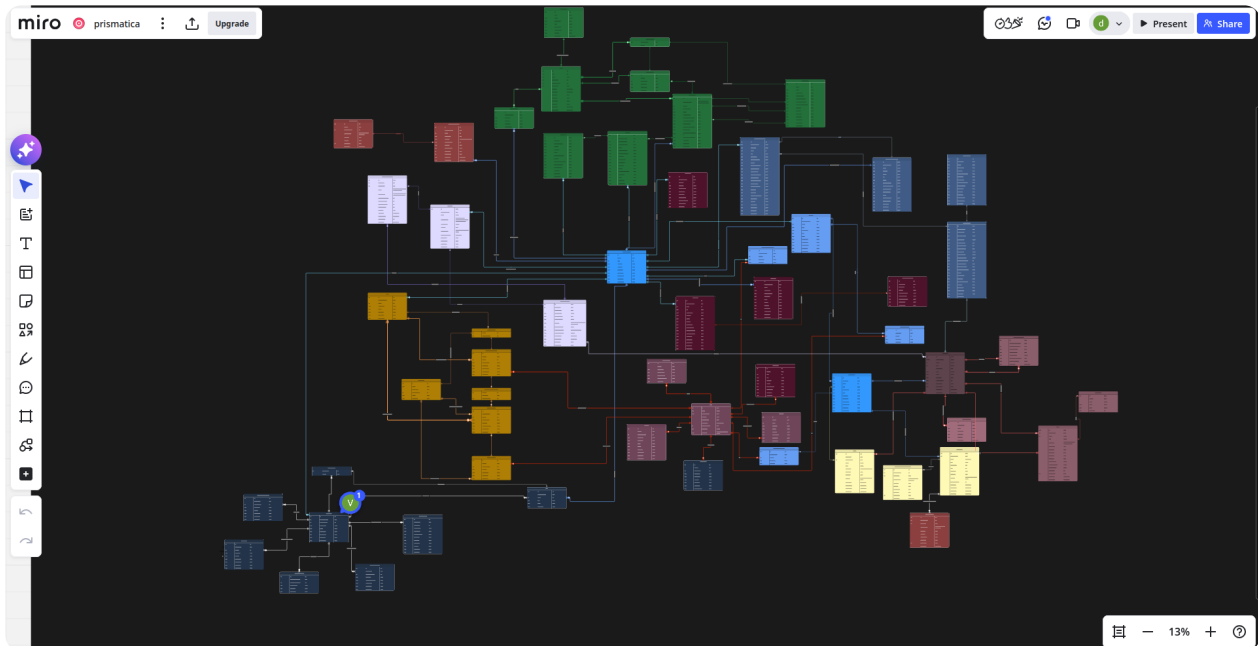
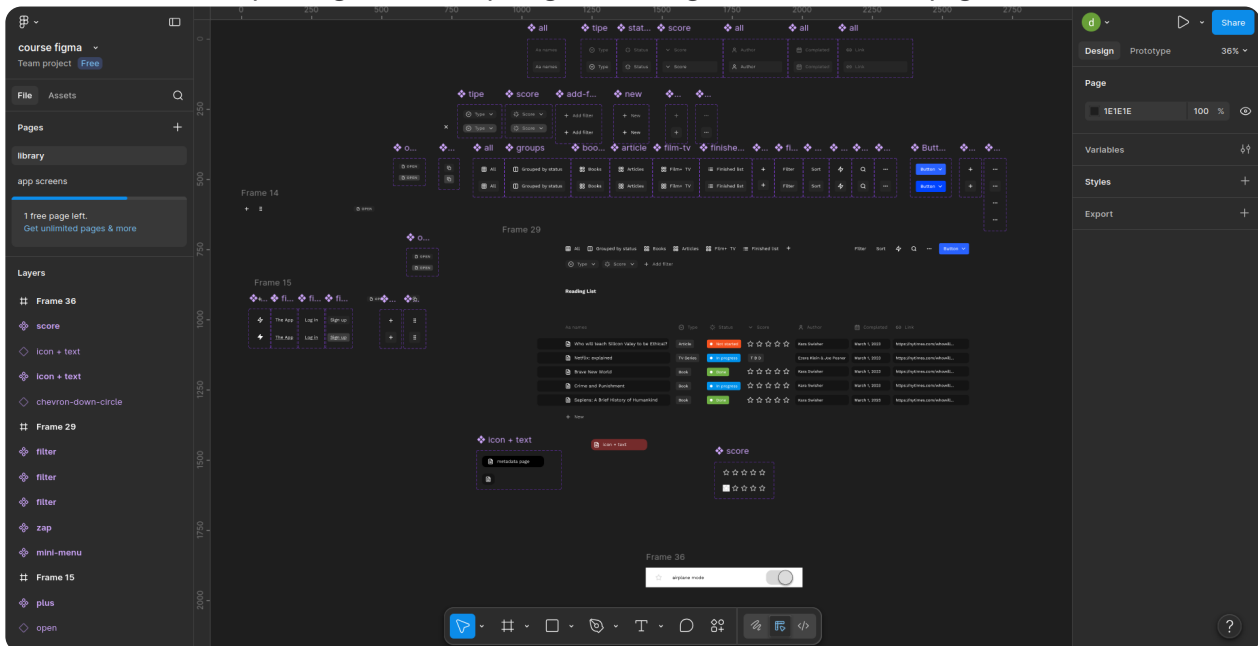


Diagramme entité-relation — schéma de la base de données conçu sur Miro en amont du développement. Il a servi de référence tout au long du projet pour structurer les relations entre pages, blocs, workspaces et utilisateurs.

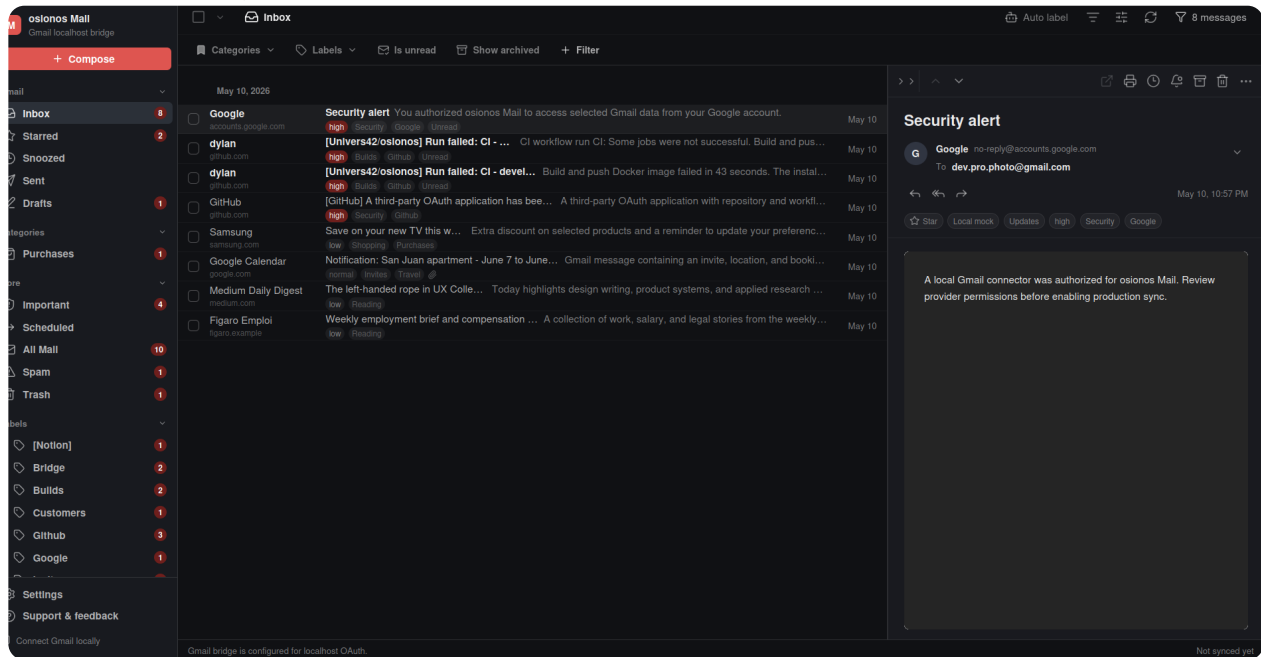


Rendu base de données — vue tabulaire d'une database Osionos, proche du rendu Notion. Chaque colonne est un champ configurable, chaque ligne un enregistrement lié à une page.



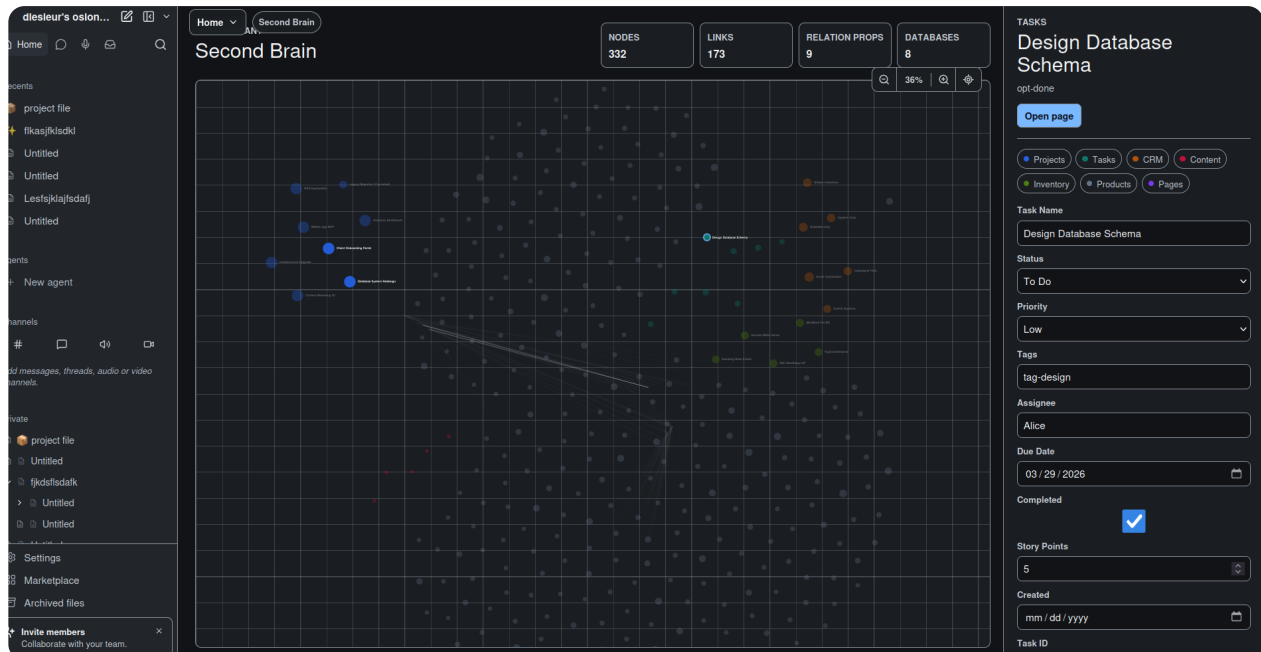
Dossier projet traduit en japonais — démonstration de la fonctionnalité de traduction intégrée : ce dossier a été traduit automatiquement en japonais depuis notre système de notation interne. Une fonctionnalité non prévue au départ : la traduction de page existait déjà pour les blocs, il a suffi de l'appliquer à un document entier. Dossier projet traduit en japonais via le système de notation

Espace mail — module de messagerie intégré à l'espace de travail, accessible directement depuis la sidebar. Permet de gérer les communications sans quitter l'app.



Portail de connexion — page d'authentification avec login 42 OAuth2 ; les jetons sont émis par GoTrue côté BaaS, et les secrets de runtime ne sont jamais versionnés (détaillé au **chapitre 5 — Sécurité**). Portail de connexion — authentification OAuth2 42

Second Brain — vue “note libre” inspirée du concept de second cerveau numérique. Un espace sans structure imposée où l'utilisateur peut penser et organiser librement avec les blocs Osionos.



Optimisations front-end et résultats mesurés

Sur Osionos, la performance front n'est pas un bonus : l'application affiche des pages longues, des blocs imbriqués, des bases de données visuelles, un graphe de connaissances, des menus contextuels et des panneaux de réglages. Sans stratégie explicite, l'interface deviendrait lente avant même que l'utilisateur ait construit un vrai workspace.

J'ai donc travaillé sur quatre axes : **ne pas rendre ce qui n'est pas visible, ne pas recalculer ce qui n'a pas changé, ne pas écrire au backend à chaque frappe, et ne pas charger les bibliothèques lourdes tant qu'elles ne sont pas nécessaires**. Les optimisations ci-dessous sont tirées du code actuel, pas d'une intention théorique.

VIRTUALISATION DES BLOCS LONGS

Le renderer lecture seule utilise `@tanstack/react-virtual` dans `apps/osionos/app/src/widgets/page-renderer/ui/PageBlocksRenderer.tsx`. La virtualisation ne s'active pas tout de suite : elle démarre seulement au-dessus du seuil défini dans `apps/osionos/app/src/entities/block/model/blockVirtualization.ts`, pour éviter de complexifier le rendu des petites pages.

```
// apps/osionos/app/src/widgets/page-renderer/ui/PageBlocksRenderer.tsx
const renderMeta = useMemo(() => createRootBlockRenderMeta(blocks), [blocks]);
const shouldVirtualize = blocks.length >= ROOT_BLOCK_VIRTUALIZATION_THRESHOLD;
const virtualizer = useVirtualizer({
  count: shouldVirtualize ? renderMeta.length : 0,
  getScrollElement: () => scrollElement,
  estimateSize: (index) => estimateBlockHeight(renderMeta[index]?.block ?? blocks[0]),
  getItemKey: (index) => renderMeta[index]?.block.id ?? index,
  overscan: ROOT_BLOCK_VIRTUALIZATION_OVERSCAN,
  scrollMargin,
});
```

Le même composant mesure le décalage réel avec un `ResizeObserver`, parce qu'une page Osionos n'a pas des lignes de hauteur fixe : un bloc peut être un paragraphe, une image, une base inline ou une table.

CACHE ET MÉMOÏSATION DU RENDU MARKDOWN

Le rendu des blocs est coûteux parce qu'un même texte passe par le moteur Markdown interne (`markengine`). Pour éviter de parser plusieurs fois le même contenu, `apps/osionos/app/src/entities/block/ui/ReadOnlyBlock.tsx` utilise un cache LRU simple, limité à 2000 entrées.

```
// apps/osionos/app/src/entities/block/ui/ReadOnlyBlock.tsx
const INLINE_MARKDOWN_CACHE_LIMIT = 2000;
const inlineMarkdownCache = new Map<string, React.ReactNode>();

function renderCachedInlineMarkdown(content: string): React.ReactNode {
  const cached = inlineMarkdownCache.get(content);
  if (cached !== undefined) {
    inlineMarkdownCache.delete(content);
    inlineMarkdownCache.set(content, cached);
    return cached;
  }

  const rendered = timed("renderInlineToReact", () => renderInlineToReact(content, {
    internalLinkRenderer: renderInternalPageLink,
  }));
  inlineMarkdownCache.set(content, rendered);

  if (inlineMarkdownCache.size > INLINE_MARKDOWN_CACHE_LIMIT) {
    const oldestKey = inlineMarkdownCache.keys().next().value;
    if (oldestKey !== undefined) inlineMarkdownCache.delete(oldestKey);
  }

  return rendered;
}
```

Le composant final est aussi protégé par `React.memo`, avec une comparaison ciblée sur le bloc, son index et sa profondeur. L'objectif n'est pas de mettre `memo` partout, mais de protéger les nœuds les plus nombreux — ceux qui se comptent par centaines sur une page longue.

```
// apps/osionos/app/src/entities/block/ui/ReadOnlyBlock.tsx
function areReadOnlyBlockPropsEqual(previous: BlockProps, next: BlockProps): boolean {
  return (
    previous.block === next.block &&
    previous.index === next.index &&
    (previous.bulletDepth ?? 0) === (next.bulletDepth ?? 0) &&
    (previous.numberedDepth ?? 0) === (next.numberedDepth ?? 0)
  );
}

export const ReadOnlyBlock = React.memo(ReadOnlyBlockImpl, areReadOnlyBlockPropsEqual);
```

SAUVEGARDE DIFFÉRÉE ET APPELS ASYNCHRONES

Chaque frappe dans l'éditeur ne déclenche pas une requête réseau. La persistance ne part plus en *fire-and-forget* (envoyer la requête sans jamais vérifier qu'elle a réussi) à chaque frappe : elle passe désormais par un **outbox** côté store ([apps/osionos/app/src/store/sync/usePageSync.ts](#)), qui s'abonne au page store, écrit chaque changement via le bridge **avec retry** (réessai en cas d'échec), et n'avance son ledger (le registre qui mémorise jusqu'où la sauvegarde est confirmée) qu'après confirmation — donc une édition faite hors-ligne n'est jamais perdue. Les anciennes fonctions de [pageStore.persistence.ts](#) sont conservées en no-op pour préserver leurs points d'appel ; les paramètres suivent une logique de persistance analogue via [settingsStoreUtils.ts](#).

```
// apps/osionos/app/src/store/pageStore.persistence.ts
// Page persistence now flows through the BaaS OUTBOX (src/store/sync/usePageSync):
// it subscribes to the page store, writes each change through the bridge WITH retry,
// and only advances its ledger on confirm – so an offline edit is never lost.

/** No-op: block content is persisted by the outbox (see usePageSync). */
export function debouncePersistContent(_pageId: string) {
  // No-op: usePageSync's store subscription detects the edit and persists it with
  // retry.
}
```

La traduction de page montre aussi une pratique de performance : chaque bloc est traduit de manière asynchrone, avec un cache de promesses pour éviter de traduire deux fois le même texte dans la même opération. C'est dans [apps/osionos/app/src/services/page-actions/index.ts](#).

```
// apps/osionos/app/src/services/page-actions/index.ts
const cacheKey = `${targetLocale}\u0000${text}`;
const cached = cache.get(cacheKey);
if (cached) return cached;

const promise = (async () => {
  for (const translator of [
    () => translateWithConfiguredEndpoint(text, targetLocale, jwt),
    () => translateWithGooglePublicEndpoint(text, targetLocale),
    () => translateWithMyMemory(text, targetLocale),
  ]) {
    try {
      const translated = await translator();
      if (translated && !looksLikePrefixTranslation(translated, targetLocale)) {
        return translated;
      }
    } catch {
      // Try the next translation provider.
    }
  }

  return text;
})();
```

CHARGEMENT DIFFÉRÉ CIBLÉ

Le lazy loading (charger une bibliothèque seulement au moment où on en a besoin, pas au démarrage) existe, mais il faut être précis : **Mermaid** est bien chargé dynamiquement par [apps/osionos/app/src/shared/ui/molecules/MermaidDiagram/MermaidDiagram.tsx](#), et le sous-système de base de données embarqué (`notion-database-sys`) utilise `React.lazy` dans son composant `object_database.tsx` pour `DatabaseBlock` , `BlockHandle` et `PageModal` . Et **KaTeX** est lui aussi chargé dynamiquement : [apps/osionos/app/src/shared/lib/math/katexRuntime.ts](#) ne charge katex et sa feuille de style (~580 KiB) qu'au premier rendu d'équation, via `import('katex')` , pour le garder hors du chunk critique de l'éditeur.

```
// apps/osionos/app/src/shared/ui/molecules/MermaidDiagram/MermaidDiagram.tsx
let mermaidInitialized = false;
let mermaidPromise: Promise<typeof import("mermaid").default> | null = null;

function loadMermaid() {
  mermaidPromise ??= import("mermaid").then((module) => module.default);
  return mermaidPromise;
}

async function ensureMermaidInitialized() {
  const mermaid = await loadMermaid();
  if (mermaidInitialized) return mermaid;
  mermaid.initialize({
    startOnLoad: false,
    securityLevel: "strict",
    theme: "default",
  });
  mermaidInitialized = true;
  return mermaid;
}
```

MESURE DE PERFORMANCE EN DÉVELOPPEMENT

La performance est instrumentée par `apps/osionos/app/src/shared/lib/perf/measure.ts` et branchée dès le root React dans `apps/osionos/app/src/app/main.tsx`. En dev, tout span supérieur à 4 ms émet un warning `[perf]`, ce qui force à voir les petits coûts qui s'accumulent.

```
// apps/osionos/app/src/app/main.tsx
createRoot(root).render(
  <StrictMode>
    <Profiler id="App" onRender={recordReactCommit}>
      <App />
    </Profiler>
  </StrictMode>,
);
```

```
// apps/osionos/app/src/shared/lib/perf/measure.ts
const WARN_THRESHOLD_MS = 4;

function warnIfSlow(name: string, durationMs: number) {
  if (durationMs > WARN_THRESHOLD_MS) {
    console.warn(`[perf] ${name}: ${durationMs.toFixed(1)}ms`);
  }
}
```

ASTUCES DE PERFORMANCE : PARALLÉLISME, DÉDUPLICATION, CACHE

Au-delà de la virtualisation et du lazy loading, plusieurs micro-optimisations récurrentes pèsent sur la fluidité perçue. Aucune n'est théorique : chacune est en place dans le code.

1. Paralléliser les I/O indépendants. Quand deux appels ne dépendent pas l'un de l'autre, on ne les attend pas en série — on les lance ensemble avec `Promise.all`, et la latence totale devient celle du plus lent au lieu de la somme des deux.

```
// apps/osionos/app/src/features/settings/permissions/usePolicyMatrix.ts
const [roleRows, policyRows, roster] = await Promise.all([
  fetchRoles(), fetchPolicies(), fetchPeople(),
]);
```

Le même motif sert en *fan-out* sur une collection : on hydrate en parallèle les pages de tous les workspaces ([App.tsx](#)), on introspecte le schéma de chaque mount en parallèle ([liveMountTables.ts](#)), et quand une source peut échouer sans bloquer les autres, on bascule sur `Promise.allSettled` ([useBaasGraph.ts](#)).

2. Charger les bibliothèques lourdes en parallèle et à la demande. Le runtime KaTeX charge le JS et sa feuille de style ensemble, au premier rendu d'équation et hors du chunk critique ([katexRuntime.ts](#) : `Promise.all([import("katex"), import("katex/dist/katex.min.css")])`) ; l'écran 2FA importe `qrcode` pendant que la requête d'enrôlement est déjà en vol ([SettingsCenter.tsx](#)).

3. Mémoïser la promesse, pas seulement le résultat. Pour dédupliquer le travail asynchrone concurrent, on met en cache la *promesse en cours* : deux appels identiques rapprochés partagent le même vol réseau. C'est le cas de la traduction de blocs ([page-actions/index.ts](#) — un même texte n'est jamais traduit deux fois) et des GET de l'api-client ([client.ts](#) : `inflightGets`), doublés d'un cache de schéma de 60 s côté live mounts.

4. Plafonner la concurrence. L'api-client borne le nombre de requêtes en vol (`MAX_CONCURRENT_REQUESTS = 6`, [client.ts](#)) : une vue qui réclame trente pages les draine poliment au lieu de noyer le backend — c'est la correction du *thundering-herd* (une avalanche de requêtes simultanées qui saturent le serveur) qui déclenchait des rafales de 429/502 (codes d'erreur HTTP : trop de requêtes / passerelle saturée).

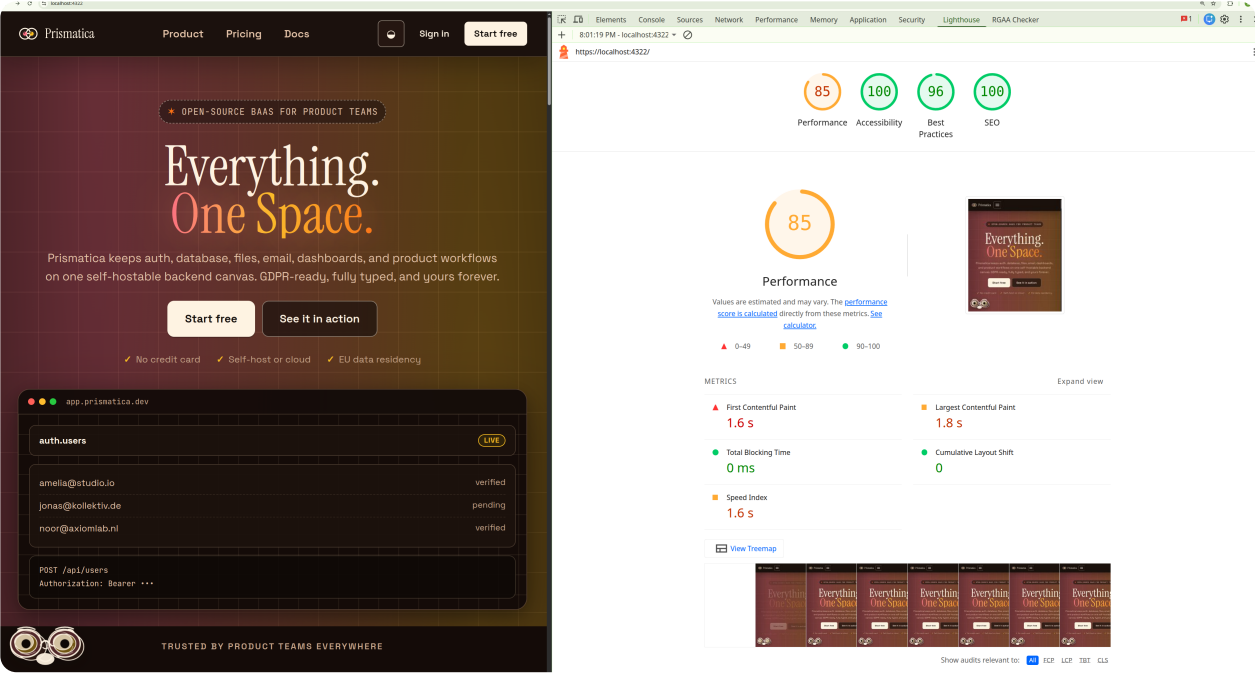
5. Cache LRU pour le rendu pur coûteux. Le rendu du markdown inline d'un bloc est mémoïisé dans un cache LRU borné (`INLINE_MARKDOWN_CACHE_LIMIT = 2000`, [ReadOnlyBlock.tsx](#)) : re-rendre un bloc ne re-parse jamais son markdown, et les entrées les plus anciennes sont évincées quand le cache déborde.

BUILD, BUNDLE ET SEO

L'application privée `osionos` est une SPA Vite : elle n'est pas pensée pour le référencement public. Son [index.html](#) garde les bases nécessaires (`lang`, `viewport`, `title`), mais la stratégie SEO du produit est portée par le site Astro `opposite-osiris`, qui rend du HTML statique et définit les balises `description`, `color-scheme`, favicon et preconnect dans [apps/opposite-osiris/src/layouts/Layout.astro](#) ; la CSP stricte de production est, elle, générée par `security.csp` d'Astro (voir [astro.config.mjs](#)).

```
<!-- apps/opposite-osiris/src/layouts/Layout.astro -->
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<meta name="color-scheme" content="light dark" />
<meta name="description" content={description} />
{isDev && <meta http-equiv="Content-Security-Policy" content={developmentCsp} />}
<link rel="preconnect" href="https://fonts.googleapis.com" />
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
<title>{title}</title>
```

La capture Lighthouse disponible dans le dossier a été réalisée sur `https://localhost:4322/`, donc sur le site marketing Prismatica, pas sur l'app privée. Elle montre : **Performance 85**, **Accessibilité 100**, **Best Practices 96**, **SEO 100**, avec **FCP 1,6 s**, **LCP 1,8 s**, **Total Blocking Time 0 ms** et **CLS 0**.



Extraits de code, interfaces utilisateur statiques (React / SCSS)

A. ORGANISATION MINIMALE DU PROJET FRONT-END

Le front d’Osionos n’est pas organisé comme une simple collection de composants React. Il suit une organisation proche de **Feature-Sliced Design** : les éléments métier vivent dans `entities`, les interactions dans `features`, les assemblages visibles dans `widgets`, l’orchestration dans `app`, et les composants réutilisables dans `shared`.

Dossier	Rôle dans Osionos	Exemples vérifiés
<code>apps/osionos/app/src/app</code>	Point d'entrée, styles globaux, shell principal	<code>main.tsx</code> , <code>App.tsx</code> , tokens CSS
<code>apps/osionos/app/src/entities</code>	Objets métier affichables	<code>page</code> , <code>block</code> , <code>user</code>
<code>apps/osionos/app/src/features</code>	Interactions utilisateur	auth, block editor, page management, slash commands, settings
<code>apps/osionos/app/src/widgets</code>	Zones UI composées	sidebar, page renderer, database view, channel messages, graph explorer
<code>apps/osionos/app/src/shared</code>	API client, hooks, primitives UI, config, perf	<code>api/client.ts</code> , <code>Modal.tsx</code> , <code>Dropdown.tsx</code> , <code>measure.ts</code>
<code>apps/osionos/app/src/store</code>	Stores Zustand et persistance	pages, database, settings
<code>apps/osionos/app/src/services</code>	Actions applicatives hors composant	page actions, realtime messages

Le point d'entrée est volontairement minimal. Il monte React 19, active le `StrictMode`, branche le `Profiler`, puis laisse `App.tsx` assembler le shell.

```
// apps/osionos/app/src/app/main.tsx
import { Profiler, StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "../App.tsx";
import { recordReactCommit } from '@shared/lib/perf/measure';
import './styles/global.css';

const root = document.getElementById("root");
if (root) {
  createRoot(root).render(
    <StrictMode>
      <Profiler id="App" onRender={recordReactCommit}>
        <App />
      </Profiler>
    </StrictMode>,
  );
}
```

`App.tsx` est le shell applicatif : il initialise la session, applique le thème, choisit entre le mode debug, l'écran de handoff Prismatic, puis le layout principal avec sidebar, contenu, settings et notifications.

```
// apps/osionos/app/src/app/App.tsx
return (
  <div
    data-testid="app-shell"
    className="relative flex h-screen w-screen overflow-hidden bg-[var(--osio-bg-page)]"
  >
    <Sidebar
      onOpenSettings={() => setSettingsOpen(true)}
      onOpenHome={() =>
        usePageStore.setState({
          activePage: null,
          showTrash: false,
          navigationPath: [],
        })
      }
      onOpenTrash={() =>
        usePageStore.setState({
          activePage: null,
          showTrash: true,
          navigationPath: [],
        })
      }
    >
    <SidebarTrigger />
    <main className="flex-1 flex min-w-0 overflow-hidden relative">
      <MainContent />
    </main>
    <WorkspaceThemePanel />
    {settingsOpen && <SettingsCenter initialTab="general" onClose={() =>
      setSettingsOpen(false)} />}
    <ToastViewport />
  </div>
);
```

La pile front est visible dans [apps/osionos/app/package.json](#) : React 19, Vite 6, TypeScript, Zustand, Playwright, lucide-react, [@tanstack/react-virtual](#), Mermaid, KaTeX, Leaflet, **ECharts** et Recharts (graphiques), [d3-force](#) (graphe), [livekit-client](#) (visio temps réel), [i18next](#) (i18n) et [@simplewebauthn/browser](#) (WebAuthn). Les scripts passent tous par [scripts/docker-run.sh](#), ce qui force le même environnement de build et de test pour tout le monde.

```
// apps/osionos/app/package.json
{
  "scripts": {
    "build": "bash scripts/docker-run.sh build",
    "typecheck": "bash scripts/docker-run.sh typecheck",
    "lint": "bash scripts/docker-run.sh lint",
    "test:e2e": "bash scripts/docker-run.sh test-e2e",
    "test:canvas": "bash scripts/docker-run.sh test-canvas",
    "test:bridge": "bash scripts/docker-run.sh test-bridge",
    "test:quality": "bash scripts/docker-run.sh quality"
  }
}
```

B. EXTRAIT DE CODE 1 : PORTAIL DE CONNEXION STATIQUE ET ACCESSIBLE

Le portail de connexion visible dans les captures ne vit pas directement dans la SPA privée `osionos`. Il est rendu côté **Astro** dans `opposite-osiris`, parce que cette partie doit être rapide, indexable et accessible avant même que l'utilisateur n'ouvre son workspace. C'est un choix important : la page publique est statique et SEO-friendly ; l'application React privée commence après le handoff sécurisé.

Le composant `apps/opposite-osiris/src/components/ui/Portal.astro` montre cette attention à l'accessibilité : `dialog`, titre lié par `aria-labelledby`, labels associés aux champs, messages `aria-live`, boutons nommés, consentements explicites et zone anti-abus Turnstile.

```
<!-- apps/opposite-osiris/src/components/ui/Portal.astro -->
<dialog
  id="portal"
  class={`portal portal--${quick ? 'quick' : 'start'}`}
  aria-labelledby="portal-title"
  data-default-mode={quick ? 'connect' : 'start'}
>
  <h2 id="portal-title" class="visually-hidden">Prismatica workspace portal</h2>
  <button class="portal__close" type="button" aria-label="Close portal" data-close-portal></button>

  <section class="portal__panel portal__panel--login" aria-label="Secure connection panel">
    <form class="portal-login" novalidate>
      <label for="portal-email">Email <span aria-hidden="true">*</span></label>
      <input id="portal-email" name="email" type="email" autocomplete="email"
inputmode="email" required />
      <p id="portal-email-inline-error" class="field-validation-message" aria-live="polite">
        We verify the email format before sending it.
      </p>

      <div class="turnstile-box" data-turnstile-widget aria-label="Anti-abuse verification"></div>
      <output id="portal-error-msg" class="portal-error" role="status" aria-live="polite" aria-atomic="true"></output>
    </form>
  </section>
</dialog>
```

La validation côté client est portée par [apps/opposite-osiris/src/hooks/useAuth.ts](#). Elle vérifie l'email, la complexité du mot de passe, le token anti-abus, puis appelle la gateway avec `credentials: 'include'` et un retry contrôlé sur les réponses [429](#).

```
// apps/opposite-osiris/src/hooks/useAuth.ts
export const RFC_5322_EMAIL_REGEX = new RegExp(String.raw`${EMAIL_LOCAL_PART}@(?:${EMAIL_DOMAIN_LABEL}\.)+[A-Za-z]{2,63}$`);
export const STRONG_PASSWORD_REGEX = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[^\A-Za-z0-9]).{8,}$/;

function validationMessage(request: AuthRequest, mode: AuthMode): string | null {
  if (!validateEmail(request.email)) return 'Use a valid email address.';
  if (mode === 'register' && !validatePassword(request.password)) {
    return 'Password must be at least 8 characters and include uppercase, lowercase, number, and symbol.';
  }
  if (mode === 'login' && request.password.length === 0) return 'Enter your password.';
  if (!request.turnstileToken) return 'Complete the anti-abuse check.';
  return null;
}
```

L'accessibilité n'est pas limitée au portail. Dans l'application React, les primitives communes portent aussi des comportements clavier : [apps/osionos/app/src/shared/ui/primitives/Modal.tsx](#) gère `role="dialog"`, `aria-modal`, `Escape`, le focus initial, le focus trap et la restauration du focus ; [apps/osionos/app/src/shared/ui/primitives/Dropdown.tsx](#) implémente `combobox` / `listbox` avec navigation clavier.

```
// apps/osionos/app/src/shared/ui/primitives/Modal.tsx
<div
  ref={dialogRef}
  role="dialog"
  aria-modal="true"
  aria-labelledby={title ? titleId : undefined}
  aria-describedby={description ? descriptionId : undefined}
  tabIndex={-1}
>
  {title ? <h2 id={titleId} className="sr-only">{title}</h2> : null}
  {description ? <p id={descriptionId} className="sr-only">{description}</p> : null}
  {children}
</div>
```

Enfin, le responsive design repose sur des tokens CSS et des valeurs fluides plutôt que sur une pile de breakpoints. [apps/osionos/app/src/pages/notion-page/ui/notionPage.css](#) utilise `clamp()` pour garder une lecture confortable sur petits et grands écrans.

```
/* apps/osionos/app/src/pages/notion-page/ui/notionPage.css */
.osionos-page-header,
.osionos-page-properties,
.osionos-page-body {
  max-width: var(--page-content-max-width, 900px);
  width: 100%;
  min-width: 0;
  margin-left: auto;
  margin-right: auto;
  padding-left: var(--page-content-padding-inline, clamp(16px, 11%, 96px));
  padding-right: var(--page-content-padding-inline, clamp(16px, 11%, 96px));
}
```

Extraits de code, partie dynamique

A. AUTHENTIFICATION : SESSION PRISMATICA, BRIDGE SÉCURISÉ ET FALLBACK OFFLINE

L'authentification côté Osionos tient en deux temps : le site Astro (`opposite-osiris`) authentifie l'utilisateur, puis l'application `osionos` consomme une session de bridge signée (un jeton à usage unique remis par le site Astro, qui permet à l'app de récupérer la session de l'utilisateur sans redemander ses identifiants). Si aucun bridge n'est disponible et que le mode offline est autorisé, l'application démarre avec des données seedées pour permettre le développement local.

Dans `apps/osionos/app/src/features/auth/model/userStore.helpers.ts`, le token de bridge est lu depuis l'URL, envoyé à l'API, puis retiré immédiatement de la barre d'adresse pour éviter qu'il reste dans l'historique visible.

```
// apps/osionos/app/src/features/auth/model/userStore.helpers.ts
export async function consumeBridgeSessionFromLocation(): Promise<BridgeSessionImport | null> {
  const token = bridgeTokenFromLocation();
  if (!token || !API_BASE) return null;
  const response = await fetch(`${API_BASE}/api/auth/bridge/consume`, {
    method: 'POST',
    headers: { Accept: 'application/json', 'Content-Type': 'application/json' },
    credentials: 'include',
    body: JSON.stringify({ token }),
  });
  if (!response.ok) throw new Error('Bridge session could not be imported.');
```

```
  const payload = await response.json() as BridgeSessionImport;
  clearBridgeTokenFromLocation();
  return payload;
}
```

Le store Zustand `apps/osionos/app/src/features/auth/model/useUserStore.ts` centralise l'état utilisateur, les sessions, les workspaces actifs et le fallback offline. Il contient un garde-fou contre le double appel de `init()` en `StrictMode`, ce qui est indispensable avec React 19.

```
// apps/osionos/app/src/features/auth/model/useUserStore.ts
let _initInProgress = false;

export const useUserStore = create<UserStore>((set, get) => ({
  personas: uniquePersonas([...INITIAL_PERSONAS.map(p => ({ ...p })), ...readPersistedPe
rsonas()]),
  sessions: {},
  activeUserId: '',
  initialized: false,
  loading: false,
  error: null,

  init: async () => {
    if (get().initialized || _initInProgress) return;
    _initInProgress = true;
    set({ loading: true, error: null });

    try {
      set(await resolveInitialState());
    } catch {
      set(bridgeOnlyMode() ? bridgeSessionRequiredState() : offlineState());
    } finally {
      _initInProgress = false;
    }
  },
}));
```

Le client API commun [apps/osionos/app/src/shared/api/client.ts](#) ajoute le JWT seulement quand il existe et transforme les erreurs HTTP en `ApiError` typées. C'est une petite couche, mais elle évite que chaque composant reconstruise sa propre logique `fetch`.

```
// apps/osionos/app/src/shared/api/client.ts
async function request<T>(method: string, path: string, body?: unknown, jwt?: string): P
romise<T> {
  if (!API_BASE) throw new Error("VITE_API_URL is not configured.");

  const headers: Record<string, string> = { 'Content-Type': 'application/json' };
  if (jwt) headers['Authorization'] = `Bearer ${jwt}`;

  const res = await fetch(`${API_BASE}${path}`, {
    method,
    headers,
    body: body == null ? undefined : JSON.stringify(body),
  });

  if (!res.ok) {
    const errorBody = await res.json().catch(() => null) as ApiErrorBody | null;
    throw new ApiError(errorBody?.error ?? errorBody?.message ?? `${method} ${path} → $
{res.status} ${res.statusText}`, res.status);
  }
  if (res.status === 204) return undefined as T;
  return res.json() as Promise<T>;
}
```

B. RÉCUPÉRATION DES DONNÉES : WORKSPACES, PAGES ET CONTENU COMPLET

La récupération dynamique porte sur les **workspaces**, les **pages** et le **contenu complet d'une page** — le flux central d'Osionos. L'utilisateur ouvre un espace, choisit une page, et l'application ne charge que ce qu'il faut pour l'afficher ou l'éditer.

Dans `apps/osionos/app/src/store/pageStore.actions.ts`, `fetchPages` vérifie d'abord le JWT utilisable par l'API pages, puis le contexte d'accès courant. Si l'utilisateur n'appartient pas au workspace demandé, la requête ne part même pas. Cette vérification front ne remplace pas la sécurité côté BaaS, mais elle évite une mauvaise UX et coupe les appels inutiles.

```
// apps/osionos/app/src/store/pageStore.actions.ts
export function createFetchPages(set: SetFn, get: GetFn) {
  return async (workspaceId: string, jwt: string) => {
    const pageJwt = pageApiJwtFromSessionToken(jwt);
    if (!pageJwt) return;
    const context = getCurrentPageAccessContext();
    if (context && !context.workspaceIds.includes(workspaceId)) return;
    if (get().loadingIds.has(workspaceId)) return;
    set((s) => ({ loadingIds: new Set([...s.loadingIds, workspaceId]) }));
    try {
      const data = await api.get<PageEntry[]>(`
        /api/pages/all?workspaceId=${workspaceId}`,
        pageJwt,
      );
      set((s) => ({
        ...derivePageState({
          ...s.pages,
          [workspaceId]: ../mergeWorkspacePages(s.pages[workspaceId], data),
        }, s.pageIdsByWorkspace),
        loadingIds: new Set([...s.loadingIds].filter((id) => id !== workspaceId)),
      }));
      savePagesCache(get().pages, workspaceId);
    } catch {
      set((s) => ({ loadingIds: new Set([...s.loadingIds].filter((id) => id !== workspaceId)) }));
    }
  };
}
```

Le contenu complet d'une page est chargé à la demande. `apps/osionos/app/src/widgets/page-renderer/ui/MainContent.tsx` ne fetch que si la page active est une vraie page, que le JWT existe et que le contenu n'est pas déjà dans le store.

```
// apps/osionos/app/src/widgets/page-renderer/ui/MainContent.tsx
useEffect(() => {
  if (!activePage || activePage?.kind !== "page" || !jwt) return;
  const page = pageById(activePage.id);
  if (!page) {
    fetchPageContent(activePage.id, jwt);
  }
}, [activePage, jwt, pageById, fetchPageContent]);
```

La fonction appelée côté store vérifie ensuite que la page existe, que l'utilisateur peut la lire, puis fusionne les champs revenus de l'API dans l'état local.

```
// apps/osionos/app/src/store/pageStore.actions.ts
export function createFetchPageContent(set: SetFn, get: GetFn) {
  return async (pageId: string, jwt: string) => {
    const pageJwt = pageApiJwtFromSessionToken(jwt);
    if (!pageJwt || !isPersistedPageId(pageId)) return;
    const page = get().pageById(pageId);
    const context = getCurrentPageAccessContext();
    if (!page || !canReadPage(page, context)) return;
    try {
      const fullPage = await api.get<PageEntry>(`/api/pages/${pageId}`, pageJwt);
      if (!fullPage) return;
      set((s) => ({
        ...derivePageState(updatePageInState(s.pages, pageId, (p) => ({
          ...p,
          content: fullPage.content ?? p.content,
          title: fullPage.title ?? p.title,
          icon: fullPage.icon ?? p.icon,
          cover: fullPage.cover ?? p.cover,
          updatedAt: fullPage.updatedAt ?? p.updatedAt,
        })), s.pageIdsByWorkspace),
      })));
      savePagesCache(get().pages, page.workspaceId);
    } catch (err) {
      console.warn("[pageStore] fetchPageContent failed:", pageId, err);
    }
  };
}
```

C. ACTIONS MÉTIER CRITIQUES : ARCHIVER, SUPPRIMER, VERROUILLER, TRADUIRE, RESTAURER

Les actions métier critiques d'Osionos : **archiver**, **supprimer**, **dupliquer** une page, **changer ses permissions implicites**, **verrouiller l'édition**, **traduire son contenu** ou **restaurer une version**. Visibles côté utilisateur, elles touchent aussi l'état local, la persistance et parfois les descendants de la page.

Avant une action dangereuse, [apps/osionos/app/src/features/page-management/ui/PageOptionsMenu.tsx](#) vérifie le contexte d'accès local via `canDeletePage` ou `canDuplicatePage`, puis appelle le store. Là encore, ce garde-fou ne remplace pas le backend : il protège l'interface et évite de proposer des actions incohérentes.

```
// apps/osionos/app/src/features/page-management/ui/PageOptionsMenu.tsx
const handleDuplicateClick = async (e: React.MouseEvent) => {
  e.stopPropagation();
  setIsMenuOpen(false);
  if (!workspaceId) return;
  if (!currentPage || !canDuplicatePage(currentPage, getCurrentPageAccessContext())) return;

  try {
    await duplicatePage(pageId, workspaceId);
  } catch (err) {
    console.error("[PageOptionsMenu] Failed to duplicate page", err);
  }
};

const handleConfirmDelete = async () => {
  if (!workspaceId) return;
  if (!currentPage || !canDeletePage(currentPage, getCurrentPageAccessContext())) return;
  await deletePage(pageId, workspaceId, jwt ?? "");
  redirectIfAffectedPageChanged();
};
```

Les règles d'accès front sont centralisées dans [apps/osionos/app/src/shared/lib/auth/pageAccess.ts](#), au lieu d'être recopiées dans chaque composant.

```
// apps/osionos/app/src/shared/lib/auth/pageAccess.ts
export function canReadPage(page: PageEntry, context: PageAccessContext | null): boolean {
  if (!context || !hasWorkspaceAccess(page, context)) return false;

  const visibility = normalizePageVisibility(page.visibility);
  if (visibility === "public") return true;
  if (visibility === "shared") return true;
  if (page.ownerId && page.ownerId === context.userId) return true;
  if (isLegacyPage(page)) return true;

  return getCollaboratorRole(page, context.userId) !== null;
}

export function canEditPage(page: PageEntry, context: PageAccessContext | null): boolean {
  if (!context || !hasWorkspaceAccess(page, context)) return false;
  if (context.sharedWorkspaceIds.includes(page.workspaceId)) return true;
  if (page.ownerId && page.ownerId === context.userId) return true;
  if (isLegacyPage(page)) return true;
  const collaboratorRole = getCollaboratorRole(page, context.userId);
  return collaboratorRole === "editor" || collaboratorRole === "owner";
}
```

L'archivage montre bien la logique métier : on patch le backend quand un JWT existe, puis on met à jour localement la page et tous ses descendants, en nettoyant aussi les pages récentes. C'est dans [apps/osionos/app/src/store/pageStore.actions.ts](#).

```
// apps/osionos/app/src/store/pageStore.actions.ts
export function createArchivePage(set: SetFn, get: GetFn) {
  return async (pageId: string, workspaceId: string, jwt: string) => {
    const page = get().pageById(pageId);
    const context = getCurrentPageAccessContext();
    if (!page || !canDeletePage(page, context)) return;

    const archivedAt = new Date().toISOString();
    const pageJwt = pageApiJwtFromSessionToken(jwt);

    if (pageJwt && isPersistedPageId(pageId)) {
      try {
        await api.patch(`/api/pages/${pageId}`, { archivedAt }, pageJwt);
      } catch {
        /* silent */
      }
    }

    set((s) => {
      const wsPages = s.pages[workspaceId] ?? [];
      const descendantIds = getAllDescendantIds(wsPages, pageId);
      const archivedIds = new Set([pageId, ...descendantIds]);
      const newRecents = s.recents.filter((r) => !archivedIds.has(r.id));
      const pages = {
        ...s.pages,
        [workspaceId]: [...wsPages.map((p) => archivedIds.has(p._id) ? { ...p,
archivedAt } : p),
      ];
      return { ...derivePageState(pages, s.pageIdsByWorkspace), recents: newRecents };
    });
    savePagesCache(get().pages, workspaceId);
  };
}
```

Les actions de page plus avancées sont regroupées dans [apps/osionos/app/src/entities/page/model/usePageActions.ts](#). Ce hook gère le compteur de mots, les versions automatiques, la traduction, l'import/export, les notifications, le mode présentation et le verrouillage de page.

```
// apps/osionos/app/src/entities/page/model/usePageActions.ts
const toggleLock = useCallback(
  () => updatePageSetting(
    { locked: !config.locked },
    'lock_page',
    config.locked ? 'Page unlocked' : 'Page locked',
  ),
  [config.locked, updatePageSetting],
);

const translate = useCallback(async (targetLocale = translateLocale) => {
  if (!page || !pageId) return;
  const label = translationLabel(targetLocale);
  await snapshot(`Before translation to ${label}`);
  const translated = await translatePage(page, jwt ?? undefined, targetLocale);
  if (translated.title) updatePageTitle(pageId, translated.title);
  if (translated.content) updatePageContent(pageId, translated.content);
  await logAction('translate', `Page translated to ${label}`, { targetLocale });
}, [jwt, logAction, page, pageId, snapshot, translateLocale, updatePageContent, updatePageTitle]);
```

Accessibilité, sécurité front et qualité mesurable

ACCESSIBILITÉ INTÉGRÉE AUX COMPOSANTS

L'accessibilité est visible à plusieurs niveaux du code : un lien d'évitement sur la page Astro, des boutons nommés, des tabs avec `aria-selected`, des breadcrumbs avec `aria-current`, un éditeur `contentEditable` annoncé comme textbox multiligne, et des modales avec focus trap.

```
<!-- skip-link : src/layouts/Layout.astro · announcer + main : src/pages/index.astro -->
<a href="#main-content" class="skip-link">Skip to main content</a>
<div aria-live="polite" aria-atomic="true" class="visually-hidden" id="global-announcer"></div>
<main id="main-content" class="swipe-stack" data-swipe-stack>
  ...
</main>
```

```
// apps/osionos/app/src/widgets/sidebar/ui/SidebarTopNav.tsx
<div role="tablist" aria-label="Sidebar navigation">
  {tabs.map((tab) => (
    <button
      key={tab.id}
      type="button"
      role="tab"
      aria-selected={tab.active}
      aria-label={tab.label}
      title={tab.label}
    >
      <span className="flex shrink-0 items-center opacity-80">{tab.icon}</span>
      <span className={tab.active ? 'ml-1.5 truncate' : 'sr-only'}>{tab.label}</span>
    </button>
  ))}
</div>
```

```
// apps/osionos/app/src/components/blocks/EditableContent.tsx
<div
  ref={ref}
  role="textbox"
  aria-multiline="true"
  tabIndex={0}
  contentEditable
  suppressContentEditableWarning
  spellCheck
  data-placeholder={hasFocus ? placeholder : ""}
  onInput={handleInput}
  onKeyDown={handleKeyDown}
  onPaste={handlePaste}
/>
```

SÉCURITÉ CÔTÉ RENDU ET NAVIGATION

Le front n'est jamais la couche finale de sécurité : la vraie barrière reste côté BaaS. Le moteur Markdown `markengine` échappe le texte HTML et filtre les schémas d'URL dangereux (le détail de `escapeHtml` / `sanitizeUrl` et son test anti-XSS sont au **chapitre 5 — Sécurité**) ; côté rendu, la valeur ajoutée propre au front est d'ajouter `rel="noopener noreferrer"` sur les liens externes via le rendu des liens.

```
// apps/osionos/app/src/shared/lib/markengine/markdown/renderers/inlineHtml.ts
function renderLink(node: Extract<InlineNode, { type: "link" }>, options: ResolvedInline
HtmlOptions): string {
  const href = sanitizeUrl(node.href);
  const attrs = [
    `href=".${esc(href || "#")}"`,
    options.externalLinks && isExternalUrl(href) ? `target="_blank" rel="noopener
noreferrer" : "",
  ].filter(Boolean).join(" ");
  return `<a ${attrs}>${renderChildren(node.children, options)}</a>`;
}
```

Le composant de coloration syntaxique `apps/osionos/app/src/shared/ui/molecules/CodeSyntaxHighlight/CodeSyntaxHighlight.tsx` utilise `dangerouslySetInnerHTML`, mais seulement après échappement manuel pour les langages inconnus et après `highlight.js` pour les langages enregistrés.

```
// apps/osionos/app/src/shared/ui/molecules/CodeSyntaxHighlight/CodeSyntaxHighlight.tsx
function escapeHtml(value: string) {
  return value
    .replaceAll("&", "&amp;")
    .replaceAll("<", "&lt;")
    .replaceAll(">", "&gt;")
    .replaceAll("'", "&quot;");
}

if (!hljs.getLanguage(normalized)) {
  return escapeHtml(code);
}
```

Enfin, le site public Astro applique une CSP stricte en production via `security.csp` dans `apps/opposite-osiris/astro.config.mjs` — Astro auto-hashe les `<script>` qu'il émet — avec `object-src 'none'`, `base-uri 'self'`, Trusted Types (mécanisme du navigateur qui force tout HTML/script dynamique à passer par une fonction de nettoyage avant d'être injecté dans la page) et `require-trusted-types-for 'script'`.

```
// apps/opposite-osiris/astro.config.mjs — security.csp.directives
"default-src 'self'",
"base-uri 'self'",
"object-src 'none'",
"form-action 'self'",
"connect-src 'self' https:",
"trusted-types prismatic-static-markup",
"require-trusted-types-for 'script'",
// scriptDirective : 'self' + https://challenges.cloudflare.com (scripts auto-hashés par Astro)
```

TROIS CHOIX QUI STRUCTURENT LE FRONT

Trois décisions résument ce chapitre. D'abord, deux frontends séparés par rôle : un site Astro statique (référéncable, accessible) pour l'entrée publique, une application React/Vite riche pour l'espace de travail privé. Ensuite, une authentification en deux temps — le site authentifie, puis remet à l'app un jeton de bridge à usage unique, avec repli sur des données locales hors-ligne. Enfin, un éditeur qui ne perd jamais le travail : chaque modification passe par une file (outbox) renvoyée au serveur avec relance jusqu'à confirmation, même après une coupure réseau.

CHAPITRE 4. Les réalisations personnelles, back-end

Ce chapitre présente la partie serveur que j'ai réellement construite ou intégrée. Le back-end n'est pas un seul serveur monolithique : c'est une plateforme composée de briques spécialisées. **Kong** joue le rôle de passerelle, **GoTrue** gère l'authentification, **PostgREST** expose PostgreSQL en REST, **MongoDB** sert les données documentaires, **MinIO** stocke les fichiers, **realtime-agnostic** diffuse les changements, et les services **NestJS** portent la logique que nous maîtrisons directement : `mongo-api`, `query-router`, `schema-service`, `permission-engine`, `storage-router`, `session-service`, `gdpr-service`, `log-service`, `email-service`, `newsletter-service`, `analytics-service` et `ai-service`. Au-delà de ces services NestJS, le back-end s'est doté d'un **plan de données Rust** (`data-plane-router`) qui exécute le CRUD multi-moteur, et d'un **plan de contrôle Go** (6 binaires : `adapter-registry`, `tenant-control`, `orchestrator`, `webhook-dispatcher`, `function-scheduler`, `scale-seed`). La bascule TypeScript→Rust suit la discipline *shadow* → *parité* → *cutover* (on fait tourner l'ancien et le nouveau en parallèle, on vérifie qu'ils donnent le même résultat, puis on bascule). Par prudence, le code laisse cette bascule désactivée par défaut ; le `docker-compose` réellement déployé, lui, l'active. Le chemin réellement servi est `/query/v1` (le `query-router` NestJS résout le montage via l'`adapter-registry` puis **forwarde l'exécution** au plan de données Rust, effectif sur le stack servi) ; la porte directe `/data/v1` existe en parallèle et est elle aussi active dans le compose. À noter : `adapter-registry` est passé en **Go** pour de bon, son équivalent TypeScript ayant été retiré après preuve de parité.

La logique générale est simple : **le navigateur ne connaît que des API HTTP**, et les services internes ne se parlent pas par import de code, mais par **réseau Docker**, avec des URLs de service (`http://adapter-registry-go:3021` , `http://permission-engine:3050` , `mongo:27017` , `postgres:5432`) et des jetons internes quand il faut franchir une limite de confiance.

Architecture de l'API et modèle de données

A. ARCHITECTURE DE L'API RESTFUL

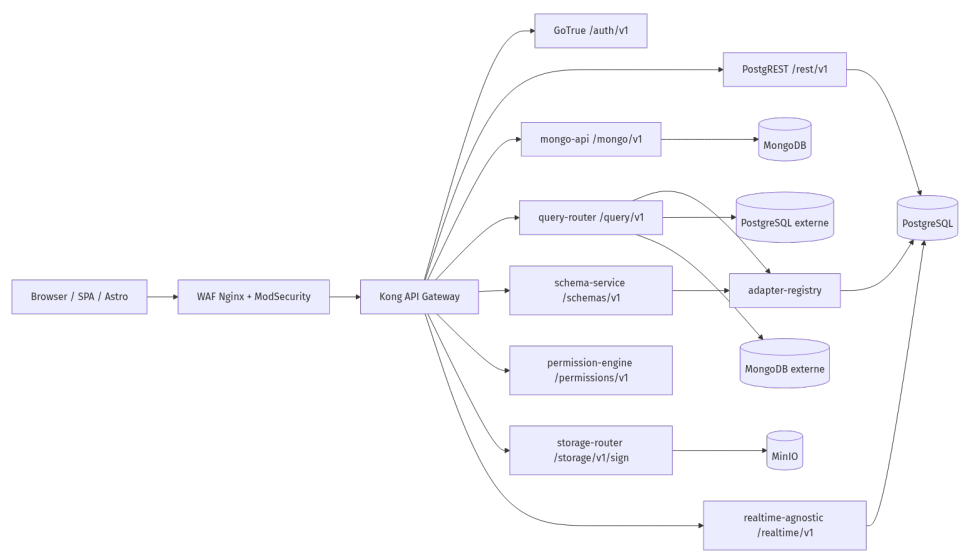
Une API RESTful expose des **ressources** (`users` , `posts` , `databases` , `schemas` , `collections` , `pages`) et laisse les verbes HTTP exprimer l'intention : `GET` pour lire, `POST` pour créer ou déclencher une opération, `PATCH` pour modifier partiellement, `DELETE` pour supprimer. Elle est aussi **stateless** : chaque requête porte son identité dans un JWT, une clé API ou un token de service ; le serveur n'a pas besoin de garder une session applicative en mémoire pour comprendre la requête.

Dans notre projet, l'API est RESTful dans son usage concret : les routes sont structurées par ressources, les services NestJS utilisent des contrôleurs HTTP, PostgREST expose directement les tables PostgreSQL sous `/rest/v1` , et Kong applique les mêmes couches transverses à l'entrée. On reste pragmatique sur un point : toute l'API n'est pas une implémentation REST "académique" avec stratégie HTTP cache complète ; les caches vérifiés sont surtout applicatifs (`query-router` avec TTL local/Redis, cache front, et persistance locale côté Osionos). Mais la séparation client/serveur, l'absence d'état de session serveur et l'usage uniforme des ressources HTTP sont bien là.

Entrée publique	Service interne	Ressource principale	Rôle
<code>/auth/v1/*</code>	GoTrue	utilisateurs, sessions, OAuth	inscription, connexion, JWT
<code>/rest/v1/<table></code>	PostgREST	tables PostgreSQL	CRUD REST protégé par RLS
<code>/mongo/v1/collections/:name/documents</code>	<code>mongo-api</code>	collections MongoDB	CRUD document owner-scoped (chaque utilisateur ne voit/modifie que ses propres documents)
<code>/admin/v1/databases</code>	<code>adapter-registry</code>	bases enregistrées	stockage chiffré des connexions
<code>/query/v1/:dbId/tables/:table</code>	<code>query-router</code>	table ou collection distante	exécution normalisée multi-moteur
<code>/schemas/v1/schemas</code>	<code>schema-service</code>	table ou collection créée	DDL (créer/modifier des tables) contrôlé et enregistré
<code>/permissions/v1/permissions/check</code>	<code>permission-engine</code>	rôles et politiques ABAC (autorisation selon des attributs, pas seulement le rôle)	décision d'autorisation
<code>/storage/v1/sign/:bucket/*</code>	<code>storage-router</code>	objet MinIO/S3	URL présignée avec préfixe utilisateur

Entrée publique	Service interne	Ressource principale	Rôle
<code>/realtime/v1</code>	<code>realtime-agnostic</code>	événements DB	WebSocket / CDC (<i>Change Data Capture</i> : capter les changements de la base et les diffuser)

Le flux d'une requête ressemble à ceci :



Kong est aussi le point où l'identité devient exploitable par les services internes. La configuration [kong.yml](#) vérifie les JWT, applique `key-auth`, le rate limiting, les limites de payload, CORS, les headers de sécurité, puis injecte des headers de confiance (`X-User-Id` , `X-User-Email` , `X-User-Role`). Les services NestJS ne revalident donc pas chacun le JWT : ils lisent l'identité déjà validée par la passerelle.

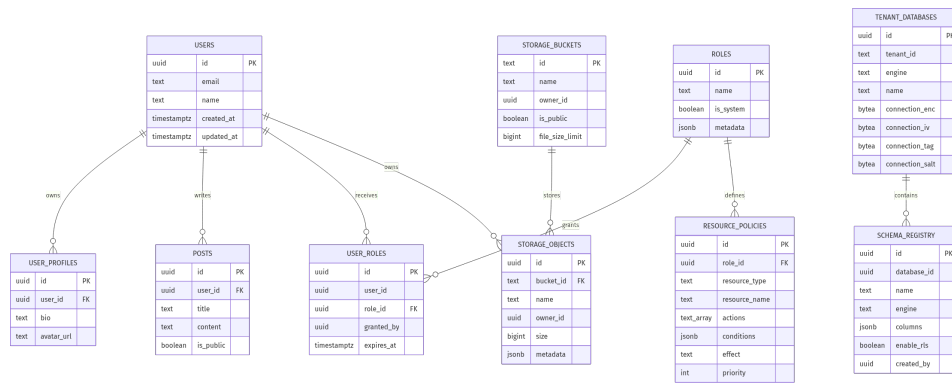
```
# infra/docker/services/kong/conf/kong.yml
- name: rest
  url: http://postgrest:3000
  routes:
    - name: rest-routes
      paths: [/rest/v1]
      strip_path: true
      plugins:
        - name: key-auth
          config:
            key_names: [apikey]
            hide_credentials: false
        - name: jwt
          config:
            header_names: [authorization]
            key_claim_name: iss
            claims_to_verify: [exp]
            run_on_preflight: false
            anonymous: __KONG_ANON_UUID__
        - name: rate-limiting
          config:
            policy: local
            limit_by: ip
            minute: 180
            hour: 5000
```

Cet extrait est **représentatif** : le `kong.yml` réel fait aujourd'hui ~1200 lignes et déclare **40 services routés**. Au-delà de `/rest/v1`, il route le plan de données Rust (`/data/v1` → `data-plane-router-rust` , ouvert par `DATA_PLANE_BYPASS_ENABLED=1` selon la discipline *shadow* → *cutover* décrite en tête de chapitre), le plan de contrôle Go (`/admin/v1/` {`provision,tenants,keys,webhooks,migrate,rotate`}), `/functions/v1` , `/sql` (Trino) et `/studio` .

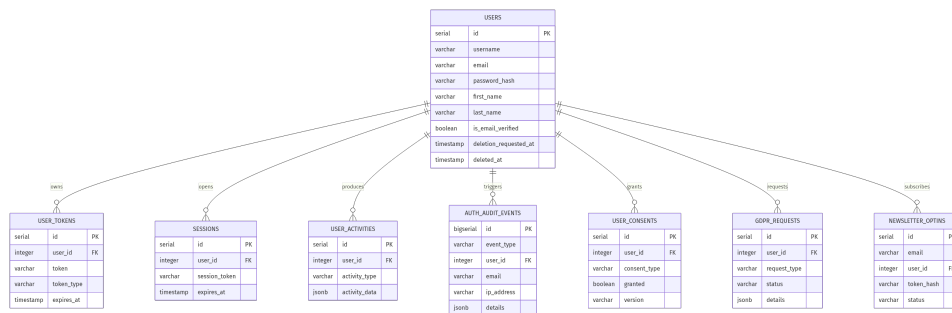
B. SCHÉMA CONCEPTUEL DE DONNÉES

La difficulté du projet, c'est qu'il n'y a pas une seule base de données. Il y a un **socle relationnel** pour l'identité, les rôles, les permissions, les registres et les données structurées ; il y a un **modèle Osionos** orienté workspace/pages ; et il y a un **plan document / multi-engine** pour les collections dynamiques et les bases enregistrées par l'utilisateur.

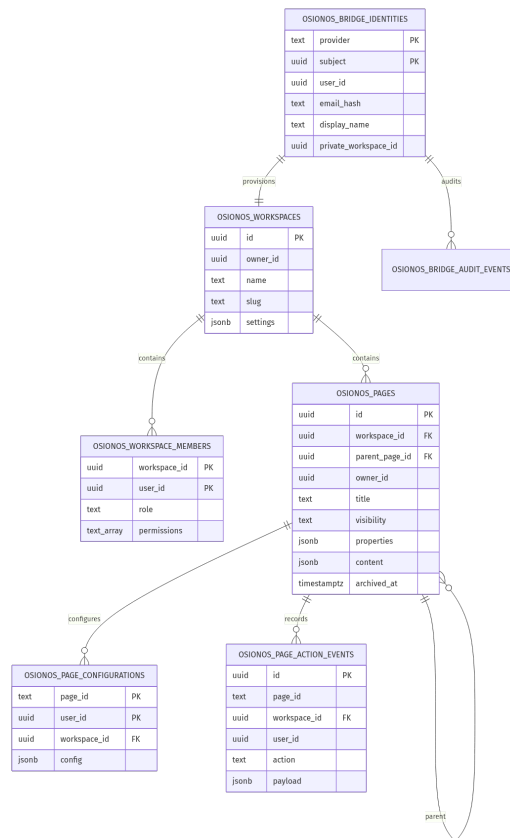
Le premier schéma représente le cœur BaaS : les utilisateurs, les contenus de démonstration, les rôles, les politiques, les bases enregistrées, les schémas créés et les objets de stockage.



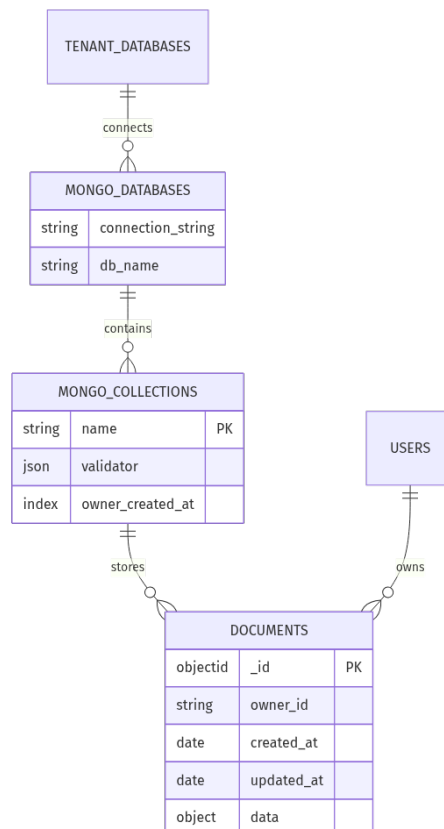
Le second schéma est celui utilisé par le profil **track-binocle** / Prismatic / opposite-osiris. Il est volontairement relationnel : un compte possède des tokens temporaires, des sessions, des activités, des consentements et des demandes RGPD. Les fichiers qui définissent ce modèle sont [models/user.sql](#), [models/auth-security-migration.sql](#) et [models/gdpr-migration.sql](#). (schéma applicatif osionos — défini dans le monorepo Track-Binocle, hors du dépôt grobase autonome)



Le troisième schéma décrit la partie Osionos. Le navigateur manipule des pages et des workspaces ; le backend conserve la correspondance durable entre l'identité Prismatic, le workspace privé, les pages, les configurations par utilisateur et les événements d'action. Cette partie est définie dans [models/osionos-bridge-migration.sql](#) (schéma applicatif osionos du monorepo Track-Binocle).



Enfin, le plan MongoDB est plus souple : il ne cherche pas à figer toutes les formes de documents à l'avance. Les collections créées par **schema-service** reçoivent un validateur JSON Schema, un index sur **owner_id**, et les opérations de **mongo-api** ou **query-router** injectent ou filtrent systématiquement par propriétaire.



C. SCHÉMA PHYSIQUE DE DONNÉES (SPD) ET SCRIPTS SQL

Le modèle physique est matérialisé par deux familles de scripts.

La première famille est le socle BaaS dans `scripts/migrations/postgresql` : création de `auth.uid()`, tables système, RLS, registre d'adapters, ABAC, stockage et triggers realtime.

```
-- scripts/migrations/postgresql/001_initial_schema.sql
CREATE OR REPLACE FUNCTION auth.uid() RETURNS UUID AS $$
  SELECT (current_setting('request.jwt.claims', true)::jsonb->>'sub')::uuid;
$$ LANGUAGE SQL STABLE;

CREATE TABLE IF NOT EXISTS public.posts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES public.users(id) ON DELETE CASCADE,
  title TEXT NOT NULL,
  content TEXT,
  is_public BOOLEAN DEFAULT false,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

ALTER TABLE public.posts ENABLE ROW LEVEL SECURITY;

CREATE POLICY posts_select ON public.posts
  FOR SELECT USING (is_public OR auth.uid()::text = user_id::text);
```

Le registre des bases externes est un point sensible : il contient les chaînes de connexion vers des bases utilisateur. Jamais en clair. Le stockage physique conserve le ciphertext (le texte chiffré), l'IV (vecteur d'initialisation, un aléa qui rend deux chiffrements identiques différents), le tag GCM (preuve que le contenu n'a pas été altéré) et le sel (aléa qui durcit la dérivation de la clé).

```
-- scripts/migrations/postgresql/004_add_adapter_registry.sql
CREATE TABLE IF NOT EXISTS public.tenant_databases (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID NOT NULL,
  engine TEXT NOT NULL CHECK (engine IN ('postgresql', 'mongodb', 'mysql', 'redis', 'sqlite')),
  name TEXT NOT NULL,
  connection_enc BYTEA NOT NULL,
  connection_iv BYTEA NOT NULL,
  connection_tag BYTEA NOT NULL,
  created_at TIMESTAMPTZ DEFAULT now(),
  last_healthy_at TIMESTAMPTZ,
  UNIQUE(tenant_id, name)
);

ALTER TABLE public.tenant_databases ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_databases_owner_crud ON public.tenant_databases
  FOR ALL USING (auth.uid()::text = tenant_id::text)
  WITH CHECK (auth.uid()::text = tenant_id::text);
```

Le modèle de permissions est physique lui aussi. Les rôles et les politiques sont en base, et `permission-engine` appelle la fonction SQL `has_permission()` pour prendre une décision reproductible.

```
-- scripts/migrations/postgresql/007_permissions_system.sql
CREATE TABLE IF NOT EXISTS public.resource_policies (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  role_id     UUID NOT NULL REFERENCES public.roles(id) ON DELETE CASCADE,
  resource_type TEXT NOT NULL,
  resource_name TEXT NOT NULL,
  actions     TEXT[] NOT NULL DEFAULT ARRAY['select'],
  conditions  JSONB DEFAULT '{}'::jsonb,
  effect      TEXT NOT NULL DEFAULT 'allow' CHECK (effect IN ('allow', 'deny')),
  priority    INTEGER DEFAULT 0
);

CREATE OR REPLACE FUNCTION public.has_permission(
  p_user_id UUID,
  p_resource_type TEXT,
  p_resource_name TEXT,
  p_action TEXT
) RETURNS BOOLEAN AS $fn$
DECLARE
  pol RECORD;
  found BOOLEAN := false;
BEGIN
  FOR pol IN
    SELECT rp.effect, rp.conditions
    FROM public.resource_policies rp
    JOIN public.user_roles ur ON ur.role_id = rp.role_id
    WHERE ur.user_id = p_user_id
      AND (rp.resource_type = p_resource_type OR rp.resource_type = '*')
      AND (rp.resource_name = p_resource_name OR rp.resource_name = '*')
      AND p_action = ANY(rp.actions)
    ORDER BY rp.priority DESC, rp.effect ASC
  LOOP
    IF pol.effect = 'deny' THEN
      RETURN false;
    END IF;
    found := true;
  END LOOP;

  RETURN found;
END;
$fn$ LANGUAGE plpgsql STABLE SECURITY DEFINER;
```

La deuxième famille de scripts est spécifique aux applications : [models/user.sql](#) pour le modèle utilisateur relationnel, [models/auth-security-migration.sql](#) pour l'audit d'authentification, [models/gdpr-migration.sql](#) pour les consentements et demandes RGPD, et [models/osionos-bridge-migration.sql](#) pour les workspaces/pages Osionos (modèle applicatif du monorepo Track-Binocle).

```
-- models/osionos-bridge-migration.sql
CREATE TABLE IF NOT EXISTS public.osionos_pages (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  workspace_id UUID NOT NULL REFERENCES public.osionos_workspaces(id) ON DELETE CASCADE,
  parent_page_id UUID REFERENCES public.osionos_pages(id) ON DELETE SET NULL,
  owner_id UUID,
  title TEXT NOT NULL DEFAULT 'Untitled',
  visibility TEXT NOT NULL DEFAULT 'private' CHECK (visibility IN ('private', 'shared',
'public')),
  collaborators JSONB NOT NULL DEFAULT '[]'::jsonb,
  properties JSONB NOT NULL DEFAULT '[]'::jsonb,
  content JSONB NOT NULL DEFAULT '[]'::jsonb,
  archived_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

ALTER TABLE public.osionos_pages ENABLE ROW LEVEL SECURITY;

CREATE POLICY osionos_pages_update_member ON public.osionos_pages
FOR UPDATE TO authenticated USING (
  EXISTS (
    SELECT 1 FROM public.osionos_workspace_members member
    WHERE member.workspace_id = public.osionos_pages.workspace_id
      AND member.user_id = auth.uid()
      AND member.permissions && ARRAY['update', 'admin']::TEXT[]
  )
);
```

D. EXTRAIT DU SCRIPT SQL DE CRÉATION ET COHÉRENCE

La cohérence des données est assurée à plusieurs niveaux, pas seulement par le code applicatif.

1. Les clés étrangères évitent les données orphelines (`ON DELETE CASCADE` pour profils, tokens, sessions, pages enfant de workspace).
2. Les contraintes `CHECK` limitent les états possibles (`visibility` , `role` , `engine` , `effect`).
3. Les index matérialisent les requêtes critiques (`workspace_id` , `parent_page_id` , `updated_at` , `owner_id`).
4. La RLS impose l'isolation même si une route applicative se trompe.
5. Les triggers realtime installés globalement permettent de propager les changements sans écrire un trigger à la main pour chaque future table.

L'extrait suivant montre ce dernier point : la migration `012_realtime_triggers_all_tables.sql` installe automatiquement un trigger `AFTER INSERT OR UPDATE OR DELETE` sur les tables existantes et futures.

```

CREATE OR REPLACE FUNCTION public.realtime_notify()
RETURNS TRIGGER AS $fn$
DECLARE
    payload JSON;
BEGIN
    payload := json_build_object(
        'table',      TG_TABLE_NAME,
        'schema',     TG_TABLE_SCHEMA,
        'operation',  TG_OP,
        'data',       CASE WHEN TG_OP = 'DELETE' THEN row_to_json(OLD) ELSE row_to_json(NEW)
    END,
        'old_data',   CASE WHEN TG_OP = 'UPDATE' THEN row_to_json(OLD) ELSE NULL END
    );

    PERFORM pg_notify('realtime_events', payload::text);
    RETURN COALESCE(NEW, OLD);
END;
$fn$ LANGUAGE plpgsql SECURITY DEFINER;

CREATE EVENT TRIGGER realtime_auto_trigger_on_create
ON ddl_command_end
WHEN TAG IN ('CREATE TABLE')
EXECUTE FUNCTION public.realtime_auto_trigger();

```

Côté diffusion, le plan realtime (Rust) applique l'optimisation symétrique : quand un évènement part vers des centaines d'abonnés WebSocket, on ne le sérialise **qu'une seule fois**. L' `EventEnvelope` mémorise son fragment JSON dans un `Arc<OnceLock<String>>` (`envelope.rs` — `rendered_payload_json()`), partagé par tous les abonnés via le clone de l' `Arc` ; chaque connexion n'échappe plus que son propre `sub_id` avant d'écrire la trame. Le résultat est byte-identique à une re-sérialisation par connexion (test de non-régression dans `writer.rs`), pour une fraction du coût CPU sous forte charge.

Extrait de code, structure et sécurité de l'API

A. CHOIX TECHNIQUES, CONTEXTE ET LOGIQUE

NestJS a été choisi pour les services qui demandent une logique applicative claire : validation DTO, injection de dépendances, contrôleurs REST, guards, Swagger, logs structurés et healthchecks. Les services partagent des librairies internes (`@mini-baas/common` , `@mini-baas/database`) mais restent déployables séparément grâce au Dockerfile multi-app.

Le bootstrap d'un service comme `query-router` montre la structure commune : validation stricte, filtre d'erreurs homogène, correlation-id, Swagger, arrêt propre.

```
// src/apps/query-router/src/main.ts
async function bootstrap() {
  const app = await NestFactory.create(AppModule, { bufferLogs: true });

  app.useLogger(app.get(PinoLogger));
  app.useGlobalPipes(createValidationPipe());
  app.useGlobalFilters(new AllExceptionsFilter());
  app.useGlobalInterceptors(new CorrelationIdInterceptor());
  app.enableShutdownHooks();

  const swaggerConfig = new DocumentBuilder()
    .setTitle('Query Router')
    .setDescription('Universal data plane – routes queries to registered databases')
    .setVersion('2.0.0')
    .build();

  const config = app.get(ConfigService);
  const port = config.get<number>('PORT', 4001);

  await app.listen(port);
}
```

La validation est stricte : un champ non attendu dans un DTO déclenche une erreur `400` au lieu d'être silencieusement accepté.

```
// src/libs/common/src/pipes/validation.pipe.ts
export function createValidationPipe(): NestValidationPipe {
  return new NestValidationPipe({
    whitelist: true,
    forbidNonWhitelisted: true,
    transform: true,
    transformOptions: { enableImplicitConversion: true },
  });
}
```

L'identité utilisateur est fournie par Kong puis lue par `AuthGuard`.

```
// src/libs/common/src/guards/auth.guard.ts
@Injectable()
export class AuthGuard implements CanActivate {
  canActivate(context: ExecutionContext): boolean {
    const req = context.switchToHttp().getRequest<Request>();
    const identity = resolveRequestIdentity(req, true);
    if (!identity) throw new UnauthorizedException('Missing verified identity');
    req.identity = identity;
    req.user = identityToUserContext(identity, req.headers['x-user-email'] as string | undefined);
    return true;
  }
}
```

Les appels internes sensibles, par exemple `query-router` qui demande à `adapter-registry` de déchiffrer une connexion, ne passent pas par un JWT utilisateur classique. Ils utilisent un token de service et un `X-Tenant-Id` explicite. Le service appelé reconstitue alors un contexte `service_role` limité au tenant demandé.

```
// src/libs/common/src/guards/service-token.guard.ts
if (serviceToken && expectedToken && timingSafeStringEqual(serviceToken,
expectedToken)) {
  const serviceId = (req.headers['x-service-id'] as string | undefined) ?? 'internal-
service';
  const identity = serviceIdentityFromHeaders(req, serviceId);
  req.identity = identity;
  req.user = identityToUserContext(identity, 'service@internal');
  return true;
}
```

B. EXTRAIT 1 : CONTRÔLE D'OWNERSHIP ET RÈGLE D'HABILITATION CRITIQUE

Le contrôle d'ownership est doublé à dessein : **PostgreSQL le fait avec la RLS**, et **MongoDB le fait avec un filtre `owner_id` injecté dans les requêtes**. Ce double modèle est nécessaire parce que PostgreSQL sait appliquer une politique au niveau ligne, alors que MongoDB demande de le faire dans la couche applicative.

Côté PostgreSQL, les requêtes tenant passent par `tenantQuery()`. La méthode ouvre une transaction, pose la variable locale `app.current_user_id`, exécute la requête, puis commit ou rollback. Les politiques SQL peuvent alors comparer `owner_id` ou `tenant_id` à cette valeur.

```
// src/libs/database/src/postgres/postgres.service.ts
async tenantQuery<T extends QueryResultRow = Record<string, unknown>>(
  identityOrUserId: TenantQueryContext | string,
  text: string,
  params?: unknown[],
): Promise<T[]> {
  const identity = this.resolveTenantQueryContext(identityOrUserId);
  const client = await this.tenantPool.connect();
  try {
    await client.query('BEGIN');
    await client.query(
      `SELECT set_config('app.current_tenant_id',$1,true),
set_config('app.current_user_id',$2,true), set_config('request.jwt.claims',$3,true)`,
      [
        identity.tenantId,
        identity.userId,
        JSON.stringify({
          sub: identity.userId,
          tenant_id: identity.tenantId,
          project_id: identity.projectId,
          app_id: identity.appId,
          role: identity.role,
          scopes: identity.scopes ?? [],
        }),
      ],
    );
    const result = await client.query<T>(text, params);
    await client.query('COMMIT');
    return result.rows;
  } catch (err) {
    await client.query('ROLLBACK');
    throw err;
  } finally {
    client.release();
  }
}
```

Côté MongoDB, le service retire les champs interdits (`_id` , `owner_id`) fournis par le client, injecte son propre `owner_id` , et ajoute ce propriétaire dans tous les `find` , `patch` et `delete` .

```
// src/apps/mongo-api/src/collections/collections.service.ts
async create(collectionName: string, userId: string, data: Record<string, unknown>) {
  const { _id: __, owner_id: __, ...clean } = data;

  const doc = {
    ...clean,
    owner_id: userId,
    created_at: new Date(),
    updated_at: new Date(),
  };

  const col = this.getCollection(collectionName);
  const result = await col.insertOne(doc);

  return this.normalizeDoc({ _id: result.insertedId, ...doc });
}

async patch(collectionName: string, userId: string, docId: string, patch:
Record<string, unknown>) {
  const { _id: __, owner_id: __, ...clean } = patch;

  const result = await col.findOneAndUpdate(
    { _id: new ObjectId(docId), owner_id: userId },
    { $set: { ...clean, updated_at: new Date() } },
    { returnDocument: 'after' },
  );
}
```

Le même principe existe dans Osionos, mais à l'échelle workspace/page. Avant de lire ou modifier une page, le bridge vérifie à la fois le token applicatif et l'appartenance au workspace dans PostgREST.

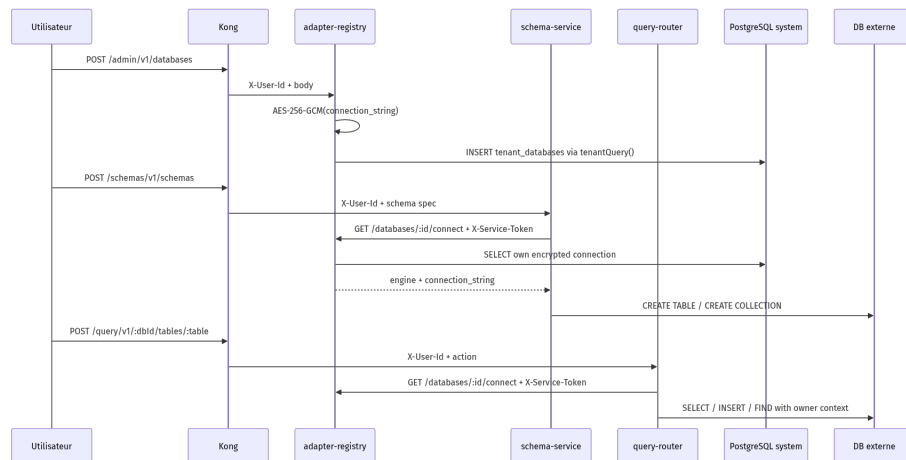
```
// apps/osionos/app/scripts/bridge-api.mjs
export async function requireWorkspaceAccess(request, workspaceId, permission, config, fetchImpl = fetch) {
  const normalizedWorkspaceId = requireUuid(workspaceId, 'workspaceId');
  const authContext = verifyAppSessionToken(bearerToken(request), config);
  if (!authContext.workspaceIds.includes(normalizedWorkspaceId)) {
    throw Object.assign(new Error('App session is not scoped to this workspace.'), { status: 403 });
  }

  const query = postgrestQuery({
    workspace_id: `eq.${normalizedWorkspaceId}`,
    user_id: `eq.${authContext.userId}`,
    select: 'role,permissions',
    limit: '1',
  });
  const rows = await baasRest(config, fetchImpl, `osionos_workspace_members?${query}`);
  const member = Array.isArray(rows) ? rows[0] : null;

  if (!memberHasPermission(member, normalizePermission(permission))) {
    throw Object.assign(new Error('Workspace permission denied.'), { status: 403 });
  }
}
```

C. EXTRAIT 2 : ACTION MÉTIER CRITIQUE, CONNECTER UNE BASE ET L'UTILISER

Une action critique du backend est la connexion d'une base externe. Cette action touche plusieurs risques : secret de connexion, ownership, moteur de base, validation de schéma, puis exécution de requête. C'est pour cela qu'elle traverse plusieurs services.



Le contrôleur REST du **query-router** expose seulement deux familles d'actions : exécuter sur une table/collection ou lister les tables/collections disponibles.

```

// src/apps/query-router/src/query/query.controller.ts
@ApiTags('query')
@Controller() // racine : Kong ajoute /query/v1 puis le retire (strip_path) ; un
@Controller('query') doublerait le segment → 404
@UseGuards(AuthGuard)
export class QueryController {
  @Post('/:dbId/tables/:table')
  async execute(
    @CurrentUser() user: UserContext,
    @Param('dbId', ParseUUIDPipe) dbId: string,
    @Param('table') table: string,
    @Body() dto: ExecuteQueryDto,
  ) {
    return this.service.executeQuery(dbId, table, user.id, dto);
  }

  @Get('/:dbId/tables')
  async listTables(@CurrentUser() user: UserContext, @Param('dbId', ParseUUIDPipe)
  dbId: string) {
    return this.service.listTables(dbId, user.id);
  }
}

```

Le **query-router** ne connaît jamais directement les secrets de connexion stockés. Il les demande au registre via HTTP interne, avec un token de service.

```
// src/apps/query-router/src/query/query.service.ts
private async fetchConnectionFromRegistry(dbId: string, userId: string): Promise<Adapter
Response> {
  const path = `/databases/${dbId}/connect`;
  const { data } = await firstValueFrom(
    this.http.get<AdapterResponse>(`${this.registryUrl}${path}`, {
      headers: { ...serviceAuthHeaders(this.serviceToken, 'GET', path, ''), 'X-Tenant-
Id': userId },
    )),
  );
  return data;
}
```

Le registre chiffre au moment de l'enregistrement, puis déchiffre seulement pour les appels autorisés. Le registre TS d'origine a été retiré au profit du service Go `adapter-registry-go` ; le chiffrement (AES-256-GCM + dérivation `scrypt`, schéma sel/IV/tag byte-compatible avec l'ancienne disposition de colonnes Node) vit désormais dans `crypto.go`.

```
// src/control-plane/internal/adapterregistry/crypto.go
func (e *Encryptor) Encrypt(plaintext string) (EncryptedPayload, error) {
  // AES-256-GCM + scrypt (N=16384, r=8, p=1) ; sel/IV/tag séparés
  // pour rester compatible avec la disposition de colonnes héritée.
}
```

Pour PostgreSQL, la validation des noms de tables/colonnes, le paramétrage des valeurs, l'injection de `owner_id` à l'insert et la pose du contexte RLS suivent toujours ce principe. Le moteur TS d'origine du `query-router` (`engines/postgresql.engine.ts`) a été retiré : l'exécution concrète passe désormais par le plan de données Rust (`src/data-plane-router`, validation des identifiants via une liste blanche `quote_ident`, owner-scoping par requête), et la même paire de regex de validation DDL vit aujourd'hui dans `schema-service`.

```
// src/apps/schema-service/src/engines/postgres-schema.engine.ts:17,19
// liste blanche stricte des identifiants DDL (table/colonne), bornée à 64 caractères
const TABLE_REGEX = /^[a-zA-Z_]\w{0,63}$/;
const COLUMN_REGEX = /^[a-zA-Z_]\w{0,63}$/;
```

Le contexte d'isolation (`BEGIN` + `set_config('app.current_user_id', ..., true)` + `COMMIT`) et l'estampillage `owner_id` à l'écriture sont posés par requête dans le plan de données — voir l'extrait `postgres.service.ts` (`tenantQuery`) plus haut et l'owner-scoping Rust de `src/data-plane-router`.

Pour MongoDB, le moteur applique le filtre propriétaire, limite les résultats et **rejette** (il ne supprime pas en silence) les constructions dangereuses comme `$where` : toute clé préfixée `$` ou contenant un point déclenche une erreur `400`, récursivement jusque dans les objets imbriqués.

Ce comportement est implémenté dans `mongo-api/collections.service.ts` :

`assertNoMongoOperators` (lignes 109-122) rejette par une `400` toute clé préfixée `$` ou contenant un point, récursivement ; `assertSafeFieldName` (lignes 97-107) valide les noms de champs ; et la lecture owner-scopée (`findAll`, lignes 157-196) injecte le filtre `owner_id` à chaque requête. L'extrait concret de cette méthode owner-scopée figure plus bas (§ extrait DAO MongoDB).

D. EXTRAIT 3 : ACTION MÉTIER OSIONOS, CRÉER ET MODIFIER UNE PAGE

Osionos a un backend plus léger, écrit en Node natif dans `bridge-api.mjs`. (*application osionos — définie dans le monorepo Track-Binocle, hors du dépôt grobase autonome*) Son rôle est de recevoir une assertion signée depuis Prismatica, créer une session applicative courte, puis servir des routes REST pour les pages. C'est ici que l'on voit le lien réel entre le front riche et le BaaS.

La première barrière est HMAC (Hash-based Message Authentication Code : une signature calculée avec un secret partagé, qui prouve que le message vient bien de l'émetteur et n'a pas été modifié) : Prismatica signe le payload avec un secret partagé, le bridge vérifie le timestamp, la signature et le `jti` pour éviter le rejeu (un attaquant qui rejouerait une requête déjà signée et interceptée).

```
// apps/osionos/app/scripts/bridge-api.mjs
export function verifyBridgeRequest({ headers, payload, secret, now = Date.now(), replay
Store = new Map() }) {
  const timestampHeader = headers['x-prismatic-bridge-timestamp'];
  const signatureHeader = headers['x-prismatic-bridge-signature'];
  const timestamp = Number(timestampHeader);
  if (!Number.isFinite(timestamp) || Math.abs(now - timestamp) > DEFAULT_TIMESTAMP_SKEW_
MS) {
    throw Object.assign(new Error('Bridge assertion timestamp is outside the allowed
window.'), { status: 401 });
  }

  const normalizedPayload = validateBridgePayload(payload);
  const expected = bridgeSignature(secret, String(timestampHeader), normalizedPayload);
  if (typeof signatureHeader !== 'string' || !safeCompareHex(expected,
signatureHeader)) {
    throw Object.assign(new Error('Bridge signature is invalid.'), { status: 401 });
  }

  if (replayStore.has(normalizedPayload.jti)) {
    throw Object.assign(new Error('Bridge assertion replay rejected.'), { status: 409 });
  }
  replayStore.set(normalizedPayload.jti, { expiresAt: now + DEFAULT_TIMESTAMP_SKEW_MS });
  return normalizedPayload;
}
```

Ensuite, les routes pages restent REST : `GET /api/pages`, `POST /api/pages`, `PATCH /api/pages/:id`, `DELETE /api/pages/:id`. Chaque écriture repasse par `requireWorkspaceAccess()`.

```
// apps/osionos/app/scripts/bridge-api.mjs
async function handlePageUpdate(url, request, response, config, fetchImpl) {
  const pageId = pageIdFromPath(url.pathname);
  if (!pageId) return false;
  const existing = await fetchPageRow(pageId, config, fetchImpl);
  if (!existing) throw Object.assign(new Error('Page not found.'), { status: 404 });
  await requireWorkspaceAccess(request, existing.workspace_id, 'update', config, fetchImpl);

  const payload = await readJson(request, PAGE_JSON_BODY_LIMIT_BYTES);
  const updateRow = pageUpdateRowFromPayload(payload);
  const rows = await baasRest(config, fetchImpl, `osionos_pages?id=eq.${pageId}`, {
    method: 'PATCH',
    body: updateRow,
    prefer: 'return=representation',
  });

  json(response, 200, pageRowToEntry(Array.isArray(rows) ? rows[0] : rows), config);
  return true;
}
```

Extraits de code de composants d'accès aux données (DAO)

A. CHOIX TECHNIQUES, CONTEXTE ET LOGIQUE

Dans ce projet, je n'ai pas créé une couche de repositories figés comme dans un back-end CRUD classique. Le besoin était plus large : il fallait parler à PostgreSQL, MongoDB, MinIO, PostgREST et à des bases externes enregistrées dynamiquement. Le rôle de DAO est donc porté par des **services d'accès aux données** et des **engines** :

- **PostgresService** : pool admin + pool tenant avec contexte RLS.
- **MongoService** : client MongoDB partagé, pool, healthcheck.
- **adapter-registry-go** (Go) : registre des bases et chiffrement AES-256-GCM des connexions (l'ancien **DatabasesService** TypeScript a été retiré).
- **QueryService** : orchestration entre utilisateur, adapter-registry et plan d'exécution.
- **data-plane-router** (Rust) : exécution concrète des opérations (les moteurs TS **PostgresqlEngine** / **MongodbEngine** ont été retirés).
- **SchemasService** : création des tables/collections à partir d'un schéma unifié.

Ce choix explique aussi pourquoi nous n'avons pas retenu Prisma comme ORM principal. Prisma est excellent quand le modèle relationnel est stable, connu à l'avance et majoritairement PostgreSQL/MySQL. Ici, une partie du produit repose sur des **schémas créés par l'utilisateur**, des **bases externes enregistrées au runtime**, une exécution **PostgreSQL + MongoDB**, et une dépendance forte à la **RLS** et aux variables de session SQL (`SET LOCAL app.current_user_id`). Un client généré statiquement aurait été moins adapté. Le coût de ce choix, c'est qu'on perd une partie du confort type-safe d'un ORM ; on compense par des DTO stricts, des regex de noms d'identifiants, des requêtes paramétrées, des polices SQL et des tests ciblés.

Il faut aussi être honnête sur la couverture : le plan de données Rust route aujourd'hui **postgresql**, **cockroachdb**, **mongodb**, **mysql**, **mariadb**, **redis**, **sqlite**, **mssql** et **http** (cutover live, parité prouvée via **parity-probe.sh**), mais l'étendue des fonctionnalités varie selon le moteur.

B. EXTRAIT 1 : RÉCUPÉRATION DE DONNÉES OWNER-SCOPED

La récupération de documents MongoDB ne dépend pas d'un filtre envoyé par le front. Même si le client envoie un filtre, le service ajoute `owner_id = userId` et retire les champs qui ne doivent pas être contrôlés par le client.

```
// src/apps/mongo-api/src/collections/collections.service.ts
async findAll(
  collectionName: string,
  userId: string,
  opts: { limit: number; offset: number; sort?: string; filter?: string },
) {
  const col = this.getCollection(collectionName);
  let query: Record<string, unknown> = { owner_id: userId };

  if (opts.filter) {
    try {
      query = { ...query, ...this.parseFilter(opts.filter) };
    } catch (error) {
      if (error instanceof BadRequestException) throw error;
      throw new BadRequestException('Invalid JSON in filter parameter');
    }
  }

  let sort: Sort = { created_at: -1 };
  if (opts.sort) {
    const [field, dir] = opts.sort.split(':');
    if (field && dir) {
      sort = { [field]: dir.toLowerCase() === 'asc' ? 1 : -1 };
    }
  }

  const [data, total] = await Promise.all([
    col.find(query).sort(sort).skip(opts.offset).limit(opts.limit).toArray(),
    col.countDocuments(query),
  ]);

  return { data: data.map((d) => this.normalizeDoc(d as Record<string, unknown>)),
    meta: { total, limit: opts.limit, offset: opts.offset } };
}
```

C. EXTRAIT 2 : CRÉATION D'UN SCHÉMA UTILISABLE PAR PLUSIEURS MOTEURS

`schema-service` est un bon exemple de service métier backend : il ne se contente pas de faire un `CREATE TABLE`. Il vérifie que le moteur demandé correspond à la base enregistrée, crée la structure côté moteur, puis écrit une trace dans `schema_registry`.

```
// src/apps/schema-service/src/schemas/schemas.service.ts
async create(userId: string, dto: CreateSchemaDto) {
  const { engine, connection_string } = await this.fetchConnection(dto.database_id, userId);

  if (engine !== dto.engine) {
    throw new BadRequestException(
      `Engine mismatch - database is ${engine} but schema spec says ${dto.engine}`,
    );
  }

  if (engine === 'postgresql') {
    const result = await this.pgEngine.createTable(
      connection_string,
      dto.name,
      dto.columns,
      dto.enable_ri != false,
    );

    await this.pg.adminQuery(
      `INSERT INTO schema_registry (database_id, name, engine, columns, enable_ri,
created_by)
VALUES ($1, $2, $3, $4::jsonb, $5, $6)
ON CONFLICT (database_id, name) DO UPDATE SET columns = $4::jsonb, enable_ri =
$5`,
      [dto.database_id, dto.name, engine, JSON.stringify(dto.columns), dto.enable_ri !=
false, userId],
    );

    return result;
  }
}
```

La partie PostgreSQL ajoute automatiquement `id`, `owner_id`, `created_at`, `updated_at`, puis installe une policy `owner_isolation` si `enable_ri` est actif.

```
// src/apps/schema-service/src/engines/postgres-schema.engine.ts
const colDefs: string[] = [
  `id UUID PRIMARY KEY DEFAULT gen_random_uuid()`,
  `owner_id UUID NOT NULL`,
  `created_at TIMESTAMPTZ DEFAULT now()`,
  `updated_at TIMESTAMPTZ DEFAULT now()`,
];

await client.query(`ALTER TABLE public."${tableName}" ENABLE ROW LEVEL SECURITY`);
await client.query(
  `DO $$ BEGIN
    IF NOT EXISTS (SELECT 1 FROM pg_policies WHERE tablename = '${tableName}' AND
    polycyname = 'owner_isolation') THEN
      CREATE POLICY owner_isolation ON public."${tableName}" FOR ALL
      USING (owner_id::text = auth.current_user_id()::text)
      WITH CHECK (owner_id::text = auth.current_user_id()::text);
    END IF;
  END $$`,
);
```

La partie MongoDB crée ou met à jour un validateur JSON Schema et un index utile aux requêtes owner-scoped.

```
// src/apps/schema-service/src/engines/mongo-schema.engine.ts
const properties: Record<string, unknown> = {
  owner_id: { bsonType: 'string' },
  created_at: { bsonType: 'date' },
  updated_at: { bsonType: 'date' },
};

if (existing.length) {
  await db.command({ collMod: collectionName, validator, validationLevel: 'strict' });
} else {
  await db.createCollection(collectionName, { validator });
  await db.collection(collectionName).createIndex({ owner_id: 1, created_at: -1 });
}
```

D. PRÉPARATION AU DÉPLOIEMENT, ORCHESTRATION ET RÉCUPÉRATION

Le déploiement est préparé avec Docker Compose et un Dockerfile multi-stage. L'idée n'est pas de construire une image différente à la main pour chaque service NestJS : le même Dockerfile reçoit `ARG APP`, compile seulement l'application demandée, supprime les dépendances de développement, puis exécute le service avec un utilisateur non-root.

```

# src/Dockerfile
FROM public.ecr.aws/docker/library/node:${NODE_VERSION}-alpine AS deps
WORKDIR /app
COPY --link package.json package-lock.json ./
RUN --mount=type=cache,target=/root/.npm,sharing=locked \
    npm ci --ignore-scripts --prefer-offline --no-audit --no-fund

FROM deps AS build
ARG APP
COPY --link tsconfig.json tsconfig.build.json nest-cli.json ./
COPY --link libs/ ./libs/
COPY --link apps/${APP}/ ./apps/${APP}/
RUN npx nest build ${APP}

FROM public.ecr.aws/docker/library/node:${NODE_VERSION}-alpine AS runtime
ARG APP
ENV NODE_ENV=production APP_NAME=${APP}
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser
CMD ["sh", "-c", "node dist/apps/${APP_NAME}/apps/${APP_NAME}/src/main.js"]

```

Compose orchestre les dépendances avec `depends_on`, `healthcheck`, `restart: unless-stopped`, des volumes persistants, des limites CPU/mémoire et des profils (`data-plane`, `control-plane`, `adapter-plane`, `storage`, `observability`). Exemple avec `query-router` : il ne démarre que si `adapter-registry` et `permission-engine` sont en bonne santé, et il parle aux autres services par DNS Docker.

```
# orchestrators/compose/base/app-services.yml
query-router:
  build:
    context: ./src
    dockerfile: Dockerfile
    args:
      APP: query-router
  environment:
    PORT: 4001
    ADAPTER_REGISTRY_URL: http://adapter-registry-go:3021
    PERMISSION_ENGINE_URL: http://permission-engine:3050
    QUERY_ROUTER_REDIS_URL: redis://redis:6379
    ADAPTER_REGISTRY_SERVICE_TOKEN: ${ADAPTER_REGISTRY_SERVICE_TOKEN}
  depends_on:
    adapter-registry-go:
      condition: service_healthy
    data-plane-router-rust:
      condition: service_healthy
    permission-engine:
      condition: service_healthy
    redis:
      condition: service_started
  networks:
    - mini-baas
  restart: unless-stopped
  healthcheck:
    test: ["CMD-SHELL", "wget -qO- http://localhost:4001/health/live || exit 1"]
```

Là encore, extrait **représentatif** : le `docker-compose.yml` réel résout aujourd'hui **~47 services** dans les profils principaux (et ~64 une fois activés tous les profils déclarés, dont les apps de démonstration `vendor/` et le lakehouse). Le registre d'adapters est désormais le service Go `adapter-registry-go:3021`, et le `query-router` dépend aussi du plan de données Rust (`data-plane-router-rust`) vers lequel il forward le CRUD multi-moteur.

L'overlay de production `docker-compose.prod.yml` retire les ports directs des bases (`postgres`, `mongo`, `gotrue`, `postgrest`, `redis`) et garde l'accès via les services prévus. Cela limite la surface d'exposition : en production, la base n'est pas censée être appelée directement depuis l'extérieur. Les limites de ressources ne sont plus posées ici : elles utilisent la forme courte `mem_limit / cpus` dans le compose de base (l'overlay a été réécrit pour ne plus dupliquer `deploy.resources.*`, qui entrerait en conflit).

```
# orchestrators/compose/docker-compose.prod.yml
postgres:
  ports: []

mongo:
  ports: []
```

Si un serveur applicatif tombe, Compose peut le redémarrer grâce aux healthchecks et aux politiques `restart`. Si `mongo-api` tombe, les documents ne disparaissent pas : ils sont dans le volume `mongo-data`. Si `query-router` tombe, il perd ses caches mémoire, mais les données restent dans PostgreSQL,

MongoDB ou la base externe. Si `postgres` redémarre, le volume `postgres-data` conserve les données. Si `realtime-agnostic` redémarre, les prochains changements repartent depuis la base ; la base reste la source de vérité.

Cette configuration Compose tient pour un environnement local, de démonstration ou un petit déploiement, mais ce n'est pas encore une haute disponibilité multi-noeud. Il n'y a pas de failover automatique PostgreSQL multi-réplicas dans ce fichier. Pour une production critique, il faudrait ajouter une stratégie de backup planifiée, un stockage externe, des replicas, une supervision d'alerting et des procédures de restauration testées.

Les scripts de backup/restore existent déjà pour PostgreSQL et MongoDB. Ils montrent la direction opérationnelle : `pg_dump` en format custom pour PostgreSQL, `mongodump` en archive pour MongoDB, puis restauration explicite.

```
# infra/docker/services/postgres/tools/backup.sh
BACKUP_FILE="backup_$(date +%Y%m%d).dump"
docker compose exec postgres pg_dump -U postgres -Fc > "${BACKUP_FILE}"

# infra/docker/services/postgres/tools/restore.sh
docker compose exec -T postgres pg_restore -U postgres -d postgres < "${BACKUP_FILE}"

# infra/docker/services/mongo/tools/backup.sh
BACKUP_FILE="mongo_backup_$(date +%Y%m%d).archive"
docker compose exec mongo mongodump --archive > "${BACKUP_FILE}"
```

Les secrets ne sont pas intégrés aux images. Le profil `control-plane` contient Vault, et les scripts d'environnement récupèrent ou génèrent les valeurs nécessaires sans les écrire en clair dans le code source. Le script `ensure-osionos-runtime-secrets.mjs` génère par exemple les secrets du bridge Osionos en local, tandis que `vault-env.mjs` centralise les familles de variables attendues pour les services.

E. LE BAAS COMME PRODUIT : ARCHITECTURE TROIS PLANS, COÛTS MESURÉS, VIABILITÉ

L'idée derrière l'architecture

Le BaaS a dépassé son rôle de « back-end d'Osionos » : l'idée structurante est de **mettre chaque responsabilité dans le langage qui lui coûte le moins cher** — la répartition en trois plans (Rust pour le chemin chaud, Go pour le contrôle, TypeScript pour l'orchestration historique en retrait) est rappelée en tête de chapitre. Ce qui compte ici, c'est que ce choix est **mesuré**, pas esthétique (`cost-analysis.md`, artefacts `footprint-*.json`) :

Plan	Langage	RAM mesurée par processus
Plan de données (<code>data-plane-router-rust</code>)	Rust	11,5 MiB (l'équivalent Node : 127 MiB)
Realtime (<code>realtime-agnostic</code>)	Rust	~18 MiB
Plan de contrôle (gotrue, adapter-registry, tenant-control...)	Go	7-59 MiB
Orchestration (query-router, permission-engine, log-service...)	Node	46-84 MiB chacun

Le chemin de données qui tournait dans 127 MiB de Node tourne aujourd'hui dans **11,5 MiB de Rust** — **~11× plus léger et 5× plus rapide** (requête chaude ~2 ms). Et comme un hébergeur facture la RAM (~5 \$/Go/mois chez Fly.io), **chaque MiB économisé est de l'argent**.

La stratégie : mesurer, shadower, ne jamais supprimer sans preuve

Quatre mouvements, chacun mesuré avant/après — jamais de chiffre sans artifact, jamais de réécriture big-bang :

- 1. Réécrire le chemin chaud en Rust, en *shadow* → *parité* → *cutover*** (discipline décrite en tête de chapitre) : la bascule n'a eu lieu qu'après le gate de parité (m36). Gain : -127 MiB par déploiement, latence ÷5, zéro risque de régression pris.
- 2. Consolider l'orchestration Node en Go (R2).** Six services Node consolidés dans **un seul binaire Go**, pour réduire l'empreinte mémoire et le coût d'hébergement. Après sa phase *shadow* (fidèle à la discipline), il a été **basculé live** (cutover A4 du 2026-06-13, `ORCHESTRATOR_PRODUCT_MODE=enabled`) : Kong route désormais `/logs`, `/sessions`, `/newsletter`, `/gdpr`, `/email` vers ce binaire Go, les six orchestrateurs Node étant mis en quarantaine dans le profil `legacy-node`.
- 3. Construire des éditions plancher.** Le même plan Rust, compilé en statique avec features gâtées, donne **binocle-nano** : un binaire de **5,16 Mo, 2,1 MiB de RAM**, SQLite in-process — CRUD, graph, clés scopées, SSE. Mesuré tête-à-tête contre PocketBase sur la même machine : **inserts 3,8× plus rapides à 1/26° de la RAM** (et une défaite assumée, documentée : PocketBase garde 1,27× sur le débit de lecture en liste). Coût d'hébergement : **~2 \$/mois, < 1 \$ à l'arrêt** (scale-to-zero : *la machine s'éteint quand personne ne l'utilise et ne facture plus que le stockage*).
- 4. Prouver la densité multi-tenant** (*plusieurs clients* — « tenants » — *isolés sur la même instance partagée, sans que l'un voie les données de l'autre*). Un run réel à **10 000 tenants** a invalidé notre propre hypothèse (les pools allaient bien ; le vrai mur était la vérification de clés Argon2id (*un algorithme de hachage volontairement lent et gourmand en mémoire, conçu pour les mots de passe — trop coûteux ici pour des clés API*) qui saturait un service plafonné en mémoire). Deux correctifs mesurés : un hash adapté aux clés à haute entropie (chemin froid 263 → 45 ms) et le partage de pools pour les tenants `shared_rls` — l'isolation étant portée par la requête (RLS, owner-scoping), pas par le pool, ce qui a été **prouvé neutre en live sur tous les moteurs** (gate m46 : deux tenants sur un pool partagé, zéro fuite). Résultat : **10 000 tenants → 1 pool, zéro 5xx** (`server_errors:0`, relevé dans le bench `multitenant-10000-sharepools.json` — non committé dans ce dépôt autonome, reproductible via `scripts/verify/m46-share-pools-isolation.sh`). Le nombre de pools est désormais indépendant du nombre de tenants — la propriété qui permet d'amortir un nœud à **moins de 1 \$/tenant/mois**.

Ce que ça coûte, par forme de déploiement

Chaque tier est une forme réelle et reproductible (`make up PACKAGE=<tier>`), chaque chiffre est mesuré en live et gardé par un gate de régression (m32) :

Forme	RAM mesurée	Coût infra (Fly.io)	Pour quoi
nano / one	2,1 MiB · 1 binaire	~2 \$/mois (< 1 \$ idle)	une app privée, classe PocketBase, sans Docker
basic	~460 MiB · 11 services	~6 \$/mois	CRUD Rust sans Node, SQLite + PostgreSQL

Forme	RAM mesurée	Coût infra (Fly.io)	Pour quoi
essential	~950 MiB · 19 services	~13 \$ → 6,5 \$ post-R2	un produit complet (agrégats, orchestration)
pro	~1,4 GiB · 28 services	~21 \$/mois	multi-engine + realtime + storage, < 1 \$/tenant amorti
max	~3,1 GiB · 41 services	~41 \$/mois	plateforme multi-tenant, analytics, sécurité max

Les offres elles-mêmes ont été **critiquées puis reconstruites** ([offer-sheet-v2.md](#)) : la v1 avait des rate-limits inventés et un plan gratuit aliasé sur le tier le plus cher ; la v2 dérive chaque rps d'un benchmark de capacité et différencie les tiers par **capacité fonctionnelle**, pas seulement par débit.

Alors, un BaaS comme celui-ci est-il viable en production ?

La réponse honnête est : **oui, par formes — et pas encore pour tout.**

Viable aujourd'hui : l'app privée mono-utilisateur ou mono-équipe (`nano` / `basic` , la classe PocketBase — et PocketBase fait tourner de vraies productions avec moins que ça) ; le produit unique mono-tenant (`essential` , ~1 Go, backups + RLS + secrets Vault) ; et la densité multi-tenant est **prouvée à 10 000 tenants réels** sur une machine, zéro 5xx. Le chemin de données Rust sert déjà le trafic réel en cutover, parité démontrée. La sécurité tient sur plusieurs couches superposées (WAF, JWT, RLS, secrets hors dépôt) détaillées au **chapitre 5 — Sécurité**, le chiffrement AES-256-GCM des credentials externes l'étant au **chapitre 4**. Chaque chiffre cité plus haut est rattaché à un gate ou un fichier de mesure reproductible.

Pas encore, et c'est documenté : la haute disponibilité multi-nœud (pas de failover PostgreSQL automatique — un déploiement critique exige des réplicas et des restaurations testées), les traces distribuées (M4), le pinning d'images par digest (plusieurs images de service restent en `:latest` comme repli GHCR ; `realtime-agnostic` est déjà épinglé à `0.2.1`), et `function-scheduler` (le seul composant Go) tourne encore en *shadow* — par choix : on ne coupe jamais avant la preuve de parité.

La marche restante vers la production critique est identifiée, chiffrée, et sur la roadmap plutôt que sous le tapis.

CHAPITRE 5. Éléments de sécurité de l'application

La sécurité, c'est la partie du projet où j'ai le plus appris à dire "je sais pas, on va vérifier". Du code qui marche, c'est facile — du code sécurisé, ça se vérifie.

L'architecture de sécurité repose sur deux services centraux : **GoTrue** (authentification, hashage bcrypt, émission des JWT — *JSON Web Token* : un jeton signé par le serveur qui prouve l'identité de l'utilisateur sans qu'il ait à renvoyer son mot de passe) et **Kong** (API gateway, vérification des JWT, contrôle CORS, injection des claims — les informations contenues dans le jeton : identité, e-mail, rôle — en headers internes). Le reste du chapitre détaille comment ces deux services s'assemblent avec les couches applicatives. Mais avant de parler d'identité, il faut sécuriser le **transport** lui-même — c'est par là que je commence.

Sécurisation du transport : HTTPS / TLS (la liaison navigateur ↔ back-end)

Avant même l'authentification, une question plus basique : **est-ce que les données circulent en clair sur le réseau ?** Non — jamais. Tout le trafic public est en **HTTPS**, donc chiffré par **TLS**, ce qui m'apporte trois garanties :

- **Confidentialité** — personne sur le réseau (un wifi public, un FAI, un proxy d'entreprise) ne peut lire le contenu : ni le mot de passe, ni le JWT, ni les données métier.
- **Intégrité** — si un seul octet est modifié en route, la connexion casse. On ne peut pas injecter de contenu en douce.
- **Authenticité du serveur** — le navigateur vérifie qu'il parle bien à *notre* serveur, et pas à un imposteur (l'attaque dite « de l'homme du milieu »).

L'image que j'utilise en soutenance : **TLS, c'est une enveloppe scellée et infalsifiable ; le certificat, c'est la carte d'identité du serveur ; et l'autorité de certification (CA), c'est le notaire** qui garantit que cette carte est authentique.

La chaîne de certificats que je génère (`generate-localhost-cert.sh`) suit exactement la logique de production :

- Je crée d'abord une **autorité de certification (CA) locale** (« Track Binocle Local Development CA ») : clé RSA **4096 bits**, certificat auto-signé en SHA-256, marqué `CA:TRUE`, `pathlen:0` et limité à la signature de certificats (`keyCertSign`, `cRLSign`). C'est mon « notaire ».
- Cette CA signe ensuite le **certificat serveur** : clé RSA **2048 bits**, usage `serverAuth` uniquement, `CA:FALSE`, valable **397 jours** (juste sous le plafond de 398 jours imposé par les navigateurs depuis sept. 2020), avec les SAN `DNS: localhost, host.docker.internal, local-https-proxy` et `IP: 127.0.0.1, ::1` — car un navigateur moderne valide le nom via les SAN, plus via le vieux champ CN.
- Les permissions sont strictes : clé privée de la CA en `600`, clé serveur en `640` (lisible seulement par le groupe du WAF). En développement, je fais confiance à ma CA en l'important dans le magasin de confiance du système et du navigateur via le script `trust-localhost-cert.sh`.

Où le TLS se termine, et comment la requête voyage — c'est le cœur de la réponse « comment la liaison front ↔ back est sécurisée » :

```

Navigateur —HTTPS (TLS 1.2/1.3)—> WAF nginx (ModSecurity + OWASP CRS)
                                | seul port public exposé (443)
                                ▼ réseau Docker privé, non joignable de
l'extérieur
                                Kong (clé d'API + JWT + rate-limit + en-têtes)
                                ▼
                                services back-end (auth, REST, données...)

```

- Le **WAF nginx est le seul point d'entrée public** — c'est écrit noir sur blanc dans `infra/docker/services/waf/conf/nginx.conf` : « *This is the ONLY public-facing listener; Kong's :8000 becomes internal.* » Il écoute en `443 ssl http2`, n'accepte que **TLS 1.2 et 1.3** (les versions anciennes et vulnérables — SSLv3, TLS 1.0/1.1 — sont refusées), puis relaie en interne vers `http://kong:8000`. C'est ce qu'on appelle la **terminaison TLS**.
- Le saut WAF → Kong se fait sur un **réseau Docker privé** : même en clair, il n'est jamais joignable depuis Internet. On centralise ainsi le déchiffrement et le filtrage en un seul endroit plutôt que de distribuer des certificats à chaque micro-service.

- Au passage, Kong **ajoute les en-têtes de sécurité du transport** via son plugin `response-transformer`. La configuration réellement chargée par la stack est `kong.yml` (le profil `kong.track-binocle.yml` n'est pas le profil actif) : elle pose un **HSTS** plus fort encore (*HTTP Strict Transport Security* : un en-tête qui ordonne au navigateur de ne plus jamais parler au site en clair) — `Strict-Transport-Security: max-age=63072000; includeSubDomains; preload` (2 ans + preload), qui force le navigateur à n'utiliser que HTTPS, plus `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy`, `Permissions-Policy`, `Cross-Origin-Opener-Policy` et `Cross-Origin-Resource-Policy`, et retire `Server` / `X-Powered-By` / `Via`. Ces en-têtes sont en plus *renforcés* au niveau applicatif par `helmet` dans chaque service NestJS.

Et en production ? Le chiffrement ne s'arrête pas au navigateur. Avec `SECURITY_MODE=max` (`SECURITY.md`), une base de données externe branchée par un client doit présenter une chaîne TLS vérifiable : `sslmode=require` est automatiquement relevé en `verify-full` (on valide réellement le certificat de la base, pas juste « c'est chiffré »), avec une CA d'entreprise possible via `DATA_PLANE_TLS_CA_FILE`. En production, ma CA locale est simplement remplacée par une **autorité publique (Let's Encrypt)** : le code et la chaîne ne changent pas, seul le signataire du certificat change. Le trajet d'une donnée est donc chiffré de bout en bout : navigateur → WAF, puis back-end → base.

Authentification et gestion des rôles

Le service d'authentification — GoTrue

On n'a pas réécrit notre propre serveur d'auth. On a choisi **GoTrue v2.188.1**, le service open-source que Supabase utilise en production. La logique : un service d'auth, c'est un truc où une erreur subtile coûte cher (timing attacks, sessions volées, etc.), alors autant prendre un projet battle-tested plutôt que de faire le malin.

Configuration dans `auth-api.yml` (GoTrue n'est plus dans le `docker-compose.yml` racine, devenu un orchestrateur fin qui `include` les fichiers de base) :

- JWT signé en **HS256** (un algorithme de signature à clé secrète unique, *symétrique* : la même clé sert à signer côté GoTrue et à vérifier côté Kong)
- Expiration de **3600 secondes** (1 heure) pour les access tokens
- Le `JWT_SECRET` est fourni à GoTrue par l'environnement runtime, généré ou récupéré via le workflow Vault/Makefile — pas en clair dans le code, pas committé. Le script qui décrit ces familles de variables est `vault-env.mjs`.

Hashage des mots de passe — bcrypt

GoTrue utilise **bcrypt** par défaut pour hasher les mots de passe avant insertion dans `auth.users`. C'est le standard de l'industrie, résistant au brute-force grâce au cost factor adaptatif. On n'a pas touché à ça — c'est exactement pour cette raison qu'on a pris GoTrue plutôt que de coder notre propre `hashPassword()` avec un `crypto.pbkdf2()` mal paramétré.

Le flow login

Concrètement, quand un utilisateur se connecte :

1. Le front React envoie `email` + `password` à `/api/auth/login` (`useAuth.ts:221-226`)
2. Le gateway intermédiaire (`auth-gateway.mjs`) valide les champs, puis appelle le SDK BaaS (`auth-gateway.mjs:859-886`)

3. Le SDK fait un POST sur GoTrue : `/auth/v1/token?grant_type=password` ([sdks/js/src/domains/auth.ts:69-70](#), via le helper de route `core/routes.ts:15`)
4. GoTrue vérifie le bcrypt, signe un JWT, renvoie `access_token` (le jeton de courte durée qui accompagne chaque requête) + `refresh_token` (le jeton de longue durée qui sert à en obtenir un nouveau sans se reconnecter)
5. Le `refresh_token` est stocké en cookie **HttpOnly + Secure + SameSite=Lax** ([auth-gateway.mjs:158-160](#)) — ça, c'est important pour résister au vol par XSS

Vérification du JWT — Kong au milieu

Plutôt que chaque microservice vérifie le JWT, c'est **Kong** (l'API gateway) qui le fait une fois pour toutes :

```
# infra/docker/services/kong/conf/kong.yml:22-26
consumers:
  - username: authenticated
    jwt_secrets:
      - key: __GOTRUE_JWT_ISS__
        secret: __JWT_SECRET__
        algorithm: HS256
```

Kong intercepte la requête, valide la signature, vérifie `exp`, puis **décodes les claims et les ré-injecte en headers** vers les microservices (plugin `pre-function`, [kong.yml:99-152](#)) :

- `X-User-Id` ← claim `sub`
- `X-User-Email` ← claim `email`
- `X-User-Role` ← claim `role`

Les microservices font confiance à ces headers dans le flux normal parce qu'ils sont derrière Kong sur le réseau Docker. Concrètement : un attaquant ne peut pas envoyer `X-User-Id: 1` directement à `mongo-api` depuis l'hôte, parce que ce port-là n'est pas mappé. Pour les services et bases qui exposent un port local en développement, l'overlay de production réduit cette surface et le contrôle d'accès doit rester porté par Kong, les guards et la base.

Gestion des rôles — RBAC + ABAC

Le système de permissions va plus loin qu'un simple RBAC. On a un **ABAC** (Attribute-Based Access Control) qui se superpose aux rôles.

Les rôles sont définis en base dans [007_permissions_system.sql:81-86](#) :

- `admin` — plateforme complète
- `user` — utilisateur standard (CRUD seulement sur ce qu'il possède)
- `guest` — lecture seule
- `moderator` — modération de contenu
- `service_role` — identité service-to-service interne

Côté NestJS, on a un `RolesGuard` qui s'applique après l'`AuthGuard`. Code réel ([roles.guard.ts](#)) :

```

@Injectable()
export class RolesGuard implements CanActivate {
  constructor(private readonly reflector: Reflector) {}

  canActivate(context: ExecutionContext): boolean {
    const requiredRoles = this.reflector.getAllAndOverride<string[]>(ROLES_KEY, [
      context.getHandler(),
      context.getClass(),
    ]);
    if (!requiredRoles?.length) return true;

    const req = context.switchToHttp().getRequest<Request>();
    const userRole = req.identity?.role ?? req.user?.role;
    const roleNames = req.identity?.roleNames ?? (userRole ? [userRole] : []);

    if (!roleNames.some((role) => requiredRoles.includes(role))) {
      throw new ForbiddenException(
        `Insufficient permissions – requires one of: ${requiredRoles.join(', ')}`,
      );
    }
    return true;
  }
}

```

Utilisation concrète ([permissions.controller.ts:56-63](#)) :

```

@Delete('roles/:userId/:roleName')
@UseGuards(RolesGuard)
@Roles('admin', 'service_role')
async revoke(...) { ... }

```

La partie ABAC — évaluation par attribut, pas par rôle brut

C'est là que ça devient intéressant à expliquer. Un RBAC classique dit “tu es `member`, tu peux `can_edit`”. Un ABAC dit “pour *cette* ressource *précise*, en fonction de *qui* tu es et de *quels attributs* s'appliquent, tu as *tel* niveau de permission”. La différence c'est qu'on peut donner `can_view` à un user spécifique sur une page, même si son rôle workspace lui donnerait normalement `can_edit`.

La règle d'accès est stockée dans le modèle MongoDB `AccessRule` ([accessRule.model.ts](#)). Chaque règle a un `target` qui peut être :

```

// target.type = 'user'      → règle sur une personne précise
// target.type = 'role'      → règle sur un rôle workspace
// target.type = 'workspace' → règle par défaut pour tout le workspace
// target.type = 'public'    → accès non-authentifié
target: {
  type: 'user' | 'role' | 'workspace' | 'public',
  userId?: ObjectId, // si type = 'user'
  role?: string,     // si type = 'role'
}

```

Et le flag `explicit: boolean` qui détermine si la règle **écrase** (true) ou **hérite** (false) des règles plus générales.

La cascade de résolution dans `engine.ts:86-134` :

```
// Toutes les règles applicables : workspace global → page spécifique
const rules = await AccessRuleModel.find({
  workspaceId,
  $and: [
    { $or: [
      { resourceId: null, resourceType: 'workspace' }, // défaut workspace
      { resourceId }, // règle sur cette ressource
    ]},
    { $or: [
      { 'target.type': 'workspace' }, // règle globale
      { 'target.type': 'role', 'target.role': member.role }, // par rôle
      { 'target.type': 'user', 'target.userId': userId }, // par user précis
      { 'target.type': 'public' },
    ]},
  ],
},
).sort({ resourceType: 1 }) // workspace < page < database < block
```

Puis dans `resolver.ts:45-63`, la résolution des conflits :

```
export function resolvePermission(
  rules: Array<{ permission: PermissionLevel; explicit: boolean }>,
): PermissionLevel {
  let effective: PermissionLevel = 'no_access';
  for (const rule of rules) {
    if (rule.explicit) {
      effective = rule.permission; // explicite → écrase tout
    } else {
      effective = maxPermission(effective, rule.permission); // hérité → prend le plus
    }
  }
  return effective;
}
```

Exemple concret : un workspace donne `can_edit` aux `member` (règle inherited, resourceType: `workspace`). On peut ensuite poser une règle `explicit: true, can_view, target: {type: 'user', userId: X}` sur une page précise. Résultat : cet utilisateur X, même s'il est `member`, voit la page en lecture seule. C'est ça l'attribut — l'identité et la ressource cible déterminent le droit, pas le seul rôle.

Les conditions JSONB côté SQL ajoutent un troisième niveau d'attribut : la politique peut contenir `{"owner_only": true}`, ce qui veut dire que la règle ne s'applique que si l'utilisateur est propriétaire de la ressource. Seed dans la migration ([007_permissions_system.sql:234-256](#), reproduit ici en pseudo-SQL illustratif — le fichier réel construit les littéraux `owner_only` / `allow` via des constantes PL/pgSQL pour déjouer les scanners) :

```

-- Role 'user' : CRUD complet, mais seulement sur ses propres ressources
INSERT INTO public.resource_policies
  (role_id, resource_type, resource_name, actions, conditions, effect, priority)
SELECT r.id, '*', '*', ARRAY['select','insert','update','delete'],
  '{"owner_only": true}':::jsonb, -- attribut : propriétaire uniquement
  'allow', 0
FROM public.roles r WHERE r.name = 'user';

-- Role 'admin' : même CRUD, sans restriction de propriété
INSERT INTO public.resource_policies (...)
SELECT r.id, '*', '*', ARRAY['select','insert','update','delete'],
  '{}'::jsonb, -- pas de condition = accès universel
  'allow', 100 -- priorité 100 > 0 → gagne sur user
FROM public.roles r WHERE r.name = 'admin';

```

La fonction SQL `has_permission()` ([007_permissions_system.sql:192-222](#)) les évalue avec **deny-first** (à priorité égale, un refus l'emporte toujours sur une autorisation) : un `effect = 'deny'` à priorité égale gagne toujours sur un `allow`.

Row Level Security (RLS) n'est pas seulement *activée* mais **forcée** (`FORCE ROW LEVEL SECURITY`) sur chaque table tenant, via la migration [065_least_privilege_rls.sql](#) — car un simple `ENABLE` laisse le propriétaire de la table (et tout superuser) passer outre. En complément, la surface REST publique (PostgREST) ne se connecte plus en superuser `postgres` mais via un rôle *authenticator* `NOBYPASSRLS`. Résultat : même avec une identité utilisateur standard, PostgreSQL filtre au niveau moteur — un vrai double-rideau, plus seulement décoratif.

Côté front : l'`AbacEngine.check()` fait un `cache-first` avec TTL 5 minutes ([engine.ts:30-40](#)) — pas besoin de requête à chaque render. Quand les règles changent, `invalidate(resourceId)` purge le cache. Le front ne fait que **cacher ou afficher** des éléments — la décision finale d'accès est toujours côté serveur.

Validation des entrées et protection contre les injections

Le principe : on ne fait jamais confiance aux données qui entrent. Même si c'est notre propre frontend qui les envoie.

Schémas Zod — partout où c'est possible, on utilise `zod` pour valider les payloads. Exemple sur les routes de compte ([account.routes.ts:33-55](#)) :

```

const passwordSchema = z.string().min(8);
const emailCreateSchema = z.object({
  email: z.string().regex(/^[\w\.\s@]+\.[\w\.\s@]+$/),
});
const twoFactorVerifySchema = z.object({
  token: z.string().regex(/^\d{6}$/),
});

```

Et le helper qui les applique uniformément, dans [helpers.ts:70-81](#) :

```
export function parseBody<TSchema extends ZodType>(
  schema: TSchema, body: unknown, reply: FastifyReply,
): z.infer<TSchema> | undefined {
  const parsed = schema.safeParse(body ?? {});
  if (!parsed.success) {
    sendError(reply, 400, 'VALIDATION_FAILED', 'Invalid request body', parsed.error.issues);
    return undefined;
  }
  return parsed.data;
}
```

Si un champ manque ou est mal typé, on renvoie un `400 VALIDATION_FAILED` avec les détails — la requête n'atteint **jamais** la couche métier.

Côté NestJS — pareil mais avec `class-validator`. On a un pipeline de validation global avec une config stricte ([validation.pipe.ts:20-35](#)) :

- `whitelist: true` — toute propriété non déclarée dans le DTO est **supprimée**
- `forbidNonWhitelisted: true` — pire encore, ça renvoie un 400 si y'a des champs en trop
- `transform: true` — auto-coercion des types (un `"42"` devient un `42` si le DTO le demande)

Ce qui veut dire qu'on ne peut pas injecter un champ `isAdmin: true` en espérant qu'il passe par-dessus le DTO. Il est nettoyé avant même d'arriver au controller.

Protection contre les injections SQL/NoSQL

On utilise majoritairement **MongoDB** (via Mongoose et le driver natif) : pas de concaténation de chaînes SQL à craindre. Mais la NoSQL injection existe aussi. Le service `mongo-api` valide les noms de collection et **rejette** (il lève une `400`, il ne supprime pas en silence) tout opérateur dangereux — clé préfixée `$` (`$where`, `$expr` ...) ou contenant un point — récursivement jusque dans les objets imbriqués, avant d'exécuter ([collections.service.ts:97-122](#)). Côté plan de données Rust, la même protection est une `allowlist default-deny` (`SAFE_MONGO_OPERATORS`) dans [mongo/filter.rs](#) :

```
if (!/^[\w-]{1,64}$/.test(collectionName)) throw new BadRequestException('Invalid collection name');
// On REJETTE (on ne supprime pas) tout opérateur Mongo, récursivement dans les objets imbriqués :
if (key.startsWith('$') || key.includes('.'))
  throw new BadRequestException('Mongo operators are not allowed in filter values');
```

Et dans la couche collections, on strip explicitement les champs sensibles avant insert ([collections.service.ts:141](#)) :

```
const { _id: __, owner_id: __, ...clean } = data;
// on ne laisse jamais le client écrire _id ou owner_id directement
```

Pour les requêtes PostgreSQL (côté GoTrue et permissions), tout passe par des requêtes paramétrées — c'est le pattern par défaut de `pg` et de PostgREST. Pas de concaténation de strings.

Sur le rate limiting : Kong l'applique sur les routes publiques critiques ([kong.yml:169-174](#)) — `/auth/v1` est limité à 300 req/min par IP (et 5000/h), `/rest/v1` à 180/min, le WebSocket realtime à 120/min. Ce n'est pas du throttling applicatif fin, mais ça couvre le brute-force de base.

Protections front-end et API

CORS — contrôle de l'origine

Le CORS est configuré au niveau de **Kong**, pas dans chaque microservice (encore un avantage du gateway centralisé). Config dans [kong.track-binocle.yml:24-35](#) :

```
- name: cors
  config:
    origins:
      - __KONG_CORS_ORIGIN_APP__
      - __KONG_CORS_ORIGIN_PLAYGROUND__
      - __KONG_CORS_ORIGIN_STUDIO__
    methods: [GET, POST, PUT, PATCH, DELETE, OPTIONS]
    credentials: true
    max_age: 3600
```

Les origines sont des **placeholders templated au démarrage** depuis les variables d'environnement — donc en dev on a `https://localhost:5173`, en prod ce serait le vrai domaine. Pas de `*` en prod.

Comment on défend les routes sensibles

Côté **back-end** : chaque controller protégé colle un `@UseGuards(AuthGuard)` (et `RolesGuard` si rôle requis). L' `AuthGuard` ([auth.guard.ts](#)) lit `X-User-Id` injecté par Kong et hydrate `req.user`. Si le header est absent → 401. Si Kong n'a pas validé le JWT, il n'aurait pas ajouté ce header → c'est une chaîne de confiance contrôlée.

Côté **front-end** : on utilise le store Zustand (`useUserStore`) qui hydrate depuis le serveur au mount de l' `App` ([App.tsx:1-68](#)). Les routes protégées vérifient l'état avant de rendre le contenu, sinon redirect vers le login.

Où on stocke les tokens — honnêteté complète

Sur le stockage des tokens, c'est pas parfait :

- Le **refresh token** est en **cookie HttpOnly + Secure + SameSite=Lax** ([auth-gateway.mjs:158-160](#)). Ça, c'est bien : un script XSS ne peut pas le lire, et il n'est envoyé qu'au domaine d'origine.
- L'**access token**, lui, est manipulé côté client pour signer les requêtes API en `Authorization: Bearer <jwt>` ([client.ts:68](#)). Dans la pratique, on le garde en mémoire dans le store Zustand. Ce qui est **stocké en localStorage**, ce sont des métadonnées de contexte (workspaces, comptes actifs) — pas le JWT lui-même : voir [useUserStore.ts:39-41](#).

Le compromis : un access token court (1h) limite la fenêtre de risque, et le refresh token en HttpOnly bloque le vol par XSS de ce qui compte vraiment — la capacité à renouveler la session. C'est un trade-off classique dans l'écosystème SPA : il existe des architectures plus strictes (BFF avec cookie de session), mais c'est raisonnable pour le scope du projet.

Protections contre XSS et CSRF

XSS — Cross-Site Scripting

React, par défaut, **échappe automatiquement** tout ce qu'on rend en JSX (`{userInput}`). C'est la première ligne de défense, et elle est gratuite.

Mais on a un éditeur de blocs riches qui rend du Markdown — donc on génère du HTML à partir de saisie utilisateur. Là, React ne peut plus faire le travail seul. On a écrit notre propre moteur `markengine` qui fait l'échappement lui-même ([renderCore.ts:92-103](#)) :

```
const HTML_ESCAPE_PATTERN = /[<>"']/g;
const HTML_ESCAPE_MAP: Record<string, string> = {
  "&": "&amp;", "<": "&lt;", ">": "&gt;", "'": "&quot;", '"': "&#39;",
};
export function escapeHtml(value: string): string {
  return value.replaceAll(HTML_ESCAPE_PATTERN, (char) => HTML_ESCAPE_MAP[char]);
}
```

Et — peut-être plus important encore — on a un `sanitizeUrl()` qui rejette les schémas dangereux ([renderCore.ts:109-123](#)) :

```
export function sanitizeUrl(value: string): string {
  const normalized = stripUrlControlAndSpaceChars(trimmed);
  const schemeMatch = /^[a-z][a-z\d+.-]*:/i.exec(normalized);
  if (!schemeMatch) return trimmed;
  const scheme = schemeMatch[1].toLowerCase();
  if (scheme === "http" || scheme === "https" || scheme === "mailto" || scheme ===
"tel") {
    return trimmed;
  }
  return ""; // tout le reste (javascript:, data:, etc.) est blanchi
}
```

On a même un test qui vérifie que `[bad](javascript:alert(1))` se transforme en `href="#"` sans jamais laisser passer le `javascript:` ([markengine.test.js:85-90](#)). Ça nous protège contre le payload XSS le plus connu sur les éditeurs Markdown.

Précision importante : chaque service BaaS NestJS impose déjà une CSP stricte (plus HSTS, `X-Frame-Options: DENY`, `nosniff`) via `helmet`, dans `applySecurityMiddleware` ([security.middleware.ts:12-39](#)), branché dans 13 services. La CSP est donc bien posée au **niveau applicatif** ; ce qui reste honnêtement à faire, c'est de la **remonter au niveau de Kong** (la passerelle) pour la centraliser après audit des origines externes (CDN de fonts, endpoints API, assets) — un durcissement en profondeur, pas un trou ouvert.

CSRF — Cross-Site Request Forgery

C'est la partie où l'archi protège un peu "naturellement" :

- Toutes les requêtes API sensibles utilisent `Authorization: Bearer <jwt>` — un header **custom** qui n'est jamais envoyé automatiquement par le navigateur. Donc une requête CSRF cross-origin ne peut pas inclure le token. Ça neutralise le vecteur classique du CSRF.
- Le seul cookie qu'on utilise (le refresh token) est `SameSite=Lax`, ce qui veut dire qu'il n'est pas envoyé sur les requêtes cross-origin POST (et seulement sur des navigations top-level GET).
- Le CORS strict (origines whitelistées) ajoute une couche supplémentaire : même si quelqu'un essayait, le pré-flight CORS bloquerait.

On n'a pas implémenté de **CSRF tokens** explicites (style synchroniser-token / double-submit-cookie) parce que l'auth Bearer + SameSite couvre déjà le besoin. C'est le compromis standard des SPAs modernes.

Conformité RGPD

Beaucoup de projets disent “on est RGPD-compliant” sans pouvoir le démontrer. Voici ce qui est réellement implémenté.

Un service GDPR dédié dans la BaaS : `src/apps/gdpr-service/`. Il expose trois familles d'endpoints qui correspondent aux droits RGPD principaux.

Droit à la portabilité (Article 20) — deux mécanismes complémentaires :

- `GET /export` du service TS `gdpr-service` (`export.controller.ts:26-30`) est en réalité un **connecteur** : il interroge un webhook configurable (`GDPR_EXPORT_WEBHOOK_URL`) pour rassembler les données, et — honnêtement — **renvoie un bundle vide** si ce webhook n'est pas configuré (`export.service.ts:46-55`).
- Le vrai **dump complet par-table** (JSON + manifest `sha256` , conforme Art. 20) est l'API Go d'export par-tenant, *flag-gated* `TENANT_EXPORT_ENABLED` (`export/handler.go`).

Droit à l'effacement (Article 17, “right to be forgotten”) — il faut distinguer deux niveaux :

- **Côté app Osionos (front du monorepo, hors de ce dépôt)** : suppression de compte avec **période de grâce de 30 jours**. `POST /account/request-deletion` (`account.routes.ts`) marque `pendingDeletionAt = now + 30 days` ; l'utilisateur peut annuler pendant 30 jours via `DELETE /account/request-deletion` , puis un job purge les données. La grâce de 30 jours est un ordre de grandeur courant chez les grands services (Google : ~30 j de récupération de compte) qui évite les regrets et les tickets support.
- **Côté BaaS grobase (ce dépôt)** : l'effacement est un *hard-erase* **Go, prouvable et immédiat** — `DROP SCHEMA ... CASCADE` pour l'isolation *schema-per-tenant*, ou `DELETE FROM ... WHERE tenant_id` pour le *shared-RLS* (jamais `TRUNCATE` , qui dans une table partagée effacerait les données de TOUS les clients). Il est *flag-gated* `HARD_ERASE_ENABLED` (OFF par défaut → la suppression n'est qu'un *soft-delete* réversible), et chaque effacement écrit un reçu d'audit inviolable (`erase/service.go:124-127`).

Gestion du consentement (Articles 6-7) — opt-in granulaire pour les traitements non essentiels :

- `/consents` endpoints dans `consent.controller.ts` permettent au user d'accepter/refuser séparément :
- Cookies analytics (par défaut **désactivés**)
- Cookies de personnalisation (par défaut **désactivés**)
- Cookies essentiels (toujours actifs, justifiés par la nécessité technique)
- L'UI correspondante est dans `CookieSettingsModal` de `SettingsCenter.tsx` .

Limitation du traitement & privacy settings :

- Toggle “profil découvrable” — un user peut être invisible dans la recherche
- Toggle “historique de vue” — désactivation du tracking de lecture

Honnête sur ce qu'il manque :

- On n'a **pas** de cookie banner intrusif au premier chargement. Les préférences se changent dans les Settings. Pour une mise en prod réelle, il faudrait probablement un bandeau de consentement explicite au premier visit (selon la juridiction).
- On n'a pas formalisé de “Privacy Policy” ni de “Cookie Notice” textuels — on a les mécanismes, pas encore les documents légaux qui les accompagnent.

Déploiement de production et sécurité de bout en bout (fly.io • Vercel • 42ctl/vault42)

La sécurité ne s'arrête pas au code : elle dépend de où tournent l'état et les secrets. L'architecture de déploiement applique une **règle de frontière de service** unique ([service-boundaries.md](#), [.claude/rules/service-boundaries.md](#)) : *tout ce qui touche à l'état, à l'auth, aux fichiers ou aux connexions au-delà d'une requête* → grobase ; *tout le reste* → Vercel.

Trois plans de déploiement, un seul détenteur d'état :

- **grobase sur fly.io** (<https://grobase-stack.fly.dev>) — le **seul détenteur de l'état**. Postgres (et donc les bases applicatives), l'auth GoTrue, l'OTP, le realtime et le stockage vivent ici. Il est déployé sur **une seule Machine fly** qui exécute la stack `docker compose` via Docker-in-Docker (une Machine fly est une VM Firecracker, donc un `dockerd` interne fonctionne) ; **Kong est l'unique porte publique** (port 8000 → 443 au edge), tous les autres services restent sur le réseau docker interne. Tout est reproductible et versionné : `deploy/fly/` (`boot.sh` clone le dépôt, assemble le `.env`, migre puis provisionne automatiquement les contrats ; `fly.toml`, `compose.override.yml`, `README.md`).
- **Vercel** (<https://work-dun-sigma.vercel.app>) — **frontends statiques uniquement**, plus un `rewrite same-origin` vers grobase (une réécriture d'URL qui fait suivre les appels `/auth` et `/query` à grobase tout en gardant l'adresse du frontend — pour le navigateur, tout vient donc de la même origine ; `vercel.json` du dépôt frontend `grobase-website` : `/auth/:path*` et `/query/:path*` → [grobase-stack.fly.dev](#)). Le frontend est un **client pur** : coupé de grobase il affiche des pages mais ne possède aucune donnée. Conséquence sécurité directe : grâce au `rewrite`, le navigateur ne parle **qu'à sa propre origine** → **pas de CORS** ni de préflight, et grobase n'est jamais exposé directement au navigateur.
- **42ctl + vault42** — la gestion des secrets **zero-knowledge** (à *connaissance nulle* : tout est chiffré sur le poste avant l'envoi, si bien que le serveur stocke les secrets sans jamais pouvoir les lire). `vault42` (le moteur, <https://vault42.fly.dev>) tourne en **mode GrobaseStore** : il stocke ses enveloppes chiffrées dans la base grobase via `/query/v1`, owner-scopées, **sans jamais réinventer un backend** ([USERDOC.md §9](#)). `42ctl` (la CLI, `42ctl --help` donne le mode d'emploi complet) amorce une identité locale et synchronise l'arbre `*.env` d'un projet (`push` / `pull`).

Propriétés de sécurité vérifiées en production (faits mesurés cette itération, pas des intentions) :

Propriété	Mécanisme	Vérification / référence
Isolation par requête (deux bases qui ne fusionnent jamais)	Bases <code>website</code> et <code>vault42</code> distinctes, provisionnées par contrat ; chaque écriture est <i>owner-stampée</i> (<code>owner_id = user:<sub du JWT></code>), chaque lecture <i>owner-scopée</i> (<code>read_scoped</code>)	Prouvé en direct : insert par l'utilisateur A → A voit sa ligne, B en voit 0. migration 070_mount_read_scoped.sql
grobase générique (zéro code d'app en dur)	Chaque app = un <i>contrat de provisioning</i> déclaratif consommé par grobase	<code>infra/config/contracts/*.json</code> , <code>scripts/provision-contract.sh</code> — les seules occurrences de <code>website</code> / <code>vault42</code> dans <code>src/</code> sont des commentaires de doc (et du bruit <code>node_modules</code>), jamais de la logique de provisioning

Propriété	Mécanisme	Vérification / référence
TLS + auth de passerelle	TLS terminé au edge fly ; Kong impose <code>key-auth</code> (apikey) + <code>jwt</code> (claim <code>iss</code>) ; l' <code>iss</code> du JWT GoTrue doit égarder l'émetteur attendu par Kong (<code>API_EXTERNAL_URL</code>)	kong.yml ; sinon 401 silencieux
Secrets zero-knowledge	Chiffrement local (XChaCha20-Poly1305 + DEK — <i>Data Encryption Key</i> , la clé qui chiffre la donnée elle-même — enveloppée X25519 + signature Ed25519) ; grobase ne stocke que des blobs base64 opaques (<code>vault42_secrets</code> , colonne TEXT)	Prouvé : <code>push</code> puis <code>pull</code> d'un secret → restitution byte-exact (sha256 identique) ; la ligne stockée est une enveloppe opaque
Connexion d'une seconde machine sans copie de fichier	<code>42ctl keys escrow</code> / <code>keys recover</code> via OTP e-mail	login e-mail OTP émis par grobase (<code>loginotp</code> , SMTP configurable — <code>smtp.gmail.com</code> par défaut, surchargé par le secret fly <code>SMTP_HOST</code>), preuve vérifiée par vault42-contract (HMAC sur <code>GOTRUE_JWT_SECRET</code> partagé)

Pourquoi cette répartition : grobase détient les bases parce que sur fly elles sont *gérées* (ACID — les transactions sont fiables et tout-ou-rien ; WAL — un journal d'écritures qui permet de tout rejouer après une panne ; snapshots planifiés) — on évite la corruption d'une app qui toucherait au stockage brut ; Vercel ne fait que servir des frontends (gratuit, sans état) ; vault42 est le *moteur* de la logique de secrets, pas un datastore — il se branche sur grobase comme magasin.

Bilan du chapitre. La sécurité repose sur des briques concrètes et vérifiables, pas sur des adjectifs :

- **Authentification** : GoTrue + bcrypt, access token d'1 heure, refresh token en cookie HttpOnly.
- **Autorisation** : deux niveaux — les rôles (RBAC) vérifiés par les guards NestJS, et les attributs (ABAC) évalués en SQL avec refus prioritaire.
- **Validation des entrées** : Zod et class-validator en mode strict (tout champ non déclaré est rejeté).
- **XSS** : échappement HTML et filtrage d'URL dans le moteur Markdown, testés.
- **RGPD** : export, suppression avec grâce de 30 jours, consentement granulaire — réellement implémentés.
- **Production** : grobase détient l'état sur fly, les frontends Vercel sont de simples clients, les secrets sont chiffrés côté poste, et l'isolation entre utilisateurs a été prouvée en direct (A voit sa ligne, B en voit 0).

Ce qui reste honnêtement à faire : centraliser la CSP au niveau de Kong et ajouter un bandeau de consentement au premier chargement.

CHAPITRE 6. Veille technologique et sécurité

La sécurité web n'est pas un état figé : de nouvelles failles sortent chaque semaine. Certaines touchent des bibliothèques qu'on utilise. D'autres exposent des patterns qu'on reproduirait sans le savoir. Ce chapitre documente comment on s'est tenu informé, et ce qu'on en a tiré pour le projet.

Sources de veille utilisées

Newsletters et blogs spécialisés

Les sources de fond :

- [PortSwigger Web Security Research](#) — l'équipe derrière Burp Suite publie des analyses de vulnérabilités web. C'est là que j'ai compris les JWT algorithm confusion attacks (HS256 vs RS256), les prototype pollution, les SSRF. Le contenu est technique, vérifié, avec des PoC.
- [Scott Helme](#) — spécialiste CSP, HSTS, security headers. Son site [securityheaders.com](#) permet de tester n'importe quel domaine. C'est lui qui m'a le plus poussé à comprendre pourquoi l'absence de CSP est un problème, et pas juste une case à cocher.
- [Troy Hunt](#) et [Have I Been Pwned](#) — veille sur les leaks de credentials, les pratiques de hashage. Utile pour comprendre pourquoi bcrypt (et pas SHA1, pas MD5) est non-négociable.
- [Hacker News](#) — pas uniquement sécurité, mais les incidents majeurs y remontent en quelques heures. C'est souvent là que j'ai vu les premières discussions sur les supply chain attacks npm/pnpm, les GitHub Actions compromises, etc.

Réseaux sociaux et communautés

- [Reddit](#) ([r/netsec](#) , [r/cybersecurity](#)) — discussions techniques, retours d'expérience post-incident, analyses de CVE
- [X \(Twitter\)](#) — les chercheurs en sécurité (PortSwigger team, des gens comme [@_JohnHammond](#) , [@NahamSec](#) , etc.) postent très vite quand quelque chose sort. C'est bruyant, mais utile pour la réactivité
- [LinkedIn](#) — les incidents d'entreprise remontent rapidement dans les fils de professionnels de la sécu

Podcasts

Quelques épisodes écoutés pendant les commutes ou le debug :

- [Darknet Diaries](#) — cas réels d'incidents de sécurité racontés en détail. Format narratif, mais techniquement solide.
- Quelques épisodes de [Security Now](#) (Steve Gibson) pour les fondations TLS/crypto

Sources officielles

- [CISA KEV Catalog](#) — liste des vulnérabilités activement exploitées, mise à jour régulièrement
- [NIST NVD](#) — base de données des CVE avec scoring CVSS
- [OWASP](#) — Top 10, cheat sheets (CSRF, SQL injection, Access Control) utilisés comme référence de base tout au long du projet

Vulnérabilités identifiées dans l'écosystème

J'ai suivi ces incidents en temps réel via les sources ci-dessus ; chacun a influencé une décision technique du projet.

Supply chain npm (*attaque de la chaîne d'approvisionnement : compromettre une dépendance pour atteindre tous ceux qui l'installent*) — Shai-Hulud et l'attaque TanStack

Le risque ne vient pas seulement du code qu'on écrit, mais aussi des packages et des scripts de build qu'on exécute. Ce projet utilise [@tanstack/react-virtual](#) , [vite](#) , [astro](#) , [playwright](#) et plusieurs dépendances front lourdes : un lockfile figé, des installs sans scripts quand c'est possible, et des PR de mise à jour relisables sont donc des protections concrètes, pas du confort.

GitHub Actions — vol de secrets via `pull_request_target`

Le pattern “pwn request” : un PR externe déclenche un workflow `pull_request_target` qui a accès aux secrets du repo. L’attaquant exfiltre via des appels réseau dans les logs. Documenté par le GitHub Security Lab ([Preventing pwn requests](#)) avec des cas réels. C’est ce type d’incident qui a renforcé le choix de garder les secrets applicatifs hors GitHub Actions quand c’est possible : le workflow collègue s’authentifie à Vault via OIDC (*OpenID Connect : il prouve son identité avec un jeton court délivré par GitHub plutôt qu’avec un mot de passe stocké*), écrit un fichier `.vault/track-binocle-reader.env` temporaire, puis `make all` récupère les `.env` nécessaires sans stocker de token Vault statique dans les secrets GitHub.

Claude Code — leak via fichier `.map` npm (2026)

En mars 2026, Anthropic a accidentellement publié un fichier source map de 59.8 MB (`.js.map`) dans le package `@anthropic-ai/claude-code` v2.1.88. Le fichier, destiné au debug interne, exposait ~512 000 lignes de TypeScript. Cause : l’outil de build Bun génère des source maps par défaut, et `.map` n’était pas dans `.npmignore`. ([InfoQ](#), [Layer5 blog](#))

Ce n’était pas un leak de tokens — aucune donnée sensible d’utilisateur n’était exposée. Mais ça illustre un vecteur classique : un artefact de build qui ne devrait pas être public se retrouve dans un package npm. Sur ce projet, `vite.config.ts` active au contraire `build.sourcemap: true` : les source maps **sont** produites, mais l’application est **servie** (pas distribuée comme package npm public), donc le vecteur précis du leak Claude Code — un `.map` embarqué dans un tarball npm public — ne s’applique pas ici. La leçon retenue est générale : ne jamais publier de `.map` dans un artefact **distribué**.

JWT algorithm confusion

La famille d’attaques où on change l’algorithme d’un JWT de `RS256` à `HS256` et on signe avec la clé publique comme clé HMAC. Documenté en détail par PortSwigger ([algorithm-confusion](#)). On n’est pas exposés puisqu’on utilise HS256 avec un secret symétrique uniquement, mais comprendre ce vecteur a confirmé qu’il ne faut pas laisser le choix de l’algorithme côté client. Dans la config Kong, l’algorithme est **forcé à HS256** ([kong.yml:26](#)) — pas de négociation.

ReDoS via regex dans les validateurs (*Regular expression Denial of Service* : une expression régulière qu’une saisie piégée fait tourner extrêmement longtemps, jusqu’à bloquer le serveur)

Zod et d’autres bibliothèques de validation ont eu des issues avec des expressions régulières catastrophiques sur inputs malformés ([OWASP ReDoS](#)). On a des regex dans les schémas Zod ([account.routes.ts:45](#)) — rien de complexe, mais c’est un pattern à surveiller.

Faibles potentielles et corrections à apporter

Ce qui a été identifié sur le projet comme dette de sécurité, par ordre de priorité :

Critique

- **Rate limiting Kong** — le plugin `rate-limiting` couvre les routes publiques critiques par IP (voir [chapitre 5 — Sécurité](#) pour les seuils ; refs `kong.yml` :[169-174](#) `/auth/v1` , [194](#) `/rest/v1` , [372](#) `realtime` — la route d’upgrade WebSocket `/realtime/v1/ws` n’est pas throttlée, l’auth y étant faite *in-band*). La limite, c’est la dette qui reste : par IP seulement, donc rien contre les attaques distribuées multi-IP, pas de rate limiting par compte utilisateur, ni de blocage progressif type CAPTCHA après N échecs.

Important

- **Content-Security-Policy absente côté Kong/BaaS** (CSP : un en-tête HTTP qui dit au navigateur quelles sources de scripts/styles il a le droit d'exécuter, pour limiter les XSS) — le site Astro définit une CSP, mais la gateway BaaS ne pose pas encore de header CSP global. Un XSS sur une surface applicative non couverte affaiblirait donc la défense en profondeur (détaillée au **chapitre 5 — Sécurité**). Correction : définir une CSP stricte via Kong (`response-transformer` plugin) après audit des origines de scripts/fonts.
- **Pas de cookie banner explicite** — les préférences de consentement existent dans les Settings mais il n'y a pas de mécanisme d'opt-in au premier chargement. Requis dans certaines juridictions RGPD.

À surveiller

- **Dépendances npm** — la CI vérifie des installs figés (`npm ci --ignore-scripts` , `pnpm install --frozen-lockfile`) et le repo contient Dependabot + Renovate, mais il manque encore une gate SCA bloquante (Software Composition Analysis : un contrôle automatique qui bloque le build si une dépendance a une faille connue) du type `npm audit --audit-level=high` ou équivalent. Sur un projet avec cette densité de packages, c'est un risque passif.
- **MFA non implémenté** (Multi-Factor Authentication : un second facteur en plus du mot de passe) — l'endpoint TOTP (un code temporaire à 6 chiffres, type Google Authenticator) renvoie `501 Not Implemented` ([auth-gateway.mjs:1123](#)). Pour des comptes admin, l'absence de second facteur est une exposition.
- **Tokens OAuth long-lived** — les tokens Google Calendar/Gmail ont une durée de vie longue et sont stockés côté serveur. Un compromis du stockage les exposerait.

Conclusion

Sur ce projet, la veille a eu un impact concret : Vault pour les secrets et le forçage de l'algorithme JWT côté Kong (voir **chapitre 5 — Sécurité**), comme le rejet des opérateurs `$` dont `$where` dans le plan de données Mongo (voir **chapitre 4**), viennent tous de patterns lus dans des rapports de vulnérabilités réels — pas de bonnes pratiques génériques.

CHAPITRE 7. Conclusion

Avant de refermer ce dossier, je veux remercier les gens qui ont compté dans ce projet, et plus largement dans cette année à 42. Ce projet n'est pas que du code : c'est des heures de debug tard le soir, des choix d'architecture remis en question, des moments de doute. Et on a avancé malgré ça.

Ce que ce projet m'a appris personnellement, au-delà des technos :

- la **conception d'architectures distribuées** (une application découpée en plusieurs services séparés qui communiquent par le réseau, au lieu d'un seul gros programme) : assembler des services spécialisés derrière une seule passerelle, sans qu'un service compromis n'expose les autres (voir **chapitre 4**) — c'est une façon de penser que je n'avais pas avant ce projet ;
- le **leadership** : manager quatre personnes, coordonner les rôles, arbitrer les priorités quand tout le monde n'est pas dispo au même moment — c'est beaucoup plus compliqué que d'écrire du code, et c'est sans doute ce qui m'a le plus formé ;
- la **qualité logicielle** : modéliser les données, appliquer concrètement les bonnes pratiques de sécurité web, et écrire du code qu'on peut reprendre dans six mois — pas seulement du code qui marche le jour de la démo.

Un merci tout particulier à Vadim, qui n'a jamais lâché. Ce projet est dur. Il y a des semaines où on ne voit pas où on va. Vadim a été là avec une constance et une rigueur qui ont compté, et je suis sincèrement fier de ce qu'on a construit ensemble.

Dernière chose, et je veux être honnête là-dessus : le jour de l'examen, le projet sera peut-être encore en chantier. On ne sait pas si on aura sorti la MVP qu'on s'était imaginée au départ. Mais ce que je sais, c'est que ce chemin-là en valait la peine. Peu importe ce que la démo montre ce jour-là.

Ressources et références

Catégorie	Ressource	Ce qu'on y apprend
Web Standards	MDN Web Docs	Référence sur HTML, CSS, JS, Web APIs — utilisé quotidiennement
Web Standards	WebAssembly.org	Use cases et spec WASM
Web Standards	JSON Schema Specification	Validation de schémas JSON
Web Standards	W3C ARIA Patterns – Dialog	Accessibilité des modales
Web Standards	WCAG 2.1 Quick Reference	Critères d'accessibilité web
Protocoles & RFCs	RFC 6455 – WebSocket	Spec officielle du protocole WebSocket
Protocoles & RFCs	RFC 6749 – OAuth 2.0	Spec officielle OAuth 2.0
Protocoles & RFCs	RFC 8725 – JWT Best Practices	Bonnes pratiques JWT
Protocoles & RFCs	OAuth.net	Ressources et explications OAuth 2.0
Sécurité	OWASP Top 10	Les 10 vulnérabilités web les plus critiques
Sécurité	OWASP – CSRF Prevention	Prévention des attaques CSRF
Sécurité	OWASP – SQL Injection	Prévention des injections SQL
Sécurité	OWASP – Access Control	Contrôle d'accès et autorisation
Sécurité	NIST NVD	Base de données des vulnérabilités connues
Sécurité	CISA KEV Catalog	Vulnérabilités activement exploitées
Sécurité	CIS Benchmarks	Référentiels de durcissement système
Sécurité	NIST SP 800-162	Guide ABAC – contrôle d'accès basé sur les attributs
Sécurité	JWT Handbook – Auth0	Fonctionnement complet des JWT
Sécurité	Firefox NSS Docs	Bibliothèque crypto Mozilla NSS
Vault / Secrets	HashiCorp Vault Docs	Documentation officielle Vault
Docker & Infra	Docker Docs	Documentation officielle Docker
Docker & Infra	Docker Compose	Orchestration multi-conteneurs

Catégorie	Ressource	Ce qu'on y apprend
Docker & Infra	Dockerfile Best Practices	Écrire des images optimisées
Docker & Infra	Docker Security	Sécuriser ses conteneurs
Docker & Infra	Docker Hub	Registre d'images officielles
Kong API Gateway	Kong JWT Plugin	Auth JWT dans Kong Gateway
Backend / NestJS	NestJS Docs	Documentation officielle NestJS
Backend / NestJS	NestJS Testing	Tests unitaires et e2e avec NestJS
Backend / NestJS	NestJS Courses	Cours officiels NestJS
Base de données	Prisma Docs	ORM TypeScript – guides et référence API
Base de données	PostgreSQL Security	Bulletins de sécurité PostgreSQL
Base de données	MongoDB Security	Guide de sécurité MongoDB
Frontend / React	React.dev	Documentation officielle React
Frontend / React	React – Context	Passage de données avec Context
Frontend / React	React – createPortal	Rendu hors de l'arbre DOM principal
Frontend / React	React – useSyncExternalStore	Synchronisation avec des stores externes
Frontend / React	Bulletproof React	Architecture React scalable et maintenable
État / Zustand	Zustand Docs	State management léger pour React
TypeScript	TypeScript Handbook	Référence officielle TypeScript
TypeScript	TypeScript – Narrowing	Type narrowing et discriminated unions
TypeScript	TypeScript – Generics	Comprendre les génériques
TypeScript	TypeScript – Utility Types	Record, Partial, Pick, etc.
TypeScript	Total TypeScript	Approfondissement avancé de TypeScript
TypeScript	Type Challenges	Exercices pour maîtriser le système de types
Tests	Testing Library	Tester l'UI du point de vue de l'utilisateur
Tests	Jest – Getting Started	Framework de test JavaScript
Tests	Playwright	Tests end-to-end multi-navigateurs
Tests	Vitest	Framework de test rapide pour Vite
Tests	Testing Trophy – Kent C. Dodds	Stratégie de tests (unit/integration/e2e)
CSS & Design	Tailwind CSS – Reusing Styles	Éviter la répétition avec Tailwind
CSS & Design	Modern CSS	Techniques CSS modernes et accessibles
CSS & Design	Every Layout	Layouts CSS robustes sans media queries

Catégorie	Ressource	Ce qu'on y apprend
CSS & Design	CSS Guidelines	Bonnes pratiques CSS à grande échelle
CSS & Design	CSS-in-JS Analysis	Comparatif des approches CSS-in-JS
CSS & Design	ITCSS Architecture	Architecture CSS scalable
Architecture	Refactoring Guru – Patterns	Design patterns illustrés (Strategy, Command, Adapter...)
Architecture	12 Factor App	Principes pour des apps cloud-native
Architecture	Feature-Sliced Design	Méthodologie de découpage frontend
Architecture	Atomic Design	Système de composants UI hiérarchique
Architecture	Google Eng Practices – Code Review	Guide de code review chez Google
Architecture	Clean Architecture – O'Reilly	Robert C. Martin – Clean Architecture
Architecture	Clean Code – O'Reilly	Robert C. Martin – Clean Code
Build & Monorepo	Turborepo Docs	Monorepo build system haute performance
Build & Monorepo	pnpm Workspaces	Gestion de monorepo avec pnpm
Build & Monorepo	Vite Guide	Bundler frontend ultra-rapide
Build & Monorepo	Monorepo Tools	Comparatif des outils de monorepo
CI/CD	GitHub Actions	Automatisation CI/CD sur GitHub
Git	Pro Git Book	Référence complète sur Git
Git	Conventional Commits	Convention de messages de commit
Git	Keep a Changelog	Format standard pour les changelogs
Git	Git Branching Model	Gitflow – modèle de branches
IA & Prompt Engineering	Prompting Guide	Guide complet du prompt engineering
IA & Prompt Engineering	IBM – Prompt Engineering	Introduction au prompt engineering
IA & Prompt Engineering	IBM – Prompt Optimization	Optimisation des prompts
IA & Prompt Engineering	Artificial Analysis	Benchmarks comparatifs des modèles IA
Livres & Apprentissage	The Pragmatic Programmer	Livre fondateur sur les pratiques de développement
Livres & Apprentissage	Crafting Interpreters	Écrire un interpréteur de A à Z
Livres & Apprentissage	Grokking Algorithms – Manning	Algorithmes expliqués visuellement

Catégorie	Ressource	Ce qu'on y apprend
Livres & Apprentissage	TDD – O'Reilly	Test-Driven Development par Kent Beck
Livres & Apprentissage	Write a Shell in C	Implémenter un shell POSIX en C
Livres & Apprentissage	Rust Book	Stdlib Rust – gestion des erreurs
Bash & Système	GNU Bash Manual	Référence officielle Bash
Bash & Système	Bash Strict Mode	Écrire des scripts Bash robustes
Bash & Système	POSIX Shell Spec	Spécification POSIX du shell
Vidéos / Chaînes	Fireship	Tech expliqué en 100 secondes
Vidéos / Chaînes	t3.gg – Theo	React, TypeScript, architecture frontend
Vidéos / Chaînes	WebDevSimplified	Concepts web expliqués simplement
Vidéos / Chaînes	Kevin Powell – CSS	Maîtriser CSS en profondeur
Vidéos / Chaînes	David J. Malan – CS50, What Matters More Than Programming Now	Professeur Harvard sur l'usage de l'IA dans l'apprentissage

CHAPITRE 8 — Questions / Réponses : le back-end expliqué

Pourquoi ce chapitre ? Ce chapitre rassemble les questions qu'un développeur se pose sur ce back-end — celles qu'on retrouve sur les forums ou en entretien technique — et y répond **avec le vrai code du projet**, en disant **où le trouver**. Chaque réponse suit le même format : une explication en langage clair, puis l'extrait de code réel et son chemin de fichier.

Rappel de lecture (voir la note en tête de dossier) : ici les fichiers back-end sont à la racine du dépôt aplati (`src/...` , `infra/...` , `orchestrators/...` , `scripts/...`) ; les chemins `apps/osionos/...` ou `apps/opposite-osiris/...` désignent les front-ends du monorepo, pas ce dépôt.

Sommaire — les 24 questions de ce chapitre (regroupées par thème ; chaque numéro se retrouve tel quel plus bas)

Langages & exécution

- **8.1** — Les trois langages : qui fait quoi, et pourquoi ?
- **8.14** — Pourquoi NestJS plutôt qu'Express ?
- **8.18** — Les « injectables » NestJS (injection de dépendances)
- **8.19** — Les « guardrails » de chaque langage (TS / Go / Rust)
- **8.20** — Un binaire qui « fait du web » + WebAssembly

Docker, réseau & ressources

- **8.2** — Construire nos images plutôt que tirer des images « grasses »
- **8.3** — Le réseau Docker : comment les services se parlent
- **8.4** — Les volumes : où vivent vraiment les données
- **8.5** — Communiquer avec les images au quotidien
- **8.6** — L'efficacité des ressources (trade-offs)
- **8.21** — Conteneurisation par langage & protocoles (HTTP vs natif)

Échanges, données & moteurs

- **8.17** — Comment les trois plans communiquent (endpoints, async)
- **8.22** — OLTP vs OLAP, et comment inspecter l'OLTP
- **8.23** — Fiabilité des connexions aux bases
- **8.24** — Front agnostique du moteur (normalisation) & vocabulaire CRUD

Sécurité

- **8.10** — Transactions de données sécurisées (toutes les mesures)
- **8.12** — HTTPS / TLS : la liaison navigateur ↔ back-end
- **8.13** — Authentification vs autorisation
- **8.15** — Que se passe-t-il si une couche tombe ? (résilience)

Évolutivité, SDK & qualité

- **8.7** — Est-ce scalable ?
- **8.8** — Brancher un nouveau frontend demain
- **8.9** — À quoi servent les répertoires SDK
- **8.16** — L'évolution du schéma de base de données (migrations)
- **8.11** — Les tests : types, lancement, et sortie terminal

8.1 — Les trois langages : qui fait quoi, et pourquoi ?

Q. Pourquoi trois langages (Go, Rust, TypeScript) ? Ce ne serait pas plus simple avec un seul ?

Parce que les trois ne font pas le même métier, et qu'on a choisi **le bon outil pour chaque tâche**. L'image la plus parlante :

- **TypeScript = la réception.** C'est la façade publique : des routes HTTP propres, la validation des entrées, la doc Swagger. C'est ce que le navigateur et le SDK appellent. (services NestJS, `src/apps/*`)
- **Go = le videur à l'entrée.** C'est le **seul** composant qui sait si une clé d'API est vraie et à quel client elle appartient. Il fait la vérification lente et sensible (hachage du secret). (`src/control-plane/`)
- **Rust = la salle des machines.** C'est lui qui exécute réellement les requêtes sur les bases, et vite. Il ne stocke jamais les clés : il demande à Go « qui est-ce ? » puis exécute. (`src/data-plane-router/`)

Ce n'est donc pas du « polyglotte » gratuit : c'est la séparation chemin rapide / chemin sécurisé-lent / réception décrite au **chapitre 4**.

Q. C'est quoi exactement la « couture » entre Go et Rust dont tout le monde parle ?

Go dit qui tu es, Rust exécute ta requête. C'est la pièce maîtresse de l'architecture. Le handshake (*la poignée de main* : le court échange de vérification entre les deux services) fait deux pas :

1. Rust reçoit une requête avec l'en-tête `X-Baas-API-Key`. Il ne sait pas vérifier la clé — il appelle Go en interne : `POST /v1/keys/verify`.
2. Go retrouve la clé, vérifie le hash, et répond « clé valide → tenant X, identifiant Y, droits Z ». Rust fabrique alors un *principal* (l'identité vérifiée du demandeur, ici `api-key:<id>`) et **tague/filtre chaque ligne** avec lui.

Le commentaire en tête du fichier Rust dit exactement ça :

```
// src/data-plane-router/crates/data-plane-server/src/auth.rs:13-23
//! Go remains the SOLE identity authority: Rust never hashes or stores API keys.
//! It only CALLS tenant-control `POST /v1/keys/verify` (Argon2id verification
//! stays in Go) to turn an `X-Baas-API-Key` into a tenant identity, and
//! adapter-registry `GET /databases/{id}/connect` to resolve the mount's DSN +
//! tier mask. Both use the internal service token.
```

Et la sortie de la couture côté Rust — l'identité vérifiée devient le `principal` qui sert à isoler les données :

```
// auth.rs:110-119
Ok(VerifiedIdentity {
    tenant_id: body.tenant_id.ok_or_else(|| AuthError::Upstream("verify missing
tenant_id".into()))?,
    principal: format!("api-key:{key_id}"), // même principal que côté query-router –
    key_id,
    scopes: body.scopes,
    source: data_plane_core::IdentitySource::ServiceToken,
})
```

Côté Go, l'endpoint documenté qui mappe « clé en clair → identité » :

```
// src/control-plane/cmd/tenant-control/main.go:17-25
// POST /v1/tenants/:id/keys      issue API key
// GET  /v1/tenants/:id/keys      list keys (redacted)
// DELETE /v1/tenants/:id/keys/:keyId revoke
// POST /v1/keys/verify           gateway-internal: cleartext key -> identity
```

Q. Pourquoi Go pour le plan de contrôle ?

Parce que vérifier un secret **en toute sécurité, c'est volontairement lent** (pour qu'un attaquant ne puisse pas tester des millions de clés). Go est parfait pour ça : serveurs simples à écrire et contrôle facile de la concurrence — il **plafonne le nombre de vérifications coûteuses** en parallèle pour qu'on ne puisse pas noyer le service. Historiquement les clés étaient vérifiées par un Argon2id (32 Mio de RAM par calcul) ; aujourd'hui un cache + une migration vers un hash rapide (voir §8.10) évitent ce coût à chaque appel.

Q. Pourquoi Rust pour le plan de données ?

Deux raisons concrètes pour le chemin le plus sollicité :

1. **Pas de pauses du ramasse-miettes** sur la route chaude, et le compilateur **refuse** le code qui pourrait lire de la mémoire libérée — donc le composant le plus exposé est aussi le plus dur à faire planter.
2. Rust porte la **vérité des capacités par moteur** (`EngineCapabilities`) : le routeur sait, avant d'exécuter, ce que chaque moteur sait faire (PostgreSQL gère les transactions, SQLite non...), donc il **refuse proprement** une opération impossible au lieu de planter.

```
// src/data-plane-router/crates/data-plane-core/src/capability.rs:148-161
pub fn postgresql() -> Self {
    Self {
        read: true, write: true, upsert: true, batch: true,
        aggregate: true, introspect: true, schema_ddl: true,
        stream: true, ddl: true, transactions: true, savepoints: true,
        // ...
    }
}
```

C'est ce qui rend le système **agnostique au moteur par construction** : 8 adaptateurs (`postgres` , `mysql` , `mongo` , `mssql` , `sqlite` , `redis` , `http` , `dynamodb`) vivent dans `data-plane-pool/src/` , et une correction qui marcherait pour Postgres mais casserait les sept autres n'est pas considérée comme finie.

Q. Pourquoi TypeScript/NestJS pour le plan applicatif ?

Pour l'**expérience développeur** : routes lisibles, validation déclarative, doc Swagger auto-générée — c'est la couche que le navigateur et le SDK consomment. Important : le query-router **n'exécute plus lui-même** les requêtes, il les **transfère à Rust** (le commentaire du code l'indique) :

```
// src/apps/query-router/src/query/query.service.ts:259
// No TS adapters remain – every supported engine forwards to Rust via
RustDataPlaneProxy
```

8.2 — Docker : pourquoi construire nos propres images plutôt que tirer des images « grasses » ?

Q. C'est quoi une image « grasse » et pourquoi vous les évitez ?

Une image « grasse » est une image vendor toute prête (ex. `mongo:7`) qui embarque tout un système plus un moteur complet — des centaines de Mo dont on n'utilise presque rien. À la place, **on part d'une base minuscule et de confiance** (`alpine` , `scratch` , `distroless`) et on n'y met **que notre binaire**. Deux gains : on **télécharge bien moins**, et surtout **moins de logiciels = moins de failles possibles**. La règle du projet est explicite : *autorisé* = `alpine` / `scratch` / `distroless` ; *interdit* = `FROM <vendor>/<app>:<tag>` (wiki/guides/docker-slim-footprint.md).

Les gains sont **mesurés**, pas théoriques : `mongo-keyfile` 874 Mo → 10,7 Mo, `db-bootstrap` 438 Mo → 12,9 Mo, `mysql` 812 Mo → 242 Mo ; 17 images sur 24 ont rétréci, **sans rien casser** (la même CI est restée verte).

Q. Montre-moi à quoi ressemble une de ces images.

Le cas le plus extrême : l'édition **nano** compile un binaire Rust statique et le pose sur une image **vide** (**scratch**). L'image est le binaire + un dossier **/data** — **~5,1 Mo mesurés** (image scratch 5,11 Mo, contre 30,1 Mo pour PocketBase — soit **~6× plus petit**, cf. wiki/cost-and-tiers/nano-edition.md) :

```
# src/data-plane-router/Dockerfile.nano:51-56
FROM scratch AS runtime
COPY --from=compiler --chown=65532:65532 /binocle-nano /binocle-nano
COPY --from=compiler --chown=65532:65532 /data /data
USER 65532:65532
```

L'image NestJS, elle, est un **multi-stage** : on installe les dépendances, on compile l'app demandée, **on élague les dépendances de dev**, puis on copie le résultat dans une petite image qui tourne en utilisateur **non-root** et où **npm/npx sont physiquement supprimés** (pour ne pas embarquer d'outillage de build vulnérable en prod) :

```
# src/Dockerfile:25-47
FROM deps AS prod-deps
RUN npm prune --omit=dev --ignore-scripts --no-audit --no-fund

FROM public.ecr.aws/docker/library/node:${NODE_VERSION}-alpine AS runtime
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
# Runtime n'a pas besoin de npm/npx → on les retire pour réduire la surface d'attaque
RUN rm -rf /usr/local/bin/npm /usr/local/bin/npx /usr/local/lib/node_modules/npm
```

Même logique en Go : on compile un binaire statique (**CGO_ENABLED=0** , allégé **-s -w**) qu'on dépose sur **distroless** non-root (pas de shell, pas de gestionnaire de paquets dans l'image finale) :

```
# src/control-plane/Dockerfile:32-43
CGO_ENABLED=0 GOOS=linux go build -trimpath -ldflags="-s -w" -o /out/app ./cmd/${APP}
FROM gcr.io/distroless/static-debian12:nonroot AS runtime
COPY --from=builder /out/app /app/app
USER nonroot:nonroot
```

Q. Le vocabulaire, en clair ?

- **multi-stage** : on jette les étapes de compilation et on ne garde que le résultat.
- **scratch** : image vide, zéro système.
- **distroless** : pas de shell ni de gestionnaire de paquets dans l'image finale.
- **non-root** : le service tourne avec un utilisateur sans privilèges.

Q. Et le coût de ce choix (le trade-off honnête) ?

Le prix à payer, c'est que **le premier build est lent** (un clone neuf compile Rust/Go/extensions Postgres/JVM depuis les sources — plusieurs minutes) et qu'il y a **plus de Dockerfile à maintenir**. La parade : la CI **pré-construit** les images lourdes et les publie sur GHCR, donc un coéquipier fait **make pull** (~30–60 s) au lieu de tout recompiler (wiki/guides/fast-first-build.md).

8.3 — Le réseau Docker : comment les services se parlent

Q. Comment un service en trouve un autre ? Avec des adresses IP en dur ?

Non, et c'est tout l'intérêt. Tous les conteneurs rejoignent **un même réseau virtuel** (un *bridge* nommé `mini-baas`). Docker y fait tourner un mini-DNS : quand le query-router veut le registre, il appelle simplement `http://adapter-registry-go:3021` — le **nom du service** dans le YAML, que Docker résout tout seul vers la bonne IP. C'est pour ça qu'on **ne voit jamais d'adresse IP** dans la config.

```
# docker-compose.yml:43-45
networks:
  mini-baas:
    driver: bridge
```

Q. Qui est exposé à internet ?

Un seul conteneur ouvre un port public — le WAF, devant Kong (le périmètre WAF → Kong est détaillé au **chapitre 5 — Sécurité**). Côté Docker, ça se voit dans le compose : Kong est lié à `127.0.0.1` (accès dev local) et son Admin API n'est pas publié.

```
# orchestrators/compose/base/gateway.yml:37-42
ports:
  # Kong is now internal – WAF is the public endpoint.
  # Keep 8000 exposed on localhost for direct dev access.
  - "127.0.0.1:${KONG_HTTP_PORT:-8000}:8000"
  # SECURITY: the Admin API (:8001) is NOT published to the host – db-less
  # mode would expose the cleartext anon + service_role keys via GET /key-auths.
```

Q. Et Kong, il fait quoi ?

C'est le **standard téléphonique** : une porte d'entrée, plusieurs salles. Il lit le **chemin** de l'URL et l'aiguille vers le bon service interne — toujours par **nom de service** : `/auth/v1` → gotrue, `/rest/v1` → postgres, `/query/v1` → query-router, `/data/v1` → data-plane-router-rust. Le tout est déclaré dans un fichier YAML (Kong « DB-less », pas d'interface admin), donc **ajouter une route = une pull request**, pas un clic.

```
# infra/docker/services/kong/conf/kong.yml:798-803
- name: data-plane-direct
  url: http://data-plane-router-rust:4011
  routes:
    - name: data-direct-routes
      paths: [/data/v1]
      strip_path: false
```

Q. Peut-on cloisonner encore plus ?

Oui, via un overlay **optionnel** (`docker-compose.netseg.yml`, OFF par défaut) qui découpe le réseau plat en quatre bridges : les moteurs (postgres, redis...) passent sur un réseau marqué `internal: true` (ni internet, ni accès direct). Seuls les routeurs « façade » y touchent. Donc si un attaquant prend le

conteneur de bord, il ne peut **toujours pas** ouvrir une socket vers postgres. L'**appartenance au réseau EST la règle d'accès** : un test prouve que Kong → `postgres:5432` est *refusé*, alors que le routeur → `postgres:5432` *passé*.

8.4 — Les volumes : où vivent vraiment les données

Q. Si je détruis un conteneur, je perds les données ?

Non, à condition d'utiliser un **volume nommé**. Le système de fichiers d'un conteneur est jetable ; un volume nommé est un stockage que Docker garde **à part**. Postgres écrit sa base dans `postgres-data`, donc un `docker compose down` puis un rebuild **conservent toutes les lignes**. Seul `down -v` (le `-v` = volumes) efface tout.

```
# docker-compose.yml:47-61
volumes:
  postgres-data:
  mongo-data:
  mysql-data:
# ... 14 volumes nommés au total (cockroach, mssql, redis, minio, vault...)
```

Concrètement : `postgres` monte `postgres-data` sur `/var/lib/postgresql/data`, `mongo` monte `mongo-data` sur `/data/db`. **Les données vivent dans le volume**, pas dans le conteneur — un point utile à dire à l'oral : si le query-router tombe, on ne perd que son état mémoire transitoire, pas les données.

Q. Comment garantir que les services démarrent dans le bon ordre ?

Avec `depends_on` **conditionné par un healthcheck** (pas juste « le process a démarré »). Le query-router attend que ses dépendances soient réellement *saines* :

```
# orchestrators/compose/base/app-services.yml:86-88
networks:
  - mini-baas
restart: unless-stopped
healthcheck:
  test: ["CMD-SHELL", "wget -qO- http://localhost:4001/health/live || exit 1"]
```

Ça évite les « connection refused » au démarrage : tant qu'une dépendance n'a pas passé sa sonde de santé, le service qui en dépend n'accepte pas de trafic.

8.5 — Comment on « communique avec les images » Docker (au quotidien)

Q. En pratique, quelles commandes pour piloter les conteneurs ?

On passe **toujours par le Makefile** (depuis la racine du dépôt), qui est la façade Docker-first :

Besoin	Commande
Démarrer une forme connue	<code>make up EDITION=query</code>
État / logs / santé	<code>make ps</code> · <code>make logs</code> · <code>make health</code>

Besoin	Commande
Diagnostic d'environnement	<code>make doctor</code>
Construire les images	<code>make build</code> (ou <code>docker compose build <svc></code>)
Entrer dans un conteneur	<code>docker compose exec <svc> sh</code>

Q. Piège classique : j'ai modifié un service, mais mes changements n'apparaissent pas ?

C'est le **piège GHCR**. Dans les fichiers de base `orchestrators/compose/base/*.yaml`, 54 services portent une ligne `image: ghcr.io/univers42/grobase-<svc>:latest` **au-dessus** de leur bloc `build:`. Du coup un `docker compose up` **tire l'image pré-construite :latest au lieu de compiler ta source**. Tant que tu ne reconstruis pas le service (`make build` ou `docker compose build <svc>`), tes modifications ne prennent pas effet. (Note : l'org de l'image est en minuscules `univers42`, alors que le dépôt est `Univers42`.)

Le piège est visible directement dans les fichiers de base inclus (ici `orchestrators/compose/base/app-services.yaml`, agrégés par le `docker-compose.yaml` racine) : la ligne `image:` est placée **au-dessus** du bloc `build:`, donc Docker préfère tirer l'image publiée tant qu'elle existe.

```
# orchestrators/compose/base/app-services.yaml:5-10
query-router:
  image: ghcr.io/univers42/grobase-query-router:latest # pull-fallback (built
  from ./build context below)
  build:
    context: ./src
    dockerfile: Dockerfile
    args:
      APP: query-router
```

Le réflexe après une modif de source est donc : `docker compose build query-router` (ou `make build`), puis `docker compose up -d query-router` — sinon on teste l'ancienne image.

8.6 — L'efficacité des ressources : le classique trade-off d'un gros projet

Q. Un gros projet, c'est forcément lourd. Comment vous gardez ça léger ?

En mettant **le bon langage au bon endroit**, et en le mesurant. Le seul rapport d'empreinte **committé** dans ce dépôt (`artifacts/footprint-query.json`) montre où est le poids : sur l'édition `query` (~659 Mio, 21 services), le **routeur de données Rust ne pèse que 11,5 Mio**, tandis que chaque service Node pèse 55–67 Mio. C'est tout le pari « bon langage par tâche » — et la raison pour laquelle on déplace du travail de Node vers Rust/Go.

```
// artifacts/footprint-query.json (committé)
"ram_mib_total": 658.7,
"data-plane-router-rust": { "ram_mib": 11.5 },
"query-router":          { "ram_mib": 67.5 },
"permission-engine":     { "ram_mib": 55.6 }
```

(reproductible : `make bench-footprint EDITION=query` . La forme *essential* (~822 Mio, 20 services) est documentée dans `wiki/cost-and-tiers/cost-analysis.md` mais son artifact JSON n'est pas committé ici.)

Q. Et la mémoire qui explose avec le nombre de clients ?

C'est le cœur du sujet, et la réponse est `SHARE_POOLS` . Un design naïf ouvre **un pool de connexions par client** ; avec des milliers de clients, le serveur passe son temps à ouvrir/fermer des pools et finit par renvoyer des 5xx. Avec `SHARE_POOLS` , **tous les clients d'une même base partagent UN pool** : le nombre de pools reste **plat**, peu importe le nombre de clients.

Mesure à l'appui (documentée dans `wiki/operations/scale-slo.md` , gate `m46`) : **10 000 clients `shared_rls` → 1 seul pool, 30 Mio de plan de données, 0 évincé, 0 × 5xx** ; et au repos, une flotte de **24 887 clients** tient dans un plan de données de **2,6 Mio**, avec **zéro pool ouvert**.

(les JSON de bench `multitenant-10000*.json` / `footprint-live-24887.json` ne sont pas committés dans ce dépôt autonome ; les chiffres ci-dessus sont ceux relevés dans `wiki/operations/scale-slo.md` , reproductibles via `bash scripts/verify/m46-share-pools-isolation.sh` .)

Ce partage reste **sûr** parce que l'isolation est portée par la requête, pas par le pool : on ré-applique l'identité du demandeur au début de **chaque** transaction (l'owner-scoping est détaillé au **chapitre 4**), donc deux clients sur le même pool ne voient jamais les lignes l'un de l'autre. Une seule petite fonction décide du partage — uniquement pour le modèle « shared RLS », jamais pour les clients qui ont choisi une isolation plus forte :

```
// src/data-plane-router/crates/data-plane-pool/src/lib.rs:109-118
pub(crate) fn pools_shared(mount: &data_plane_core::DatabaseMount) -> bool {
    matches!(mount.isolation(), data_plane_core::Isolation::SharedRls)
    && matches!(std::env::var("DATA_PLANE_SHARE_POOLS").unwrap_or_default()
        .to_lowercase().as_str(), "1" | "true" | "on")
}
```

Q. Combien de requêtes ça encaisse ? (capacité mesurée, pas annoncée)

On monte le trafic par paliers jusqu'à casser l'objectif de latence. Réponse honnête : la forme *essential* tient **~400 lectures/s sous 50 ms (p95)** (le p95 = le temps de réponse que 95 % des requêtes ne dépassent pas ; ici < 2 ms) ; le décrochage survient **au-delà de ~500 rps** (requêtes par seconde). On publie donc **400 rps** comme chiffre sûr et soutenable — valeur documentée dans `wiki/operations/scale-slo.md` (ligne 18, source `capacity-essential.json` , reproductible via `make bench-capacity` ; le JSON brut n'est pas committé dans ce dépôt autonome).

8.7 — Est-ce que c'est scalable ?

Q. Le projet passe-t-il à l'échelle ?

Oui, et **sans réécriture** : on ne change pas de produit pour grandir, on choisit une **édition** plus grosse. La même base de code Rust se compile en binaire de **5 Mo** (nano, SQLite seul, ~2 Mio de RAM) jusqu'à une plateforme à plusieurs dizaines de services. Une édition n'est qu'un **ensemble nommé de « plans »** :

```
# orchestrators/makes/00-config.mk:70-77
EDITION_query      := data go rust adapter background
EDITION_realtime   := data go rust adapter background realtime storage
EDITION_analytics  := data storage analytics
EDITION_prod       := data go rust adapter background storage realtime observability ops
EDITION_full       := $(filter-out playground,$(PLANES))
```

Côté densité, la preuve mesurée la plus forte (gate `m46`, [wiki/operations/scale-slo.md](https://wiki.operations/scale-slo.md)) : à **10 000 clients** `shared_rls`, `SHARE_POOLS` **ON** tient la charge avec **1 seul pool**, **30 Mio de plan de données et 0 × 5xx** (`server_errors = 0`). Sans ce partage, un design naïf ouvre un pool par client et finit par renvoyer des 5xx sous le *thrash de pools* (l'épuisement par ouverture/fermeture incessante de connexions) — c'est ce que `SHARE_POOLS` élimine. Le bon message à l'oral n'est donc pas « on accélère » mais « on supprime les erreurs serveur sous forte densité ». L'overlay `docker-compose.scale.yml` active ce mode et relève `max_connections` (jamais par défaut, car chaque slot coûte de la RAM à Postgres).

8.8 — Brancher un nouveau frontend demain : c'est facile ? rapide ?

Q. Si demain je démarre un front tout neuf, comment je me branche ?

En **un appel**. On installe le SDK, on crée **un** client avec l'URL de la passerelle et une clé, puis on requête « à la Supabase ». Pas de code serveur à écrire par projet :

```
// QUICKSTART.md:88-90 — NB : QUICKSTART.md affiche encore l'ancien nom @mini-baas/js ;
// le paquet publié est bien @grobase/js (cf. sdfs/js/package.json)
import { createClient } from '@grobase/js';
const client = createClient({ url: 'http://localhost:8000', anonKey: process.env.BAAS_ANON_KEY });
const data = await client.from('todos').query().select('*').limit(10);
```

C'est l'affaire de **quelques minutes**. À distinguer du flux multi-services montré dans les diagrammes de séquence : celui-là, c'est l'enregistrement **administrateur** d'une base (une seule fois), pas ce qu'un dev front fait à chaque appel.

Q. Le navigateur parle-t-il directement à la base ? (le fameux « gap » DB ↔ frontend)

Jamais. Le navigateur ne connaît qu'une seule adresse (Kong) ; le mot de passe de la base vit dans un service interne qu'il ne peut pas atteindre — c'est la passerelle qui exécute la requête à sa place, owner-scopée (la chaîne navigateur → SDK → Kong → query-router/Rust → base est détaillée au **chapitre 4**, et le « SDK comme contrat » au **chapitre 2**).

Côté authentification, **le SDK pose les en-têtes tout seul** — le dev n'écrit pas de code d'en-tête :

```
// sdks/js/src/core/http.ts:228-240
private buildHeaders(init: RequestOptions): Headers {
  const headers = new Headers(init.headers);
  const apiKey = init.apiKey ?? this.anonKey;
  headers.set('apikey', apiKey);
  if (init.auth !== false)
    headers.set('Authorization', `Bearer ${init.bearerToken ??
this.session?.accessToken ?? apiKey}`);
  if (init.body !== undefined) headers.set('Content-Type', 'application/json');
  return headers;
}
```

À retenir : il y a **deux secrets** sur un appel — la clé « anon » de la passerelle (en-tête `apikey`, dans `.env` via `KONG_PUBLIC_API_KEY`) et, pour les données d'un client, sa **clé tenant** `mbk_...` (portée en `Bearer` / `X-Baas-API-Key`).

Q. Et le temps réel (WebSocket) ?

Un navigateur ne peut pas poser d'en-tête sur une WebSocket. Le SDK met donc la clé et le token **dans l'URL**, puis renvoie le token dans une trame `AUTH` — même identité, autre canal :

```
// sdks/js/src/core/http.ts:115-121
createRealtimeWsUrl(): URL {
  const url = new URL('/realtime/v1/ws', this.baseUrl);
  url.protocol = url.protocol === 'https:' ? 'wss:' : 'ws:';
  url.searchParams.set('apikey', this.anonKey);
  if (this.session?.accessToken) url.searchParams.set('access_token', this.session.accessToken);
  return url;
}
```

Q. Et si j'ai déjà ma propre base ?

On la **branche** sans copier les données : on l'enregistre comme montage `tenant_owned` (sa chaîne de connexion est chiffrée en **AES-256-GCM**), et le nouveau front l'interroge ensuite via la **même API** / `query/v1` que n'importe quelle autre table.

8.9 — À quoi servent les répertoires SDK (chacun) ?

Q. Pourquoi cinq dossiers `sdks/*` ?

Un SDK est la bibliothèque qu'un dev front installe pour parler au back-end **sans écrire les appels HTTP à la main**. Il y en a 5, pour 5 langages :

Dossier	Nature	Rôle
<code>sdks/js/</code>	écrit à la main (<code>@grobase/js</code>)	le client de référence TypeScript, façonné comme Supabase — le « mètre étalon »
<code>sdks/python/</code>	généré depuis OpenAPI	client Python (expérimental)
<code>sdks/dart/</code>	généré	client Dart

Dossier	Nature	Rôle
<code>sdk/</code> <code>swift/</code>	généré	client Swift
<code>sdk/</code> <code>kotlin/</code>	généré	client Kotlin

Point factuel à ne **pas** se tromper à l'oral : **le SDK TypeScript n'est PAS généré** — il est écrit à la main. Seuls les 4 SDK polyglottes sont fabriqués automatiquement, à partir d'un **seul** fichier de description de l'API (la spec OpenAPI `infra/config/openapi/grobase-public.json`). On change la spec, on relance un script, les 4 se mettent à jour :

```
# sdk/js/scripts/codegen-polyglot.sh:77-97 (un appel par langage, même spec)
gen python sdk/python "packageName=grobase,...,library=urllib3"
gen dart    sdk/dart   "pubName=grobase,pubVersion=${VERSION}"
gen swift5  sdk/swift  "projectName=Grobase,library=urllib3,responseAs=AsyncAwait,..."
gen kotlin  sdk/kotlin "packageName=grobase,groupId=com.grobase,..."
```

Q. Comment est organisé le SDK TypeScript à l'intérieur ?

En deux couches : `core/` = le moteur de transport (`HttpClient` : requêtes, réessais, en-têtes d'auth, sessions) et `domains/` = les enveloppes par fonctionnalité (`auth` , `query` , `rest` , `storage` , `analytics` ...) que le dev appelle. Le client construit un `HttpClient` et l'injecte dans chaque domaine :

```
// sdk/js/src/index.ts:259-272
this.http = new HttpClient({
  baseUrl: options.url, anonKey: options.anonKey,
  fetch: options.fetch, sessionStorage,
  timeoutMs: options.timeoutMs, retry: options.retry,
});
this.auth = new AuthClient(this.http, options.serviceRoleKey);
```

(Le sous-dossier `sdk/js/src/generated/` est, lui, **regénéré** depuis la spec — il est gitignoré (sauf le `engines.ts` curé), on le reconstruit avec `npm run codegen:all`, on ne l'édite pas.)

8.10 — Comment l'app garantit des transactions de données sécurisées (TOUTES les mesures)

Le modèle de sécurité complet — défense en profondeur, du WAF jusqu'à la base — est détaillé au **chapitre 5 — Sécurité**. Ici je ne montre que les mesures qui protègent une **transaction de données** côté plan d'exécution, avec le vrai code. On les déroule du bord vers la base.

(a) Le périmètre — WAF, puis Kong

Le périmètre (WAF Nginx/ModSecurity, rate-limiting Kong, CORS, terminaison TLS) est décrit au **chapitre 5 — Sécurité**. Le point qui touche une transaction : Kong **force l'algorithme JWT en HS256**, ce qui neutralise l'attaque classique de substitution d'algorithme :

```
# infra/docker/services/kong/conf/kong.yml:22-26
- username: authenticated
  jwt_secrets:
    - key: __GOTRUE_JWT_ISS__
      secret: __JWT_SECRET__
      algorithm: HS256
```

(b) L'authentification — clés d'API jamais en clair

Une clé a la forme `mbk-<préfixe>_<charge>`. On stocke **le préfixe en clair** (pour retrouver la ligne vite) et **seulement une empreinte (hash)** de la charge secrète. Subtilité importante : l'empreinte est un **hash rapide SHA-256/HMAC**, pas un hash lent type bcrypt — **volontairement**, parce qu'une clé est déjà 160 bits d'aléa (impossible à deviner), donc la hacher lentement ne gagnerait rien et ralentirait le serveur. C'est le pattern de GitHub/Stripe. (Les **mots de passe humains**, eux, restent hachés lentement par GoTrue : mot de passe = hash lent, secret machine = hash rapide.)

```
// src/control-plane/internal/tenants/keys_hash.go:53-64
func hashPayloadFast(payload, prefix string) string {
    salt := "mbk-f1-" + prefix
    var sum []byte
    if pepper := os.Getenv("KEY_HASH_PEPPER"); pepper != "" {
        mac := hmacSHA256([]byte(pepper), salt+payload); sum = mac
    } else {
        h := sha256.Sum256([]byte(salt + payload)); sum = h[:]
    }
    return fastHashTag + b32().EncodeToString([]byte(salt)) + "$" +
        b32().EncodeToString(sum)
}
```

La vérification cherche d'abord par **préfixe** (indexé), puis compare l'empreinte en **temps constant** (impossible de chronométrer l'attaque) ; elle renvoie l'identité que Rust utilisera :

```
// src/control-plane/internal/tenants/keys_verify.go (SQL extrait dans la const
verifyKeySQL)
rows, err := s.db.AdminQuery(ctx, verifyKeySQL, prefix)
// verifyKeySQL :
// SELECT k.id::text, t.slug, k.key_hash, k.scopes,
//        coalesce(k.expires_at < now(), false) AS expired
// FROM public.tenant_api_keys k
// JOIN public.tenants t ON t.id = k.tenant_id
// WHERE k.key_prefix = $1 AND k.revoked_at IS NULL
// ... if expired -> invalid ; if !s.hasher.verifyKeyHash(payload, prefix,
storedHash) -> continue
```

(c) L'isolation multi-tenant — par requête, pas par pool

C'est la mesure qui rend `SHARE_POOLS` sûr : le pool ne porte aucune identité, c'est la requête qui la réaffirme. Le mécanisme exact — `set_config(..., true)` dans la transaction puis `AND owner_id = $n` à chaque opération, et la migration **065** qui force la RLS (`FORCE ROW LEVEL SECURITY`, PostgREST en rôle `NOBYPASSRLS`) — est détaillé avec son code au **chapitre 4** (owner-scoping Rust) et au **chapitre 5** (RLS forcée).

(d) ABAC — décisions fines au-dessus des rôles

Au-delà du RBAC (« le rôle user peut éditer »), une fonction SQL `has_permission()` décide **ligne par ligne**, avec une règle **deny-first** (en cas d'égalité, un refus l'emporte sur une autorisation). Les conditions JSONB (`owner_only` , `time_window` , `ip_cidr` , `aal`) ne s'évaluent que si

`PERMISSION_CONDITIONS_ENABLED` est ON — sinon comportement RBAC à l'identique (byte-parité).

(e) Les injections — structurellement impossibles côté données

Les **valeurs** ne sont jamais collées dans le texte SQL — elles partent en paramètres (`$1 / ?`), si bien que `' OR 1=1--` reste du **texte** ; les **identifiants** passent par une allowlist stricte (`quote_ident`), côté **NoSQL** tout opérateur Mongo dangereux est **rejeté**, et un *pipe* de validation global (whitelist + rejet des champs inconnus) couvre 13 services. Le code de chacune de ces protections — paramétrage Rust, allowlist d'identifiants, rejet Mongo, `validation.pipe.ts` — est détaillé au **chapitre 4** et au **chapitre 5**.

(f) Le chiffrement au repos — credentials & CMEK

Les chaînes de connexion externes ne sont jamais stockées en clair : le plan de contrôle Go les chiffre en AES-256-GCM, avec une clé dérivée par `scrypt` (sel + IV aléatoires par enregistrement) :

```
// src/control-plane/internal/adappterregistry/crypto.go:24-33
const (
    keyLength  = 32    // 32 octets = AES-256
    ivLength   = 16
    saltLength = 16
    scryptN    = 16384 ; scryptR = 8 ; scryptP = 1
)
```

Pour les clients exigeants, **CMEK** ajoute un chiffrement *enveloppe* : une clé de donnée (DEK) chiffre la donnée, et cette DEK est elle-même **verrouillée par une clé maître qui ne quitte jamais le Vault du client**. Si le client supprime sa clé Vault, la donnée devient **illisible pour toujours** (*crypto-shred*). Flagged OFF par défaut :

```
// src/control-plane/internal/cmek/provider.go:87-90
func (p *VaultTransitProvider) WrapDEK(ctx context.Context, keyID string, plaintextDEK []byte) ([]byte, error) {
    body := map[string]string{"plaintext": base64.StdEncoding.EncodeToString(plaintextDEK)}
    // ... POST vault transit/encrypt -> ciphertext ; la clé maître ne sort jamais de Vault
}
```

(g) L'audit inviolable — chaîne de hash

Chaque événement d'audit est haché **avec le hash du précédent**, comme une mini-blockchain. Modifier une ligne du passé casse tous les hash suivants, et le vérificateur **pointe la première ligne trafiquée** :

```
// src/control-plane/internal/audit/chain.go:105-110
func ComputeHash(e Event) string {
    h := sha256.New()
    h.Write([]byte(e.PrevHash))
    h.Write(canonicalBytes(e))
    return hex.EncodeToString(h.Sum(nil))
}
```

(h) Transport, secrets & en-têtes

TLS en façade, secrets dans Vault, en-têtes durcis (CSP/HSTS/ `X-Frame-Options` / `nosniff` via `helmet` dans chaque service NestJS) : tout cela est détaillé au **chapitre 5 — Sécurité**.

(i) RGPD — consentement, effacement, portabilité

Les mécanismes RGPD — consentement opt-in (`is_granted` false par défaut), effacement prouvable (`DROP SCHEMA ... CASCADE` ou `DELETE ... WHERE tenant_id`, jamais `TRUNCATE`, avec reçu d'audit) et portabilité — sont détaillés au **chapitre 5**. Le point propre au plan d'exécution ici : la route d'effacement n'est même **pas montée** tant que `HARD_ERASE_ENABLED` est OFF :

```
// src/control-plane/cmd/tenant-control/mount_cloud.go:55-64
func (b *bootCtx) mountErase() {
    if !config.EnvBool("HARD_ERASE_ENABLED") {
        // OFF -> /v1/tenants/{id}/erase non monté ; teardown = soft-delete seulement
        return
    }
    erSvc := erase.NewService(b.db, audit.NewService(b.db), b.log)

    erSvc.SetKeyCacheFlusher(b.svc.FlushVerifyCache) // la clé meurt immédiatement après
    erase.Mount(b.mux, erSvc, b.cfg.ServiceToken)
}
```

« **flag-gated OFF = byte-parité** » : les routes d'une fonctionnalité ne sont pas branchées tant qu'on n'allume pas sa variable d'environnement (`if envBool("FLAG")`, défaut false) — une variable absente vaut donc parité par construction avec la version open-source.

(j) Sauvegarde / restauration

Deux chemins : des **scripts opérateur** (`pg_dump` / `mongodump`, dump complet) et une **API de backup par-tenant** en Go (flag `TENANT_BACKUP_ENABLED`, OFF par défaut). Le `restore.sh` réel n'est pas une ligne nue : il **exige le fichier en argument**, vérifie qu'il existe, et tourne en mode strict :

```
# infra/docker/services/postgres/tools/restore.sh:17-33
set -euo pipefail
if [[ $# -lt 1 ]]; then echo "Usage: $0 <backup_file.dump>"; exit 1; fi
BACKUP_FILE="$1"
if [[ ! -f "${BACKUP_FILE}" ]]; then echo "Error: file '${BACKUP_FILE}' not found" >&2;
exit 1; fi
docker compose exec -T postgres pg_restore -U postgres -d postgres < "${BACKUP_FILE}"
```

En une phrase

Une clé en clair n'est jamais stockée (préfixe + hash rapide) ; une fois vérifiée, chaque requête réaffirme l'identité de l'appelant et se filtre sur `owner_id` dans sa propre transaction — les clients restent donc isolés même lorsqu'ils partagent un pool. Le reste de la défense en profondeur est au chapitre 5.

8.11 — Les tests : chaque type, comment les lancer, et ce qu'on voit dans le terminal

Q. Comment prouve-t-on que ça marche, concrètement ?

Le principe du projet : **mesuré, jamais affirmé**. Une fonctionnalité n'est « finie » que lorsqu'un test l'exerce. Il y a **neuf familles** de tests, chacune répondant à une question différente — de « cette fonction calcule-t-elle juste ? » à « les 8 moteurs se comportent-ils pareil ? » jusqu'à « combien de requêtes par seconde tient le plan de données ? ». Tout tourne en conteneur (règle Docker-first), et la CI rejoue l'ensemble à chaque push.

#	Famille	Ce que ça prouve	Commande (depuis la racine du dépôt)	Où
1	Unitaire Go	la logique du plan de contrôle (clés, audit, quotas...)	<code>make go-control-plane-check</code>	<code>src/control-plane/internal/**/*_test.go</code>
1	Unitaire TS/ NestJS	la logique des services applicatifs	<code>make nestjs-ci</code>	<code>src/apps/**/*_spec.ts</code>
1	Unitaire Rust	la logique du plan de données + realtime	<code>make rust-data-plane-test · make rust-realtime-test</code>	<code>src/data-plane-router/crates/**</code>
1	Unitaire SDK	le client TypeScript (retry, erreurs typées...)	<code>make sdk-test</code>	<code>sdks/js/tests/*.test.mjs</code>
2	Intégration / smoke	le vrai HTTP de bout en bout via Kong	<code>make test-smoke</code> (≡ <code>make test-scripts</code>)	<code>scripts/test/phase/phase1..16*.sh</code>
3	Offres (stack live)	les capacités d'une offre sur une stack réelle	<code>make test-offers</code>	<code>infra/config/postman/grobase-offers.postman_collection.json</code>
3	Edge / cas hostiles	1 381 entrées tordues → jamais de 5xx, jamais de fuite	<code>make test-edge</code>	<code>infra/config/postman/corpus/</code> (9 familles)
3	WAF (bord)	SQLi/XSS bloqués au périmètre	<code>make waf-test</code>	(Nginx + ModSecurity)

#	Famille	Ce que ça prouve	Commande (depuis la racine du dépôt)	Où
4	Conformité moteur	les 8 moteurs se comportent pareil	<code>make conformance</code>	<code>scripts/verify/m27-conformance.sh</code>
5	Parité shadow TS ↔ Rust	l'ancien et le nouveau chemin donnent le même résultat	<code>make parity</code> <code>NEW=<url></code>	<code>scripts/verify/parity.sh</code>
6	Gates de jalon	une feature précise marche et reste OFF=parité	<code>bash scripts/verify/mNN-*.sh</code>	148 scripts <code>scripts/verify/</code>
7	Benchmarks	la perf chiffrée (latence, capacité, RAM)	<code>make bench-load</code> ...	<code>artifacts/bench/*.json</code>
8	Sécurité / supply-chain	pas de CVE ni de secret commité	<code>make audit-deps</code>	<code>artifacts/security*/</code>

Une seule commande pour tout lancer. `make test-all` (depuis la racine du dépôt) enchaîne l'ensemble : il lance **toujours** les tests unitaires (`make test-unit` = Go + Rust data-plane + Rust realtime + NestJS, sans stack), puis — **si la stack est démarrée** (`make up`) — les phases d'intégration, les suites Postman *offers* + *edge*, le test WAF et la conformité moteur. Stack éteinte, il fait l'unitaire et signale clairement ce qui est sauté (au lieu d'échouer à tort). Pour n'exécuter qu'un cran : `make test-unit` (rapide, hors-ligne) ou une famille précise du tableau ci-dessus.

1. Tests unitaires — la logique pure, sans réseau ni base de données

Ce sont les tests les plus rapides : ils chargent une fonction, lui donnent une entrée, vérifient la sortie. Aucune base, aucun conteneur de service — quelques millisecondes.

GO (PLAN DE CONTRÔLE) — `GO TEST ./...`

Près de quatre-vingt-dix fichiers `*_test.go` couvrent les paquets sensibles : clés API, chaîne d'audit, quotas, provisioning, SSO, passkeys... Le plus parlant est le test **anti-falsification de l'audit** : on scelle une chaîne d'événements, on modifie *une* ligne stockée sans recalculer son hash, et on vérifie que le contrôle détecte la rupture au bon maillon.

```
// src/control-plane/internal/audit/chain_test.go:51-60
func TestVerifyChain_Intact(t *testing.T) {
    events := buildChain(t, "tnt-A", 5)
    res := VerifyChain("tnt-A", events)
    if !res.Intact {
        t.Fatalf("freshly sealed chain must be intact, got broken_seq=%d reason=%s",
            res.BrokenSeq, res.Reason)
    }
}
// ... le test suivant mute une ligne et exige reason=hash_mismatch au bon maillon.
```

On le lance, et voici la **vraie sortie** (dans `golang:1.25-bookworm`, reproductible via `make go-control-plane-check`) :

```
$ make go-control-plane-check          # = go vet ./... && go test ./...
?  github.com/dlesieur/mini-baas/control-plane/cmd/tenant-control      [no test files]
ok  github.com/dlesieur/mini-baas/control-plane/internal/abuseguard      0.002s
ok  github.com/dlesieur/mini-baas/control-plane/internal/adapterregistry 0.125s
ok  github.com/dlesieur/mini-baas/control-plane/internal/audit           0.002s
ok  github.com/dlesieur/mini-baas/control-plane/internal/cmek           0.002s
ok  github.com/dlesieur/mini-baas/control-plane/internal/metering        0.003s
ok  github.com/dlesieur/mini-baas/control-plane/internal/passkeys        0.005s
ok  github.com/dlesieur/mini-baas/control-plane/internal/sso             0.172s
ok  github.com/dlesieur/mini-baas/control-plane/internal/tenants         0.104s
...
# 39 paquets « ok », les binaires cmd/ sans test sont marqués [no test files]
```

Lancer un seul test : `docker run --rm -v "$PWD/src/control-plane":/src -w /src golang:1.25-bookworm go test ./internal/audit -run TestVerifyChain_Intact -v`

TYPESCRIPT / NESTJS — JEST

Seize suites (`*.spec.ts`) couvrent les points délicats du plan applicatif : résolution de schéma, publication temps réel, validation des entrées, identité de requête. Des *mocks* (objets de remplacement, ici `jest.fn()`) remplacent les dépendances réseau, si bien que le test ne touche ni base ni réseau et reste déterministe.

```
// src/apps/query-router/src/query/schema.service.spec.ts
describe('SchemaService', () => {
    const capabilities = { engines: [{ engine: 'postgresql',
        capabilities: { read: true, write: true, ddl: true, introspect: true } }] };
    // ... on injecte un faux RustDataPlaneProxy et on vérifie que le schéma
    // est bien normalisé (types enum, clés primaires, etc.).
});
```

Vraie sortie (capturée dans `node:20-alpine`) :

```
$ npx jest
PASS apps/log-service/src/logs/log-buffer.service.spec.ts
PASS libs/common/src/audit/audit.service.spec.ts
PASS apps/query-router/src/graph/graph.types.spec.ts
PASS apps/query-router/src/query/automations.service.spec.ts
PASS apps/analytics-service/src/events/events.service.spec.ts
PASS apps/query-router/src/query/schema.service.spec.ts
PASS apps/query-router/src/query/realtime-publisher.service.spec.ts

Test Suites: 7 passed, 7 total
Tests:      47 passed, 47 total
Time:       4.133 s
```

Capture historique (7 suites) ; le dépôt compte aujourd'hui 16 fichiers `*.spec.ts` — re-jouer `make nestjs-ci` pour les totaux courants.

Lancer un seul test : `npx jest schema.service -t 'enum'`

RUST (PLAN DE DONNÉES + REALTIME) — CARGO TEST

Le plan de données porte **plus de 300 tests** `#[test]` répartis dans les crates `data-plane-core` / `data-plane-pool` / `data-plane-server` (planificateur de requêtes, filtres, montages, capacités moteur...). Exemple : un test de **compatibilité ascendante du format wire** (le format binaire échangé entre services sur le réseau) — un descripteur sérialisé *avant* l'ajout d'un champ doit toujours se relire, le champ absent valant `false` (jamais une capacité accordée par erreur).

```
// src/data-plane-router/crates/data-plane-core/src/capability.rs:453-469
#[test]
fn capabilities_payload_without_introspect_still_deserializes() {
    let mut payload = serde_json::to_value(EngineCapabilities::postgresql()).expect("des
cripteur serializes");
    payload.as_object_mut().expect("descriptor is a JSON object").remove("introspect").e
xpect("introspect was present before removal");
    let parsed: EngineCapabilities =
serde_json::from_value(payload).expect("old payload still deserializes");
    assert!(!parsed.introspect, "absent introspect defaults to false");
}
```

```
# en conteneur, comme le fait la CI :
$ docker run --rm -v "$PWD/src/data-plane-router":/src -w /src \
    rust:1-bookworm cargo test --workspace
    Compiling data-plane-core v... / data-plane-pool v... / data-plane-server v...
test result: ok. <N> passed; 0 failed; ... finished in ...s
# le workspace realtime se teste pareil : make rust-realtime-test
```

Lancer un seul test : `cargo test -p data-plane-core capabilities_payload`

SDK TYPESCRIPT — NODE:TEST

Le SDK de référence (`sdk/js/`) a dix suites `*.test.mjs` lancées par le lanceur natif de Node (`node --test` , pas Jest). Le transport `fetch` est *mocké*, donc aucune requête réseau. La suite de durcissement HTTP prouve par exemple : le `retry` (réessai) avec *back-off* (délai croissant entre deux tentatives) sur les requêtes idempotentes (rejouables sans effet de bord), l'**absence** de `retry` sur un POST de création, des erreurs **typées** (`MiniBaasConflictError` , `MiniBaasTimeoutError` ...), et l'annulation par `AbortSignal` .

```
// sdk/js/tests/http-hardening.test.mjs – transport mocké via l'option `fetch`, zéro
réseau.
import test from 'node:test';
import assert from 'node:assert/strict';
import { createClient, MiniBaasConflictError, MiniBaasTimeoutError } from '../dist/
index.js';
// ... prouve : retry idempotent, pas de retry sur POST, erreurs typées (status+body),
timeout/abort.
```

Lancer un seul test : `node --test tests/http-hardening.test.mjs` (après `npm run build`).

2. Tests d'intégration / smoke — la vraie stack, en boîte noire (`make test-smoke`)

Ici on ne *mocke* plus rien : la stack est démarrée (`make up`), et chaque script tape de **vraies requêtes HTTP** à travers Kong, exactement comme un client. `make test-smoke` enchaîne 16 phases (c'est l'une des familles que `make tests` exécute dans sa matrice complète) :

Phase	Ce qu'elle vérifie	Phase	Ce qu'elle vérifie
1-2	smoke : la stack répond	9	opérations de stockage (fichiers)
3	accès base authentifié	10	mutations + requêtes complexes
4	isolation entre utilisateurs	11	WebSocket realtime
5	métadonnées de base	12	rate limiting (Kong)
6	méthodes HTTP (GET/POST/PATCH/DELETE)	13	préflight CORS
7	gestion des erreurs	14-15	MongoDB (MVP, en <code>.sh</code> + <code>.py</code>)
8	cycle de vie des tokens	16	flux d'auth de bout en bout

```
$ make test-smoke      # nécessite la stack up (make up) ; va via Kong → http://
localhost:8000
=== Running: scripts/test/phase/phase4-user-isolation-test.sh ===
  ✓ user A ne voit que ses lignes (owner_id scope)
  ✓ user A ne peut pas lire les lignes de user B (404/empty, pas 403 fuyant)
...
All test phases passed
```

La phase 4 est la démonstration directe de l'isolation multi-tenant décrite au §8.10 : deux utilisateurs réels, et l'un ne voit jamais les données de l'autre.

3. Preuve « stack live » rejouable — Newman / Postman

Une collection Postman est rejouée par **Newman** contre la stack réelle pour prouver les capacités d'une offre. Le rapport est un vrai artefact JUnit (`artifacts/test/postman-offers.xml` , **39 tests**), avec santé des plans, provisioning d'un tenant, frappe d'une clé `mbk_` , CRUD, auth... Le rapport JUnit committé (`postman-offers.xml`) est un run **vert** ; l'extrait ci-dessous illustre le comportement *fail-closed* (en cas de doute, on refuse plutôt que d'autoriser) — le cas reproduit quand le plan de contrôle est coupé :

```
testsuites "Grobase – Offer capability proof (live stack)" tests=39 time=11.989s
  ✓ 00 Health / Data plane direct – GET /v1/health (service=data-plane-router status=ok)
  ✓ 10 Provision / Mint API key – POST /v1/tenants/{id}/keys (returns a full mbk_ key)
  x 30 Data CRUD / Create table – POST /query/v1/{dbId}/schema/ddl
    expected 503 to be within 200..299
    body: {"error":"auth_verify_unavailable", ...}
```

Cet **échec** est en réalité une **bonne nouvelle de sécurité** : dans ce scénario, le plan de contrôle (qui vérifie la clé via `POST /v1/keys/verify`) n'était pas joignable. Le plan de données n'a donc **pas exécuté la requête** — il a renvoyé `503 auth_verify_unavailable` (*fail-closed*) plutôt que d'agir sans identité prouvée. C'est exactement la couture identité→exécution du §8.1 : pas de vérification d'identité ⇒ pas d'accès aux données.

Tout le nécessaire vit sous `infra/config/postman/` : deux collections (`grobase-offers.postman_collection.json` ci-dessus, et `grobase-edge.postman_collection.json` ci-dessous), un environnement local, et un dossier `corpus/` . Deux commandes les lancent :

```
make test-offers
# collection « offres » → artifacts/test/postman-offers-report.html (39 tests)
make test-edge      # corpus edge (1 381 vecteurs) → artifacts/test/edge-report.html
```

LE GROS MORCEAU : 1 381 CAS HOSTILES (`MAKE TEST-EDGE`)

C'est ici que vivent « les centaines de tests Postman ». La suite *edge* est **pilotée par les données** : un seul template de requête, rejoué une fois par vecteur d'un corpus de **1 381 entrées distinctes** (`infra/config/postman/corpus/edge-corpus.json`), réparties en **9 familles** de cas tordus — exactement les sujets de cette question :

Famille	Vecteurs	Ce qu'elle attaque
injection-security	214	SQLi, NoSQL \$ -opérateurs, traversée de chemin, templating
unicode-encoding	210	UTF-8 mal formé, homoglyphes, octets nuls, surrogates
capability-tier	200	dépassement des capacités/tier de l'offre
tenant-isolation	172	tentatives de lire/écrire chez un autre tenant
idempotency-concurrency	133	rejoues, courses, doublons
payload-limits	127	corps géants, profondeur d'imbrication
types-and-error-mapping	121	confusion de types, mauvais JSON
numeric-boundary	111	bornes entières, NaN, $\pm\infty$, débordements

Famille	Vecteurs	Ce qu'elle attaque
malformed-protocol	93	en-têtes/HTTP cassés

Le contrat n'est pas « tel code exact » (les moteurs choisissent légitimement des 4xx différents), mais trois **invariants de fiabilité** vérifiés sur **chaque** vecteur :

1. **jamais de 5xx** (`code < 500`) — un 500 sur une entrée hostile serait un *défaut* : l'erreur moteur aurait dû être traduite en un 4xx propre, pas faire planter la requête ;
2. **statut HTTP valide** (`200..499`) — sinon c'est un blocage / une connexion coupée ;
3. **aucune fuite** — le corps ne contient jamais de *stack trace*, de `/etc/passwd`, ni de marqueur d'usurpation inter-tenant (`attacker-owner` / `spoof-tenant`).

```
EDGE_SMOKE=1 make test-edge      # sous-ensemble représentatif de 108 vecteurs (12/
famille), tient dans le cache de 30 s
```

C'est la preuve « boîte noire » qui complète les tests unitaires d'entrée du §8.11-1 : ces derniers prouvent qu'une *fonction* rejette une entrée tordue ; l'edge corpus prouve que **toute la stack**, de Kong au moteur, encaisse 1 381 entrées hostiles sans jamais cracher ni fuir.

4. Conformité moteur — `make conformance` (le test « engine-agnostic »)

Le projet promet que les **8 moteurs** (postgres, mysql, mongo, mssql, sqlite, redis, http, dynamodb) se comportent pareil. La batterie `engine-conformance` (crate Rust dédiée) rejoue **la même suite d'opérations** contre chaque moteur vivant. Une correction qui marche pour Postgres mais casse les sept autres n'est pas finie.

```
make conformance                # = scripts/verify/m27-conformance.sh, tous les moteurs
vivants
make conformance-postgresql     # un seul moteur
```

5. Parité shadow TS ↔ Rust — `make parity NEW=<url>`

C'est le **garde-fou de la migration**. Avant de basculer du chemin TypeScript (historique) vers le chemin Rust, on envoie des requêtes identiques aux deux et on compare les réponses. Tant que la parité n'est pas prouvée, on ne supprime rien (discipline *shadow* → *parité* → *cutover* → *delete*).

```
make parity NEW=http://localhost:8000    # émet un verdict ; sans NEW= la cible sort
en erreur (exit 1)
```

6. Les gates de jalon — l'unité de « fini » (148 scripts `mNN-*.sh`)

Chaque nouveauté arrive derrière un **gate numéroté** auto-suffisant : le script démarre un postgres neuf, applique les **vraies migrations SQL**, exerce la feature, **et** vérifie qu'avec le flag OFF le comportement reste byte-parité avec l'édition OSS. Un gate qui « passe à vide » n'est pas un gate.

```
bash scripts/verify/m104-audit-chain.sh    # un gate précis
bash scripts/verify/run-gate-battery.sh --enterprise  # la batterie nuit (CI)
```

Vraie sortie d'un gate (ABAC), reproductible via `bash scripts/verify/m136-abac-conditions.sh` — on y voit que le flag OFF redonne la parité, et que ON applique bien les conditions :

```
[M136] 4/4 conditions OFF=parity; ON gates allow/deny; conditional deny skip
✓ conditions OFF ⇒ ALLOW (ignore ip_cidr, 007 parity)
✓ conditions ON, ip 8.8.8.8 OUT of cidr ⇒ DENY
✓ conditions ON, ip 10.1.2.3 IN cidr ⇒ ALLOW
✓ conditional DENY applicable (ip inside) ⇒ DENY wins
[M136] PASS — stored conditions evaluate (ip_cidr); flag OFF=parity; conditional deny skip
```

7. Benchmarks — la perf est chiffrée, pas adjectivée (`make bench-*`)

Aucune affirmation de performance sans artefact reproductible. Les benchmarks écrivent des JSON dans `artifacts/bench/` (gitignorés : non committés dans ce dépôt autonome — on les régénère). La **forme** de sortie d'un run de charge CRUD, reproductible via `make bench-load PACKAGE=essential WORKLOAD=crud` :

```
$ make bench-load PACKAGE=essential WORKLOAD=crud
{ "rate_target": ..., "rps_achieved": ..., "err_pct": 0, "server_errors": 0,
  "ops": {
    "list": { "med": ..., "p95": ..., "p99": ... }, // lecture : p95 ~2 ms
    "insert": { "med": ..., "p95": ... },
    "update": { "med": ..., "p95": ... },
    "delete": { "med": ..., "p95": ... } } } // (latences en millisecondes)
```

C'est ce qui appuie l'argument « lecture p95 ~2 ms, la latence d'écriture est l'ennemi nommé » : la lecture chaude ~2 ms est corroborée par `cost-analysis.md` (« 5× faster, 8 ms vs 40 ms/req »), et `make bench-load` régénère les chiffres exacts.

8. Sécurité & chaîne d'approvisionnement — `make audit-deps` + scanners

`make audit-deps` lance **cargo-audit** (CVE des crates Rust) et **govulncheck** (CVE Go). À côté, `scripts/security/run-security-scans.sh` produit des rapports dans `artifacts/security/` : **Semgrep** (SAST TypeScript/NestJS/Docker), **npm audit** (SCA des lockfiles), **Trivy** (images + filesystem) et **TruffleHog** (aucun secret commité — historique git + arbre de travail). La CI lance aussi **shellcheck** sur tous les scripts.

```
make audit-deps                                # cargo-audit + govulncheck
bash scripts/security/run-security-scans.sh      # Semgrep + npm audit + Trivy +
TruffleHog
# rapports : artifacts/security/{semgrep.json, npm-audit.txt, trivy/trivy-fs.json,
trufflehog.json}
```

9. La CI — tout cela tourne tout seul (GitHub Actions)

Rien de tout ça ne repose sur la bonne volonté : `.github/workflows/ci.yml` rejoue ces familles à chaque push, avec un **chemin rapide par PR** et une **batterie complète la nuit**. Les jobs :

- **shellcheck** — lint de tous les scripts ;
- **unit-tests** (matrice `go · rust-data-plane · rust-realtime · nestjs`) — les tests de la famille 1 ;
- **integration-tests** — les phases (`make test-smoke`) + le gate passerelle m102, sur une stack montée ;
- **cloud-gates / gates-full** — les gates de jalon (sous-ensemble par PR, batterie entreprise la nuit) ;
- **sdk-tests** (matrice `python · kotlin · dart · swift`) + **offers** (compile chaque forme produit : nano, one, éditions) ;
- **docker-build / infra-build / app-publish** — construit et publie les images.

Autrement dit : un test qui passe en local passe aussi en CI, **avec la même commande** — c'est ce qui rend le « ça marche » vérifiable plutôt que déclaratif.

En une phrase

Chaque couche a son filet : logique pure en unitaire, vrai HTTP en intégration, égalité des 8 moteurs en conformité, ancien vs nouveau chemin en parité, chaque feature derrière un gate « OFF = parité », la perf en chiffres et la sécurité en scanners — le tout rejoué par la CI.

8.12 — HTTPS / TLS : comment la liaison navigateur ↔ back-end est-elle réellement chiffrée ?

Une question quasi systématique en soutenance — et elle mérite mieux qu'un « on est en HTTPS ». Voici la réponse complète, avec le vrai code de génération et de terminaison TLS.

Q. Concrètement, qu'est-ce que le HTTPS apporte, et où est-il en place ?

Le HTTPS, c'est du HTTP transporté **sur TLS** : tout ce qui circule entre le navigateur et mon back-end est chiffré, ce qui apporte confidentialité, intégrité et authenticité (détaillé au **chapitre 5 — Sécurité**). Dans le projet, **le seul port public est le WAF nginx, en TLS 1.2/1.3** (`infra/docker/services/waf/conf/nginx.conf`) ; tout le reste vit sur un réseau Docker privé.

Q. Certificat, autorité de certification, terminaison TLS, production : où est tout ça ?

La chaîne complète — CA locale RSA 4096 qui signe un certificat serveur RSA 2048 (SAN `localhost / 127.0.0.1 /...`), scripts `generate-localhost-cert.sh` et `trust-localhost-cert.sh`, **terminaison TLS au WAF** puis relais vers Kong sur réseau privé, **HSTS**, et en production le remplacement de la CA locale par **Let's Encrypt** + le chiffrement jusqu'aux bases via `SECURITY_MODE=max` → `verify-full` — est détaillée à son chapitre d'origine (voir **chapitre 5 — Sécurité**). Le seul fait propre à cette question : le trajet est chiffré de bout en bout, navigateur → WAF puis back-end → base.

8.13 — Authentification vs autorisation : quelle différence, et où vivent-elles ?

Q. Le jury demande souvent la nuance — comment la fais-tu dans le code ?

Ce sont deux questions distinctes, que je traite à deux endroits différents :

- **L'authentification, c'est « qui es-tu ? »** Elle se règle **au bord**, une fois pour toutes : GoTrue vérifie l'identité (bcrypt sur le mot de passe) et émet un **JWT** ; Kong valide la signature de ce JWT (HS256, algorithme figé — pas de substitution possible) à chaque requête, puis injecte l'identité en en-têtes internes (`X-User-Id` , `X-User-Role` ...). Pour les appels machine-à-machine, c'est une **clé d'API**, vérifiée par empreinte et jamais stockée en clair.
- **L'autorisation, c'est « as-tu le droit de faire ça sur cette ressource ? »** Elle se règle **plus en profondeur**, en plusieurs couches : `RolesGuard` NestJS (RBAC), puis l'**ABAC** SQL en *deny-first*, et enfin la base elle-même avec la **RLS forcée** qui a le dernier mot — le détail de chaque couche est au **chapitre 5 — Sécurité**.

En une phrase de soutenance : « *L'authentification prouve qui appelle ; l'autorisation décide ce qu'il peut toucher.* »

8.14 — Pourquoi NestJS plutôt qu'Express ?

Q. Express aurait suffi, non ? Pourquoi avoir pris NestJS ?

Express, c'est un routeur minimaliste : rapide à démarrer, mais on réécrit vite la même tuyauterie (validation, authentification, gestion d'erreurs) **à la main, route par route**. **NestJS** ajoute une **structure** par-dessus Express — modules, injection de dépendances, *guards*, *pipes*, *filters* — et c'est cette structure qui porte ma sécurité **de façon transversale**, pas dupliquée :

- un `AuthGuard` (`auth.guard.ts`) posé en décorateur `@UseGuards()` , au lieu d'un `if (!req.user)` recopié dans chaque contrôleur ;
- un `RolesGuard` (`roles.guard.ts`) pour un RBAC **déclaratif** (`@Roles('admin')`) ;
- un `ValidationPipe` **global** (`validation.pipe.ts`) qui valide et nettoie chaque payload (whitelist) une fois pour toutes ;
- un **filtre d'exception global** (`all-exceptions.filter.ts`) qui normalise toutes les erreurs en une seule forme JSON et **masque les 5xx** (un bug serveur ne fuit jamais sa stack au client).

Autrement dit : avec Express, j'aurais une sécurité « à la main, en espérant ne rien oublier » ; avec NestJS, elle est **centralisée, déclarative et testable** — chaque protection au même endroit, réutilisée par tous les contrôleurs (le détail de ces guards/pipes/filters est au **chapitre 5 — Sécurité**).

8.15 — Que se passe-t-il si une couche — ou un service — tombe ?

Q. Et la résilience ? Si le WAF, Kong, ou un service interne lâche ?

C'est tout l'intérêt de la **défense en profondeur** (détaillée au **chapitre 5 — Sécurité**) : aucune couche n'est censée suffire seule, donc la chute de l'une **n'ouvre pas** la porte.

- **Le superpositionnement des couches est canonique au chapitre 5** : WAF contourné → Kong tient (clé d'API, JWT, rate-limit) → et si Kong laissait passer, la **RLS PostgreSQL forcée** (migration 065) a le dernier mot. Auth en échec → **fail-closed** (401, jamais d'accès par défaut ; gate m37).

- Ce que cette question ajoute, c'est la dégradation : une dépendance non critique qui tombe dégrade le service au lieu de le faire planter. Concrètement : si le plan temps réel est injoignable, l'écriture réussit quand même et seule la notification est sautée ([realtime-publisher.service.ts:151-159](#)) ; si le service de capacités ne répond pas, le schéma est tout de même servi (les capacités sont optionnelles, typées `RustEngineCapabilities | null` dans [schema.service.ts](#)). Ces chemins « dépendance absente » sont exercés par leurs tests respectifs — ce ne sont pas des hypothèses.
- Au niveau infra, l'overlay de production ([docker-compose.prod.yml](#)) ajoute des *restart policies* et retire les ports directs des bases ; un container qui meurt redémarre tout seul.

8.16 — Comment gères-tu l'évolution du schéma de base de données ?

Q. Les migrations : comment ça marche, et pourquoi c'est important ?

Par des **migrations SQL versionnées**, numérotées et rejouables, dans `scripts/migrations/postgresql/` — de `001_initial_schema.sql` jusqu'à `076_login_escrow.sql`. Chaque changement de schéma est un fichier numéroté qui entre dans Git : il est donc **relu en code review**, **rejouable à l'identique** sur n'importe quel environnement, et **traçable**. Je les applique par des cibles Make dédiées :

```
make migrate          # applique les migrations PostgreSQL en attente
make migrate-status   # montre les versions déjà appliquées
make migrate-all      # PG + Mongo + MySQL d'un coup
```

L'intérêt, pour un correcteur : l'état de la base n'est jamais « bricolé à la main » sur un serveur. Il se **reconstruit depuis zéro** à partir des fichiers versionnés, donc développement, CI et production partagent le **même schéma**. Les bases Mongo et MySQL suivent la même logique (`make migrate-mongo` / `make migrate-mysql`).

8.17 — Comment les trois plans (TypeScript, Go, Rust) communiquent-ils ?

La 8.1 disait *qui fait quoi* ; voici comment les trois plans se parlent — les endpoints, le format d'échange, l'identité, et l'asynchrone.

Q. Concrètement, par quoi passent les données entre les trois langages ?

Par **HTTP + JSON**, point. Il n'y a **aucun appel « en mémoire » entre langages** : chaque plan est un **service réseau** indépendant, dans son propre container. La langue commune, c'est donc l'**HTTP**, et le format d'échange, c'est le **JSON**. L'image que j'utilise : trois bureaux qui ne partagent pas de tiroir — ils s'écrivent par **courrier recommandé** (une requête HTTP, signée, avec un contenu JSON).

Exemple réel : quand `RUST_DATA_PLANE_FORWARD=1`, le query-router NestJS n'exécute pas la requête lui-même — il **POST une enveloppe JSON** `{ identity, mount, operation }` vers le data plane Rust ([rust-data-plane.proxy.ts](#)) :

```
// src/apps/query-router/src/proxy/rust-data-plane.proxy.ts:288-306 (simplifié –
// execute() délègue à postJson('/v1/query', ...))
async execute(context, resource, op, opts): Promise<QueryResult> {
  const envelope = { identity, mount, operation }; // contrat JSON partagé
  const { data } = await firstValueFrom( // Observable rxjs →
    Promise
      .then(() => this.http.post(`${this.url}/v1/query`, envelope, { headers }),
      () => {});
  return this.normalizeResult(data); // re-normalise vers la
  // forme legacy
}
```

Détail qui compte : le JSON est en **snake_case sur le fil** (`tenant_id` , `affected_rows` ...) pour coller **exactement** aux `struct` Rust — le contrat de données est partagé entre TS et Rust, pas réinventé de chaque côté.

Q. C'est quoi un « endpoint API » ici, et qui appelle qui ?

Un **endpoint**, c'est une route HTTP qu'un service expose. On en a de deux sortes :

- **Publics** (exposés via WAF → Kong) : `/auth/v1` , `/rest/v1` ... — c'est ce que le front consomme.
- **Internes** (réseau Docker privé, jamais exposés) : le **data plane Rust** expose `/v1/query` , `/v1/schema` , `/v1/transactions` , `/v1/capabilities` ; le **control plane Go** expose `/v1/keys/verify` , `/v1/tenants/...` .

Le chemin porteur d'une requête de données : le query-router (TS) demande d'abord au **Go** « cette clé d'API correspond à quelle identité ? » (`POST /v1/keys/verify`), puis transmet la requête au **Rust** (`POST /v1/query`), qui l'exécute et l'owner-scope (voir **chapitre 4**). Chaque langage fait le métier pour lequel il est le meilleur, et se passe le relais par HTTP.

Q. Et l'identité / la confiance dans ces appels internes ?

Un appel interne n'est pas « ouvert » sous prétexte qu'il vient d'un autre container. Chaque requête service-à-service est **signée en HMAC** ([service-auth.ts](#)) :

```
X-Service-Auth: v1.<ts>.<hmac-sha256(token,
  "<ts>\n<METHOD>\n<PATH>\n<sha256hex(body)>">
```

Le point fort : **les trois langages signent à l'octet près**. Les mêmes vecteurs de test (« golden vectors ») valident la signature en Go ([serviceauth/token.go](#) + `token_test.go`), en Rust ([service_auth.rs](#)) et en TypeScript. Qu'un appel parte de TS vers Go ou de TS vers Rust, l'authentification est **identique**. À cela s'ajoute l'**enveloppe d'identité signée** (`source: 'signed_envelope'`), qui transporte tenant/user/rôle de façon infalsifiable.

Q. Et l'asynchrone (les « promises ») dans tout ça ?

Chaque langage a son modèle, mais l'idée est **la même partout** : pendant qu'on attend le réseau ou le disque, on **ne bloque pas** le thread — on libère la main pour traiter d'autres requêtes.

- **TypeScript** — `async` / `await` + `Promise` . Une méthode déclarée `async execute(...): Promise<...>` rend la main pendant l'appel réseau. L'appel HTTP via `@nestjs/axios` renvoie un `Observable rxjs`, qu'on convertit en `Promise` avec `firstValueFrom` . Exemple concret de maîtrise de

l'async : `getCapabilitiesCached()` partage **une seule Promise en vol** entre les appels concurrents — si 10 requêtes arrivent en même temps sur un cache froid, il n'y a **qu'un seul** appel réseau, pas dix (anti *thundering herd*).

- **Go** — chaque requête HTTP est servie dans sa **propre goroutine** (le serveur `net/http` le fait automatiquement), et on propage un `context.Context` pour les délais et l'annulation. Exemple : `func (b *builderAPI) createMount(w http.ResponseWriter, r *http.Request) (builder_mounts.go:28 ; route enregistrée dans builder.go:88 )`.
- **Rust** — `async / await` sur le runtime `tokio` (`#[tokio::main] async fn main()`, `main.rs` ; `pub async fn verify_key(...)`, `auth.rs`). C'est aussi là que vit l'optimisation majeure : le plan Rust garde des **pools de connexions longue durée** par mount, alors que l'ancienne version TS ouvrait un client par appel.

8.18 — Les « injectables » NestJS : c'est quoi, et comment ça marche ?

Q. Tu parles d'injectables et d'injection de dépendances — explique simplement.

Un service annoté `@Injectable()` est un **fournisseur** (*provider*) que NestJS sait **construire et fournir tout seul** là où on en a besoin. Je n'écris **jamais** `new MonService()` à la main : je déclare le service dans un module, je le **demande dans un constructeur**, et le conteneur d'**injection de dépendances** (DI) s'occupe de le fabriquer et de le brancher.

Exemple réel — le proxy vers Rust déclare ses besoins dans son constructeur :

```
// src/apps/query-router/src/proxy/rust-data-plane.proxy.ts
@Injectable()
export class RustDataPlaneProxy {
  constructor(config: ConfigService, private readonly http: HttpService) { /* ... */ }
}
```

NestJS voit que ce service a besoin d'un `ConfigService` et d'un `HttpService`, et les lui **passe automatiquement** à la construction (*injection par constructeur*). Il suffit de l'avoir déclaré une fois dans le module :

```
// src/apps/query-router/src/query/query.module.ts:50,54
@Module({
  providers: [QueryService, /* ... */ RustDataPlaneProxy],
})
export class QueryModule {}
```

Q. Pourquoi c'est utile, concrètement ?

Trois bénéfices concrets, que j'utilise au quotidien :

- **Testabilité** — en test, j'injecte un **faux** `ConfigService` et un **faux** `HttpService` au lieu des vrais : c'est exactement ce que font mes specs (`rust-data-plane.proxy.spec.ts`). Pas besoin de démarrer toute l'application pour tester une classe.
- **Singletons** — NestJS ne crée **qu'un seul exemplaire** partagé du provider. C'est pour ça que le cache de capacités du proxy est unique pour toute l'app (un seul cache, un seul fetch en vol).

- **Découplage** — un service ne sait pas *comment* ses dépendances sont fabriquées ; il déclare juste ce dont il a besoin. On peut remplacer une implémentation (ex. un faux client en test) sans toucher au code qui l'utilise.

Et surtout : c'est le **même mécanisme** qui fait fonctionner les `Guards`, `Pipes` et `Filters` de sécurité (voir **chapitre 5** et 8.14) — des injectables appliqués de façon **transversale** par décorateur, d'où une sécurité centralisée plutôt que recopiée dans chaque contrôleur.

8.19 — Les « guardrails » de chaque langage : qu'est-ce qui me protège en TS, Go, Rust ?

Q. Pourquoi ces trois langages précisément ? Qu'apporte chacun en garde-fous ?

C'est une des vraies raisons du choix à trois langages : pour chaque métier, je prends le langage dont les **garde-fous** correspondent au **risque**. Plus le code est dangereux (toucher les bases, gérer la mémoire sous charge), plus le langage doit être strict.

TypeScript — le plan applicatif (réception, validation, orchestration). Garde-fous à deux moments :

- **À la compilation** : typage statique strict — `tsconfig.json` active `strict`, `strictNullChecks`, `noImplicitAny` (`tsconfig.json`), donc `tsc --noEmit` refuse de compiler du code mal typé (c'est une étape de `make nestjs-ci`), + ESLint.
- **À l'exécution** : attention, les types TS sont **effacés au runtime** (JavaScript ne les connaît pas). Le vrai garde-fou face à une entrée hostile, c'est donc la **validation au runtime** — `class-validator` / `Zod` avec `whitelist` (chapitre 5) — *en plus* des types.

Go — le plan de contrôle (tenants, clés, facturation). Ses garde-fous : un compilateur strict, une concurrence sûre et un binaire sans dépendances système :

- Typage statique + **ramasse-miettes (GC)** : pas de gestion mémoire manuelle, donc pas de *use-after-free* à la main.
- Concurrency plus sûre : goroutines + channels, et un **détecteur de data races** intégré (`go test -race`).
- **Binaire statique** (`CGO_ENABLED=0`) : aucune dépendance système à traîner.
- Scan de vulnérabilités : `govulncheck` via `make audit-deps`.
- Garde-fou métier : le plan Go **refuse de démarrer** sur un token de service vide ou faible (*fail-fast* au boot).

Rust — le plan de données (exécution des requêtes, pools, owner-scoping — voir chapitre 4). Les garde-fous **les plus stricts, prouvés à la compilation** :

- **Ownership / borrow checker** : sécurité mémoire **garantie par le compilateur**, sans GC — pas de pointeur nul, de *use-after-free* ni de *data race* (le code fautif ne compile pas).
- **Pas de null** : `Option<T>` et `Result<T, E>` mettent l'absence et l'erreur **dans le type** (≈ 85 occurrences rien que dans `data-plane-core`), et le `match` exhaustif force à traiter **tous** les cas.
- Scan CVE : `cargo-audit` via `make audit-deps`.

C'est précisément pour ça que **le plan de données est en Rust** : c'est lui qui manipule les bases et isole les clients à chaque requête — le risque y est maximal, donc on y met le langage aux garde-fous les plus forts.

8.20 — Comment un binaire (Go, Rust) « fait du web » ? Et WebAssembly là-dedans ?

Q. Un programme compilé, ça sert des requêtes HTTP comment ?

Node/TypeScript est **interprété/JIT** et a besoin du **runtime Node** pour tourner. Go et Rust, eux, **compilent en exécutable natif qui embarque lui-même un serveur HTTP** : pas d'interpréteur, pas de machine virtuelle, le binaire est le serveur.

- **Go** : le serveur `net/http` est inclus dans le binaire ; chaque requête est servie dans une goroutine. On compile en **statique** et on livre dans une image **distroless nonroot** (sans shell ni libc) :

```
dockerfile # src/control-plane/Dockerfile CGO_ENABLED=0 GOOS=linux go build -trimpath
-ldflags="-s -w" -o /out/... FROM gcr.io/distroless/static-debian12:nonroot AS runtime
```

- **Rust** : serveur HTTP `axum` sur le runtime asynchrone `tokio` ([data-plane-server/Cargo.toml](#)) ; l'édition `nano` se livre dans une image **FROM scratch** — littéralement « l'image EST le binaire » ([Dockerfile.nano](#)) —, avec un profil release `strip = true` + LTO ([Cargo.toml](#)) pour un exécutable minuscule (≈ 5 Mo).

L'intérêt concret, pour un jury : **démarrage quasi instantané, empreinte mémoire minuscule, surface d'attaque réduite** (pas d'OS ni de shell dans l'image), et **un seul fichier** à déployer.

Q. Et WebAssembly (WASM) ? Vous l'utilisez ?

Non, pas dans le back-end — et c'est un choix assumé, pas un oubli. WASM sert surtout à deux choses : (a) faire tourner du code compilé **dans le navigateur**, et (b) côté serveur, **mettre du code non fiable dans un bac à sable portable**. Or :

- **(a) ne nous concerne pas** : mes plans Go/Rust tournent **côté serveur**, dans des containers — ils n'ont pas besoin d'être embarqués dans un navigateur. Le natif est plus simple et plus rapide pour ça.
- **(b) le besoin existe** (exécuter les **Edge Functions** — du code que les utilisateurs déposent eux-mêmes sur la plateforme pour l'exécuter côté serveur — sans qu'elles cassent la plateforme), mais je le résous **autrement** : la `functions-runtime` utilise **Deno** — un **isolat V8 frais par invocation** (un bac à sable jetable : un mini-environnement JavaScript neuf, cloisonné, recréé à chaque appel, Deno Worker), avec un **jeu de permissions minimal** (pas d'accès fichier, pas d'env, réseau limité à une allow-list) ([functions-runtime/Dockerfile](#)), le tout par-dessus l'isolation du container.

Honnêtement, **où WASM pourrait entrer un jour** : pour imposer des **plafonds CPU/RAM durs** par fonction utilisateur (via wasmtime/WASI), là où l'isolat Deno V8 actuel ne pose pas de quota CPU/RAM dur. La piste d'évolution effectivement **notée dans ce Dockerfile** n'est cependant pas WASM, mais l'exécution du worker comme un **processus enfant deno run séparé sous son propre cgroup** (`memory.max` / `cpu.max`, voir [functions-runtime/Dockerfile](#)). Dans les deux cas, ce n'est pas nécessaire aujourd'hui.

8.21 — Docker : comment chaque langage est conteneurisé, et par quel protocole les services se parlent

Q. Chaque service (TS, Go, Rust) tourne comment, et comment se trouvent-ils ?

Chaque plan est **sa propre image, son propre container**, démarré par docker-compose. Ils ne se cherchent pas par une IP codée en dur : Docker fournit un **DNS interne** où le **nom du service** est l'hôte. Le query-router NestJS appelle donc ses dépendances internes — `data-plane-router-rust:4011` ,

`adapter-registry-go:3021`, `permission-engine:3050` — par leur **nom**. S'y ajoutent les **réseaux** isolés (le public ne voit que le WAF, voir **chapitre 5**), les **healthchecks** (44 services), les **volumes**, `depends_on` et les **overlays/profils** (les éditions) — déjà décrits aux 8.3 et 8.4 ; chaque langage profite des mêmes briques, sans rien de spécifique.

Q. Ils communiquent en HTTP, ou par d'autres protocoles ?

Les deux — selon l'interlocuteur :

- **Entre services** (TS ↔ Go ↔ Rust) : **HTTP + JSON** (REST), signé en HMAC (cf. 8.17). C'est le « langage commun » des trois langages.
- **Vers les bases de données** : chaque moteur parle son **protocole natif**, via son **vrai pilote** — pas du HTTP. Le plan de données Rust embarque un pilote par moteur dans `data-plane-pool` : `tokio-postgres` (protocole PostgreSQL), `mongodb` (wire protocol Mongo), `mysql_async` (MySQL/MariaDB), `redis` (RESP), `tiberius` (TDS / SQL Server), `rusqlite` (SQLite), `aws-sdk-dynamodb`. Bref : **HTTP pour parler entre nous, protocole natif pour parler à la base** — chacun au plus efficace.

8.22 — OLTP vs OLAP : la différence, et comment je « vois » dans l'OLTP

Q. C'est quoi la différence, et nous, on est quoi ?

- **OLTP** (*Online Transaction Processing*) = beaucoup de **petites** opérations transactionnelles à **faible latence** : créer une ligne, lire une fiche, changer un statut. C'est le quotidien d'une appli.
- **OLAP** (*Online Analytical Processing*) = de **grosses agrégations** analytiques sur énormément de lignes : « chiffre d'affaires par mois et par région ». Peu de requêtes, mais lourdes.

Nous sommes d'abord un moteur OLTP : le plan de données est optimisé pour le CRUD à faible latence, et c'est **prouvé** par les gates `m25-oltp-matrix` et `m26-oltp-completeness`. Pour les besoins **OLAP**, j'ai (a) l'opération `aggregate` (`group_by` + fonctions d'agrégat) exposée uniformément, (b) l'**analytics-service** (`src/apps/analytics-service`) pour les événements, et (c) la possibilité de **fédérer** plusieurs sources via **Trino** (`infra/docker/services/trino`) quand il faut croiser des bases hétérogènes.

Q. Comment « voir » dans l'OLTP — inspecter et observer la base ?

Deux usages bien distincts :

- **Inspecter le contenu / le schéma** : `pg-meta` (route interne `/meta/v1`, restreinte aux IP privées) et **Studio** (UI type Supabase) — voir les tables, les lignes, les politiques RLS.
- **Observer la santé / la charge** : la stack **Prometheus + Grafana + Loki + Promtail** (métriques, dashboards, logs corrélés par `X-Request-ID`), plus le **metering** par tenant (table `tenant_usage`) qui compte requêtes / lignes / stockage.

8.23 — Comment être sûr que toutes les connexions aux bases sont fiables ?

Q. Avec 8 moteurs et du multi-tenant, comment garantir la fiabilité des connexions ?

Plusieurs mécanismes qui se **cumulent** :

- **Pools de connexions** par mount (côté Rust, via `deadpool_postgres` dans le module `postgres/` — `pool.rs` + `adapter.rs`, l'ancien fichier `postgres.rs` ayant été scindé en répertoire) : on **réutilise** des connexions au lieu d'en ouvrir une neuve à chaque requête (l'ancienne faiblesse du chemin TS). La politique est explicite — `min`, `max`, `idle_ttl_ms`, `max_lifetime_ms` — donc une connexion trop vieille est **recyclée**, pas gardée indéfiniment.
- **Healthchecks** : 44 services du compose déclarent un `healthcheck`, et chaque moteur implémente un `health_check() -> EngineHealth` (`ports.rs`) — une base qui ne répond plus est **détectée**, pas découverte au pire moment.
- **TLS vérifié** en production (`SECURITY_MODE=max` → `verify-full`, voir **chapitre 5**) : on ne se connecte pas à une base dont le certificat ne se valide pas.
- **Timeouts + dégradation** : un appel au data plane a un timeout (`RUST_DATA_PLANE_TIMEOUT_MS`) ; s'il échoue, l'erreur est mappée proprement (502) plutôt que de « pendre », et les dépendances non critiques dégradent au lieu de planter (cf. 8.15).
- **Montée en charge** : pour 10 000 tenants, un **pooler** (Supervisor, `docker-compose.pooler.yml`) peut s'intercaler ; `SHARE_POOLS` fait déjà tenir des milliers de tenants sur un même pool (isolation **par requête**, pas par connexion — cf. 8.10c et **chapitre 4**).
- **Preuve** : la conformité (`make conformance`, m27) et m25/m26 exercent les **8 moteurs** de la même façon — la fiabilité est **testée**, pas supposée.

8.24 — Le front sait-il si c'est MongoDB, PostgreSQL ou MariaDB ? Et comment demande-t-on un CRUD ?

Q. Le front doit-il connaître le moteur derrière ? Est-ce normalisé ?

Non — et c'est tout l'intérêt du produit. Le front (via le SDK) **ne sait pas**, et n'a pas besoin de savoir, quel moteur exécute. Tout est **normalisé** :

- **Même forme de résultat** quel que soit le moteur : le data plane renvoie toujours `QueryResult { rows, rowCount }` (`normalizeResult` dans `rust-data-plane.proxy.ts`).
- **Schéma normalisé** : chaque colonne porte un `normalized_type` commun (`text | integer | float | decimal | boolean | date | datetime | json | uuid | enum | array | objectid`), donc une date Mongo et une date Postgres arrivent sous le **même type** côté front.
- **Capacités en direct** : ce qu'un moteur sait faire vient de `/v1/capabilities` (`EngineCapabilities`, la **source de vérité** contre laquelle le SDK est typé) — pas un tableau écrit à la main.

Q. Donc on ne code pas en dur les besoins par moteur ?

Non. Le schéma est **introspecté en direct** (`POST /v1/schema`) et les capacités **lues en direct** — rien n'est figé par moteur dans le front. C'est le principe **engine-agnostic by construction** : une correction qui marche pour Postgres mais casse les 7 autres moteurs **n'est pas considérée comme finie** (vérifié par la conformité m27).

Q. Concrètement, comment demande-t-on un get / set / fetch / delete ?

Par un **vocabulaire d'opérations unique**, le même pour les 8 moteurs — `AdapterOp` dans [adapter.contract.ts](#) :

```
list | get | insert | update | delete | upsert | aggregate | batch
```

Le front envoie (via le SDK, [sdks/js/src/domains/query.ts](#)) une opération `{ op, resource, data?, filter?, sort?, limit?, offset? }`. La correspondance avec ce que tu appelles get/set/fetch/delete :

- **fetch / lister** → `op: 'list'` (+ `filter`, `sort`, `limit`) ; **lire un** → `op: 'get'` ;
- **set / créer** → `op: 'insert'` (ou `upsert`) ; **modifier** → `op: 'update'` (+ `filter`) ;
- **supprimer** → `op: 'delete'` (+ `filter`).

C'est **le même appel** quel que soit le moteur : c'est le data plane qui traduit `op: 'get'` en `SELECT ... WHERE id = $1` pour Postgres, en `find()` pour Mongo, en commande RESP pour Redis, etc. Le front parle **CRUD** ; le moteur parle **son dialecte** — et la traduction est mon travail, pas le sien.

En une phrase : *le front parle un seul langage CRUD (`list/get/insert/update/delete/upsert/aggregate`)*

et reçoit une seule forme de données normalisée ; savoir que derrière c'est Mongo, Postgres ou MariaDB, c'est le travail du data plane, jamais celui du front.